

13.3 Tries

The pattern-matching algorithms presented in Section 13.2 speed up the search in a text by preprocessing the pattern (to compute the *last* function in the Boyer-Moore algorithm or the *failure* function in the Knuth-Morris-Pratt algorithm). In this section, we take a complementary approach, namely, we present string searching algorithms that preprocess the text, rather than the pattern. This approach is suitable for applications in which many queries are performed on a fixed text, so that the initial cost of preprocessing the text is compensated by a speedup in each subsequent query (for example, a website that offers pattern matching in Shakespeare's *Hamlet* or a search engine that offers Web pages containing the term *Hamlet*).

A **trie** (pronounced “try”) is a tree-based data structure for storing strings in order to support fast pattern matching. The main application for tries is in information retrieval. Indeed, the name “trie” comes from the word “retrieval.” In an information retrieval application, such as a search for a certain DNA sequence in a genomic database, we are given a collection S of strings, all defined using the same alphabet. The primary query operations that tries support are pattern matching and **prefix matching**. The latter operation involves being given a string X , and looking for all the strings in S that begin with X .

13.3.1 Standard Tries

Let S be a set of s strings from alphabet Σ such that no string in S is a prefix of another string. A **standard trie** for S is an ordered tree T with the following properties (see Figure 13.7):

- Each node of T , except the root, is labeled with a character of Σ .
- The children of an internal node of T have distinct labels.
- T has s leaves, each associated with a string of S , such that the concatenation of the labels of the nodes on the path from the root to a leaf v of T yields the string of S associated with v .

Thus, a trie T represents the strings of S with paths from the root to the leaves of T . Note the importance of assuming that no string in S is a prefix of another string. This ensures that each string of S is uniquely associated with a leaf of T . (This is similar to the restriction for prefix codes with Huffman coding, as described in Section 13.4.) We can always satisfy this assumption by adding a special character that is not in the original alphabet Σ at the end of each string.

An internal node in a standard trie T can have anywhere between 1 and $|\Sigma|$ children. There is an edge going from the root r to one of its children for each character that is first in some string in the collection S . In addition, a path from the root of T to an internal node v at depth k corresponds to a k -character prefix

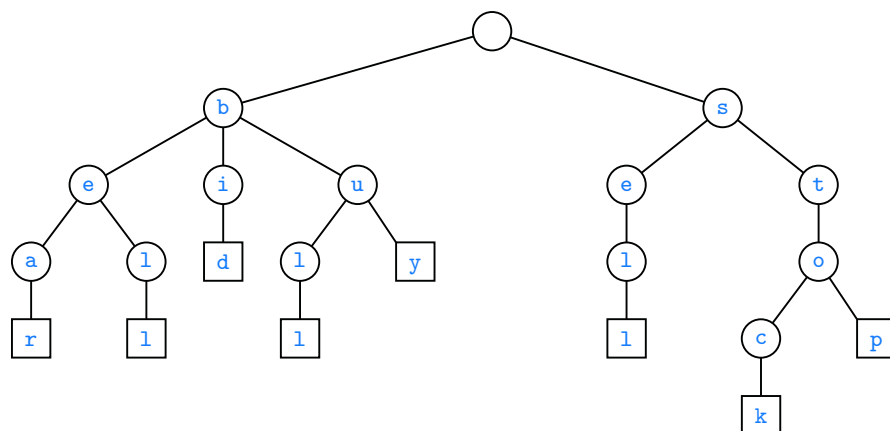


Figure 13.7: Standard trie for the strings {bear, bell, bid, bull, buy, sell, stock, stop}.

$X[0..k-1]$ of a string X of S . In fact, for each character c that can follow the prefix $X[0..k-1]$ in a string of the set S , there is a child of v labeled with character c . In this way, a trie concisely stores the common prefixes that exist among a set of strings.

As a special case, if there are only two characters in the alphabet, then the trie is essentially a binary tree, with some internal nodes possibly having only one child (that is, it may be an improper binary tree). In general, although it is possible that an internal node has up to $|\Sigma|$ children, in practice the average degree of such nodes is likely to be much smaller. For example, the trie shown in Figure 13.7 has several internal nodes with only one child. On larger data sets, the average degree of nodes is likely to get smaller at greater depths of the tree, because there may be fewer strings sharing the common prefix, and thus fewer continuations of that pattern. Furthermore, in many languages, there will be character combinations that are unlikely to naturally occur.

The following proposition provides some important structural properties of a standard trie:

Proposition 13.4: *A standard trie storing a collection S of s strings of total length n from an alphabet Σ has the following properties:*

- *The height of T is equal to the length of the longest string in S .*
- *Every internal node of T has at most $|\Sigma|$ children.*
- *T has s leaves.*
- *The number of nodes of T is at most $n + 1$.*

The worst case for the number of nodes of a trie occurs when no two strings share a common nonempty prefix; that is, except for the root, all internal nodes have one child.

A trie T for a set S of strings can be used to implement a set or map whose keys are the strings of S . Namely, we perform a search in T for a string X by tracing down from the root the path indicated by the characters in X . If this path can be traced and terminates at a leaf node, then we know X is a string in S . For example, in the trie in Figure 13.7, tracing the path for “bull” ends up at a leaf. If the path cannot be traced or the path can be traced but terminates at an internal node, then X is not a string in S . In the example in Figure 13.7, the path for “bet” cannot be traced and the path for “be” ends at an internal node. Neither such word is in the set S .

It is easy to see that the running time of the search for a string of length m is $O(m \cdot |\Sigma|)$, because we visit at most $m + 1$ nodes of T and we spend $O(|\Sigma|)$ time at each node determining the child having the subsequent character as a label. The $O(|\Sigma|)$ upper bound on the time to locate a child with a given label is achievable, even if the children of a node are unordered, since there are at most $|\Sigma|$ children. We can improve the time spent at a node to be $O(\log |\Sigma|)$ or expected $O(1)$, by mapping characters to children using a secondary search table or hash table at each node, or by using a direct lookup table of size $|\Sigma|$ at each node, if $|\Sigma|$ is sufficiently small (as is the case for DNA strings). For these reasons, we typically expect a search for a string of length m to run in $O(m)$ time.

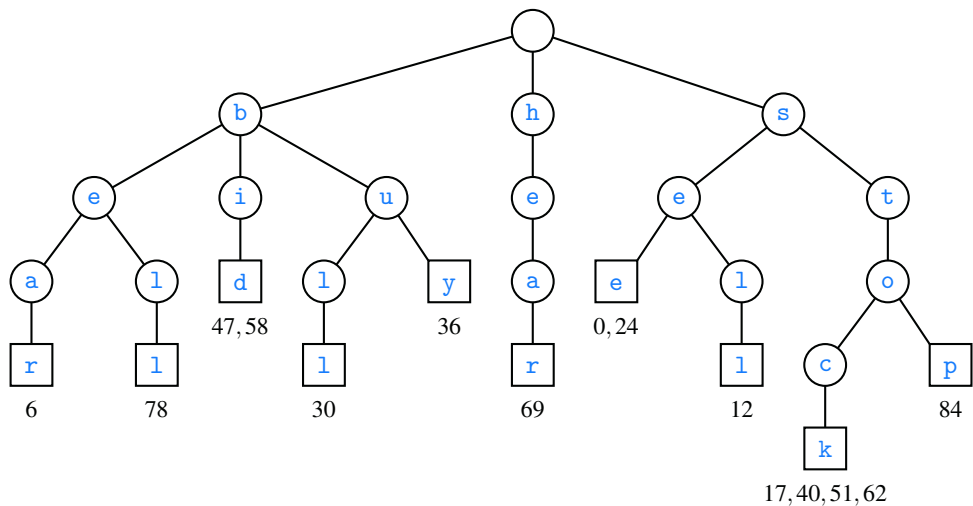
From the discussion above, it follows that we can use a trie to perform a special type of pattern matching, called **word matching**, where we want to determine whether a given pattern matches one of the words of the text exactly. Word matching differs from standard pattern matching because the pattern cannot match an arbitrary substring of the text—only one of its words. To accomplish this, each word of the original document must be added to the trie. (See Figure 13.8.) A simple extension of this scheme supports prefix-matching queries. However, arbitrary occurrences of the pattern in the text (for example, the pattern is a proper suffix of a word or spans two words) cannot be efficiently performed.

To construct a standard trie for a set S of strings, we can use an incremental algorithm that inserts the strings one at a time. Recall the assumption that no string of S is a prefix of another string. To insert a string X into the current trie T , we trace the path associated with X in T , creating a new chain of nodes to store the remaining characters of X when we get stuck. The running time to insert X with length m is similar to a search, with worst-case $O(m \cdot |\Sigma|)$ performance, or expected $O(m)$ if using secondary hash tables at each node. Thus, constructing the entire trie for set S takes expected $O(n)$ time, where n is the total length of the strings of S .

There is a potential space inefficiency in the standard trie that has prompted the development of the **compressed trie**, which is also known (for historical reasons) as the **Patricia trie**. Namely, there are potentially a lot of nodes in the standard trie that have only one child, and the existence of such nodes is a waste. We discuss the compressed trie next.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
s	e	e		a		b	e	a	r	?		s	e	l	l		s	t	o	c	k	!
23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45
	s	e	e		a		b	u	l	l	?		b	u	y		s	t	o	c	k	!
46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68
	b	i	d		s	t	o	c	k	!		b	i	d		s	t	o	c	k	!	
69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88			
h	e	a	r		t	h	e		b	e	l	l	?		s	t	o	p	!			

(a)



(b)

Figure 13.8: Word matching with a standard trie: (a) text to be searched (articles and prepositions, which are also known as *stop words*, excluded); (b) standard trie for the words in the text, with leaves augmented with indications of the index at which the given work begins in the text. For example, the leaf for the word “stock” notes that the word begins at indices 17, 40, 51, and 62 of the text.

13.3.2 Compressed Tries

A **compressed trie** is similar to a standard trie but it ensures that each internal node in the trie has at least two children. It enforces this rule by compressing chains of single-child nodes into individual edges. (See Figure 13.9.) Let T be a standard trie. We say that an internal node v of T is **redundant** if v has one child and is not the root. For example, the trie of Figure 13.7 has eight redundant nodes. Let us also say that a chain of $k \geq 2$ edges,

$$(v_0, v_1)(v_1, v_2) \cdots (v_{k-1}, v_k),$$

is **redundant** if:

- v_i is redundant for $i = 1, \dots, k-1$.
- v_0 and v_k are not redundant.

We can transform T into a compressed trie by replacing each redundant chain $(v_0, v_1) \cdots (v_{k-1}, v_k)$ of $k \geq 2$ edges into a single edge (v_0, v_k) , relabeling v_k with the concatenation of the labels of nodes $v_1, \dots, v_k$.

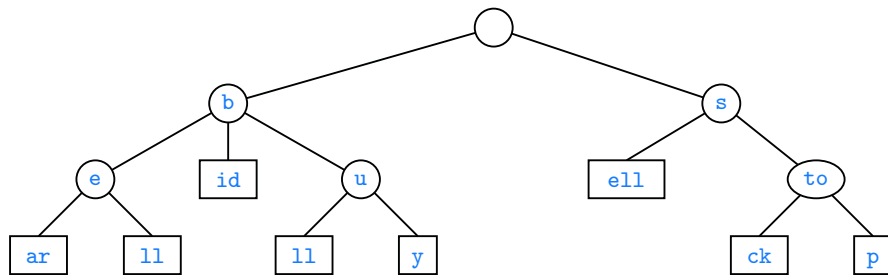


Figure 13.9: Compressed trie for the strings {bear, bell, bid, bull, buy, sell, stock, stop}. (Compare this with the standard trie shown in Figure 13.7.) Notice that, in addition to compression at the leaves, the internal node with label “to” is shared by words “stock” and “stop”.

Thus, nodes in a compressed trie are labeled with strings, which are substrings of strings in the collection, rather than with individual characters. The advantage of a compressed trie over a standard trie is that the number of nodes of the compressed trie is proportional to the number of strings and not to their total length, as shown in the following proposition (compare with Proposition 13.4).

Proposition 13.5: A compressed trie storing a collection S of s strings from an alphabet of size d has the following properties:

- Every internal node of T has at least two children and most d children.
- T has s leaves nodes.
- The number of nodes of T is $O(s)$.

The attentive reader may wonder whether the compression of paths provides any significant advantage, since it is offset by a corresponding expansion of the node labels. Indeed, a compressed trie is truly advantageous only when it is used as an *auxiliary* index structure over a collection of strings already stored in a primary structure, and is not required to actually store all the characters of the strings in the collection.

Suppose, for example, that the collection S of strings is an array of strings $S[0]$, $S[1]$, ..., $S[s-1]$. Instead of storing the label X of a node explicitly, we represent it implicitly by a combination of three integers (i, j, k) , such that $X = S[i][j..k]$; that is, X is the substring of $S[i]$ consisting of the characters from the j^{th} to the k^{th} inclusive. (See the example in Figure 13.10. Also compare with the standard trie of Figure 13.8.)

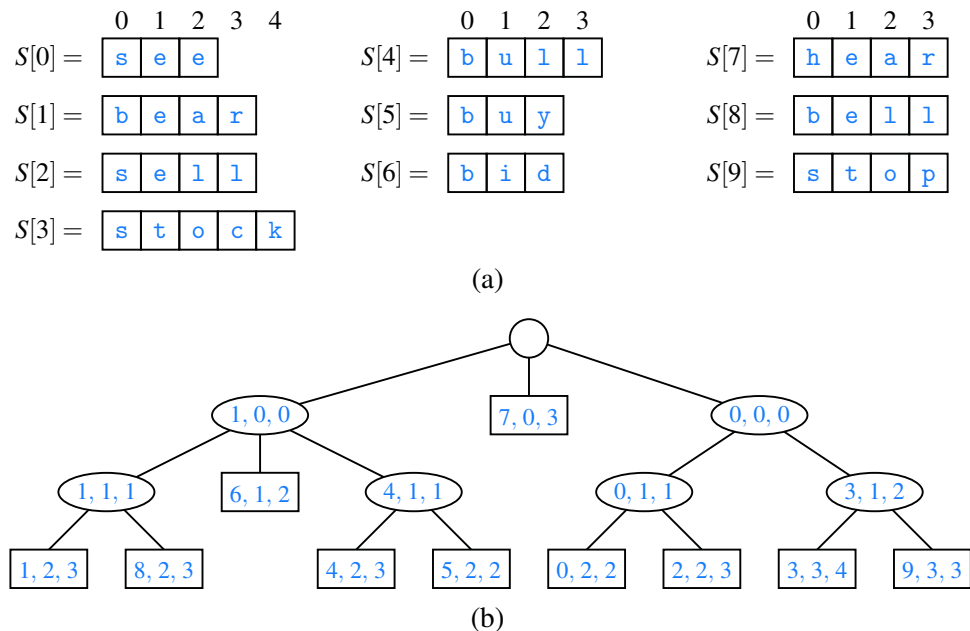


Figure 13.10: (a) Collection S of strings stored in an array. (b) Compact representation of the compressed trie for S .

This additional compression scheme allows us to reduce the total space for the trie itself from $O(n)$ for the standard trie to $O(s)$ for the compressed trie, where n is the total length of the strings in S and s is the number of strings in S . We must still store the different strings in S , of course, but we nevertheless reduce the space for the trie.

Searching in a compressed trie is not necessarily faster than in a standard tree, since there is still need to compare every character of the desired pattern with the potentially multicharacter labels while traversing paths in the trie.

13.3.3 Suffix Tries

One of the primary applications for tries is for the case when the strings in the collection S are all the suffixes of a string X . Such a trie is called the *suffix trie* (also known as a *suffix tree* or *position tree*) of string X . For example, Figure 13.11a shows the suffix trie for the eight suffixes of string “minimize.” For a suffix trie, the compact representation presented in the previous section can be further simplified. Namely, the label of each vertex is a pair “ $j..k$ ” indicating the string $X[j..k]$. (See Figure 13.11b.) To satisfy the rule that no suffix of X is a prefix of another suffix, we can add a special character, denoted with \$, that is not in the original alphabet Σ at the end of X (and thus to every suffix). That is, if string X has length n , we build a trie for the set of n strings $X[j..n-1]\$,$ for $j = 0, \dots, n-1$.

Saving Space

Using a suffix trie allows us to save space over a standard trie by using several space compression techniques, including those used for the compressed trie.

The advantage of the compact representation of tries now becomes apparent for suffix tries. Since the total length of the suffixes of a string X of length n is

$$1 + 2 + \dots + n = \frac{n(n+1)}{2},$$

storing all the suffixes of X explicitly would take $O(n^2)$ space. Even so, the suffix trie represents these strings implicitly in $O(n)$ space, as formally stated in the following proposition.

Proposition 13.6: *The compact representation of a suffix trie T for a string X of length n uses $O(n)$ space.*

Construction

We can construct the suffix trie for a string of length n with an incremental algorithm like the one given in Section 13.3.1. This construction takes $O(|\Sigma|n^2)$ time because the total length of the suffixes is quadratic in n . However, the (compact) suffix trie for a string of length n can be constructed in $O(n)$ time with a specialized algorithm, different from the one for general tries. This linear-time construction algorithm is fairly complex, however, and is not reported here. Still, we can take advantage of the existence of this fast construction algorithm when we want to use a suffix trie to solve other problems.

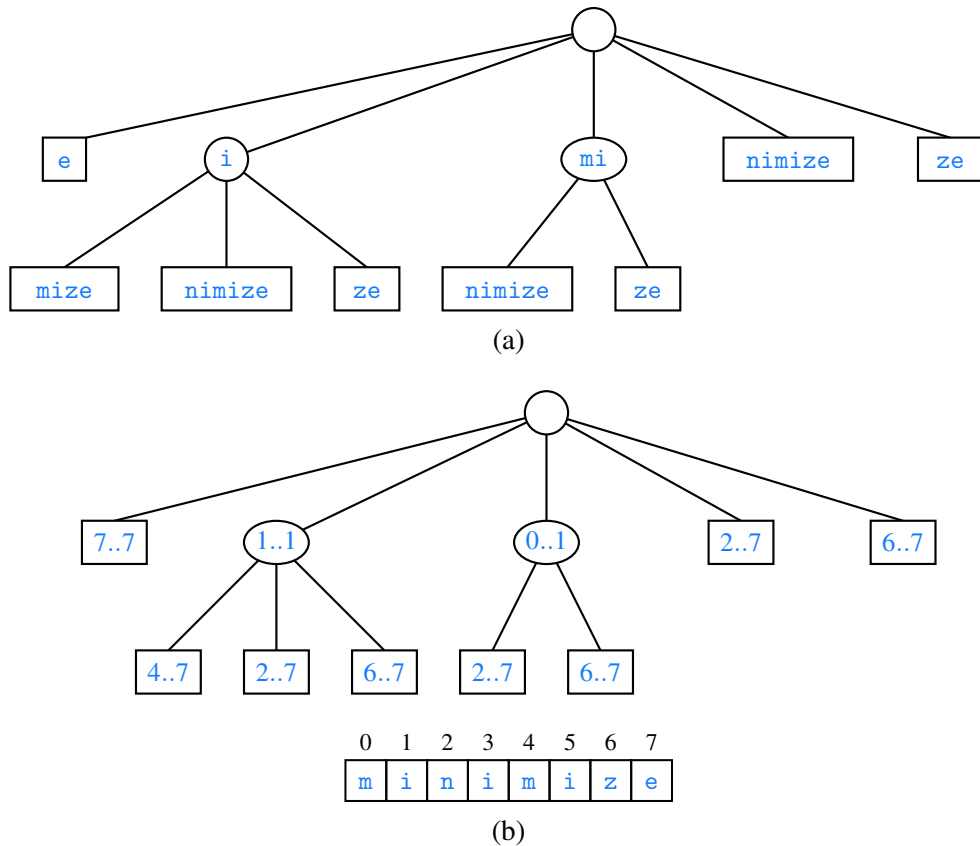


Figure 13.11: (a) Suffix trie T for the string $X = \text{"minimize"}$. (b) Compact representation of T , where pair $j..k$ denotes the substring $X[j..k]$ in the reference string.

Using a Suffix Trie

The suffix trie T for a string X can be used to efficiently perform pattern-matching queries on text X . Namely, we can determine whether a pattern is a substring of X by trying to trace a path associated with P in T . P is a substring of X if and only if such a path can be traced. The search down the trie T assumes that nodes in T store some additional information, with respect to the compact representation of the suffix trie:

If node v has label $j..k$ and Y is the string of length y associated with the path from the root to v (included), then $X[k - y + 1..k] = Y$.

This property ensures that we can compute the start index of the pattern in the text when a match occurs in $O(m)$ time.

13.3.4 Search Engine Indexing

The World Wide Web contains a huge collection of text documents (Web pages). Information about these pages are gathered by a program called a **Web crawler**, which then stores this information in a special dictionary database. A Web **search engine** allows users to retrieve relevant information from this database, thereby identifying relevant pages on the Web containing given keywords. In this section, we will present a simplified model of a search engine.

Inverted Files

The core information stored by a search engine is a map, called an ***inverted index*** or ***inverted file***, storing key-value pairs (w, L) , where w is a word and L is a collection of pages containing word w . The keys (words) in this map are called ***index terms*** and should be a set of vocabulary entries and proper nouns as large as possible. The values in this map are called ***occurrence lists*** and should cover as many Web pages as possible.

We can efficiently implement an inverted index with a data structure consisting of the following:

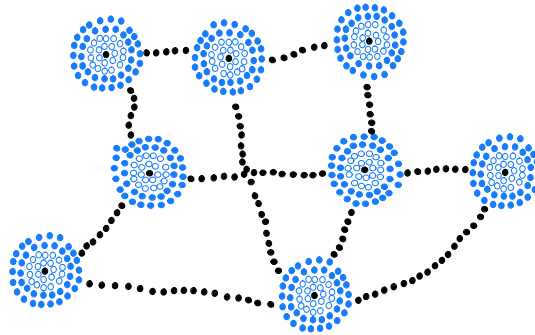
1. An array storing the occurrence lists of the terms (in no particular order).
2. A compressed trie for the set of index terms, where each leaf stores the index of the occurrence list of the associated term.

The reason for storing the occurrence lists outside the trie is to keep the size of the trie data structure sufficiently small to fit in internal memory. Instead, because of their large total size, the occurrence lists have to be stored on disk.

With our data structure, a query for a single keyword is similar to a word-matching query (Section 13.3.1). Namely, we find the keyword in the trie and we return the associated occurrence list.

When multiple keywords are given and the desired output are the pages containing ***all*** the given keywords, we retrieve the occurrence list of each keyword using the trie and return their intersection. To facilitate the intersection computation, each occurrence list should be implemented with a sequence sorted by address or with a map, to allow efficient set operations.

In addition to the basic task of returning a list of pages containing given keywords, search engines provide an important additional service by ***ranking*** the pages returned by relevance. Devising fast and accurate ranking algorithms for search engines is a major challenge for computer researchers and electronic commerce companies.



Contents

14.1 Graphs	612
14.1.1 The Graph ADT	618
14.2 Data Structures for Graphs	619
14.2.1 Edge List Structure	620
14.2.2 Adjacency List Structure	622
14.2.3 Adjacency Map Structure	624
14.2.4 Adjacency Matrix Structure	625
14.2.5 Java Implementation	626
14.3 Graph Traversals	630
14.3.1 Depth-First Search	631
14.3.2 DFS Implementation and Extensions	636
14.3.3 Breadth-First Search	640
14.4 Transitive Closure	643
14.5 Directed Acyclic Graphs	647
14.5.1 Topological Ordering	647
14.6 Shortest Paths	651
14.6.1 Weighted Graphs	651
14.6.2 Dijkstra's Algorithm	653
14.7 Minimum Spanning Trees	662
14.7.1 Prim-Jarník Algorithm	664
14.7.2 Kruskal's Algorithm	667
14.7.3 Disjoint Partitions and Union-Find Structures	672
14.8 Exercises	677

14.1 Graphs

A **graph** is a way of representing relationships that exist between pairs of objects. That is, a graph is a set of objects, called vertices, together with a collection of pairwise connections between them, called edges. Graphs have applications in modeling many domains, including mapping, transportation, computer networks, and electrical engineering. By the way, this notion of a “graph” should not be confused with bar charts and function plots, as these kinds of “graphs” are unrelated to the topic of this chapter.

Viewed abstractly, a **graph** G is simply a set V of **vertices** and a collection E of pairs of vertices from V , called **edges**. Thus, a graph is a way of representing connections or relationships between pairs of objects from some set V . Incidentally, some books use different terminology for graphs and refer to what we call vertices as **nodes** and what we call edges as **arcs**. We use the terms “vertices” and “edges.”

Edges in a graph are either **directed** or **undirected**. An edge (u, v) is said to be **directed** from u to v if the pair (u, v) is ordered, with u preceding v . An edge (u, v) is said to be **undirected** if the pair (u, v) is not ordered. Undirected edges are sometimes denoted with set notation, as $\{u, v\}$, but for simplicity we use the pair notation (u, v) , noting that in the undirected case (u, v) is the same as (v, u) . Graphs are typically visualized by drawing the vertices as ovals or rectangles and the edges as segments or curves connecting pairs of ovals and rectangles. The following are some examples of directed and undirected graphs.

Example 14.1: We can visualize collaborations among the researchers of a certain discipline by constructing a graph whose vertices are associated with the researchers themselves, and whose edges connect pairs of vertices associated with researchers who have coauthored a paper or book. (See Figure 14.1.) Such edges are undirected because coauthorship is a **symmetric** relation; that is, if A has coauthored something with B , then B necessarily has coauthored something with A .

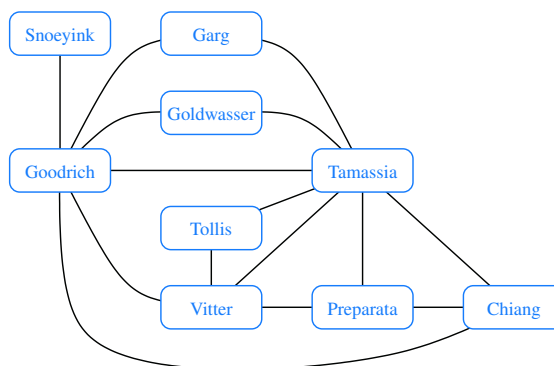


Figure 14.1: Graph of coauthorship among some authors.

Example 14.2: We can associate with an object-oriented program a graph whose vertices represent the classes defined in the program, and whose edges indicate inheritance between classes. There is an edge from a vertex v to a vertex u if the class for v inherits from the class for u . Such edges are directed because the inheritance relation only goes in one direction (that is, it is **asymmetric**).

If all the edges in a graph are undirected, then we say the graph is an **undirected graph**. Likewise, a **directed graph**, also called a **digraph**, is a graph whose edges are all directed. A graph that has both directed and undirected edges is often called a **mixed graph**. Note that an undirected or mixed graph can be converted into a directed graph by replacing every undirected edge (u, v) by the pair of directed edges (u, v) and (v, u) . It is often useful, however, to keep undirected and mixed graphs represented as they are, for such graphs have several applications, as in the following example.

Example 14.3: A city map can be modeled as a graph whose vertices are intersections or dead ends, and whose edges are stretches of streets without intersections. This graph has both undirected edges, which correspond to stretches of two-way streets, and directed edges, which correspond to stretches of one-way streets. Thus, in this way, a graph modeling a city map is a mixed graph.

Example 14.4: Physical examples of graphs are present in the electrical wiring and plumbing networks of a building. Such networks can be modeled as graphs, where each connector, fixture, or outlet is viewed as a vertex, and each uninterrupted stretch of wire or pipe is viewed as an edge. Such graphs are actually components of much larger graphs, namely the local power and water distribution networks. Depending on the specific aspects of these graphs that we are interested in, we may consider their edges as undirected or directed, for, in principle, water can flow in a pipe and current can flow in a wire in either direction.

The two vertices joined by an edge are called the **end vertices** (or **endpoints**) of the edge. If an edge is directed, its first endpoint is its **origin** and the other is the **destination** of the edge. Two vertices u and v are said to be **adjacent** if there is an edge whose end vertices are u and v . An edge is said to be **incident** to a vertex if the vertex is one of the edge's endpoints. The **outgoing edges** of a vertex are the directed edges whose origin is that vertex. The **incoming edges** of a vertex are the directed edges whose destination is that vertex. The **degree** of a vertex v , denoted $\deg(v)$, is the number of incident edges of v . The **in-degree** and **out-degree** of a vertex v are the number of the incoming and outgoing edges of v , and are denoted $\text{indeg}(v)$ and $\text{outdeg}(v)$, respectively.

Example 14.5: We can study air transportation by constructing a graph G , called a **flight network**, whose vertices are associated with airports, and whose edges are associated with flights. (See Figure 14.2.) In graph G , the edges are directed because a given flight has a specific travel direction. The endpoints of an edge e in G correspond respectively to the origin and destination of the flight corresponding to e . Two airports are adjacent in G if there is a flight that flies between them, and an edge e is incident to a vertex v in G if the flight for e flies to or from the airport for v . The outgoing edges of a vertex v correspond to the outbound flights from v 's airport, and the incoming edges correspond to the inbound flights to v 's airport. Finally, the in-degree of a vertex v of G corresponds to the number of inbound flights to v 's airport, and the out-degree of a vertex v in G corresponds to the number of outbound flights.

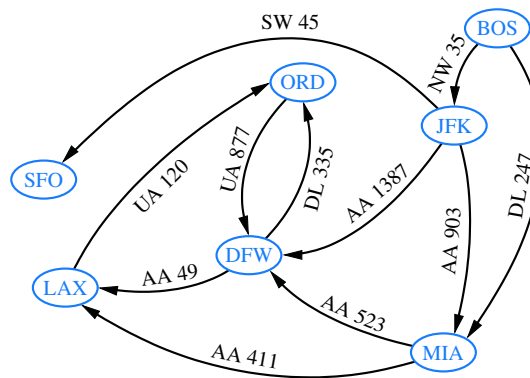


Figure 14.2: Example of a directed graph representing a flight network. The endpoints of edge UA 120 are LAX and ORD; hence, LAX and ORD are adjacent. The in-degree of DFW is 3, and the out-degree of DFW is 2.

The definition of a graph refers to the group of edges as a **collection**, not a **set**, thus allowing two undirected edges to have the same end vertices, and for two directed edges to have the same origin and the same destination. Such edges are called **parallel edges** or **multiple edges**. A flight network can contain parallel edges (Example 14.5), such that multiple edges between the same pair of vertices could indicate different flights operating on the same route at different times of the day. Another special type of edge is one that connects a vertex to itself. Namely, we say that an edge (undirected or directed) is a **self-loop** if its two endpoints coincide. A self-loop may occur in a graph associated with a city map (Example 14.3), where it would correspond to a “circle” (a curving street that returns to its starting point).

With few exceptions, graphs do not have parallel edges or self-loops. Such graphs are said to be **simple**. Thus, we can usually say that the edges of a simple graph are a **set** of vertex pairs (and not just a collection). Throughout this chapter, we will assume that a graph is simple unless otherwise specified.

A **path** is a sequence of alternating vertices and edges that starts at a vertex and ends at a vertex such that each edge is incident to its predecessor and successor vertex. A **cycle** is a path that starts and ends at the same vertex, and that includes at least one edge. We say that a path is **simple** if each vertex in the path is distinct, and we say that a cycle is **simple** if each vertex in the cycle is distinct, except for the first and last one. A **directed path** is a path such that all edges are directed and are traversed along their direction. A **directed cycle** is similarly defined. For example, in Figure 14.2, (BOS, NW 35, JFK, AA 1387, DFW) is a directed simple path, and (LAX, UA 120, ORD, UA 877, DFW, AA 49, LAX) is a directed simple cycle. Note that a directed graph may have a cycle consisting of two edges with opposite direction between the same pair of vertices, for example (ORD, UA 877, DFW, DL 335, ORD) in Figure 14.2. A directed graph is **acyclic** if it has no directed cycles. For example, if we were to remove the edge UA 877 from the graph in Figure 14.2, the remaining graph is acyclic. If a graph is simple, we may omit the edges when describing path P or cycle C , as these are well defined, in which case P is a list of adjacent vertices and C is a cycle of adjacent vertices.

Example 14.6: Given a graph G representing a city map (see Example 14.3), we can model a couple driving to dinner at a recommended restaurant as traversing a path through G . If they know the way, and do not accidentally go through the same intersection twice, then they traverse a simple path in G . Likewise, we can model the entire trip the couple takes, from their home to the restaurant and back, as a cycle. If they go home from the restaurant in a completely different way than how they went, not even going through the same intersection twice, then their entire round trip is a simple cycle. Finally, if they travel along one-way streets for their entire trip, we can model their night out as a directed cycle.

Given vertices u and v of a (directed) graph G , we say that u **reaches** v , and that v is **reachable** from u , if G has a (directed) path from u to v . In an undirected graph, the notion of **reachability** is symmetric, that is to say, u reaches v if and only if v reaches u . However, in a directed graph, it is possible that u reaches v but v does not reach u , because a directed path must be traversed according to the respective directions of the edges. A graph is **connected** if, for any two vertices, there is a path between them. A directed graph $\vec{G}$ is **strongly connected** if for any two vertices u and v of $\vec{G}$, u reaches v and v reaches u . (See Figure 14.3 for some examples.)

A **subgraph** of a graph G is a graph H whose vertices and edges are subsets of the vertices and edges of G , respectively. A **spanning subgraph** of G is a subgraph of G that contains all the vertices of the graph G . If a graph G is not connected, its maximal connected subgraphs are called the **connected components** of G . A **forest** is a graph without cycles. A **tree** is a connected forest, that is, a connected graph without cycles. A **spanning tree** of a graph is a spanning subgraph that is a tree. (Note that this definition of a tree is somewhat different from the one given in Chapter 8, as there is not necessarily a designated root.)

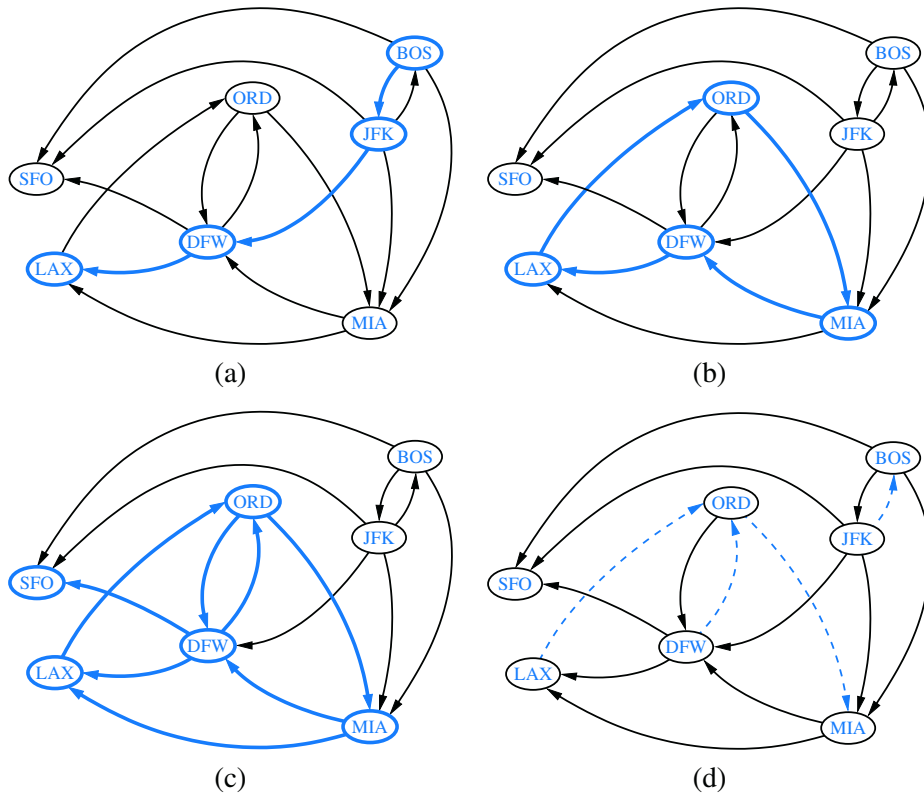


Figure 14.3: Examples of reachability in a directed graph: (a) a directed path from BOS to LAX is highlighted; (b) a directed cycle (ORD, MIA, DFW, LAX, ORD) is highlighted; its vertices induce a strongly connected subgraph; (c) the subgraph of the vertices and edges reachable from ORD is highlighted; (d) the removal of the dashed edges results in a directed acyclic graph.

Example 14.7: Perhaps the most talked about graph today is the Internet, which can be viewed as a graph whose vertices are computers and whose (undirected) edges are communication connections between pairs of computers on the Internet. The computers and the connections between them in a single domain, like wiley.com, form a subgraph of the Internet. If this subgraph is connected, then two users on computers in this domain can send email to one another without having their information packets ever leave their domain. Suppose the edges of this subgraph form a spanning tree. This implies that, if even a single connection goes down (for example, because someone pulls a communication cable out of the back of a computer in this domain), then this subgraph will no longer be connected.

In the propositions that follow, we explore a few important properties of graphs.

Proposition 14.8: *If G is a graph with m edges and vertex set V , then*

$$\sum_{v \in V} \deg(v) = 2m.$$

Justification: An edge (u, v) is counted twice in the summation above; once by its endpoint u and once by its endpoint v . Thus, the total contribution of the edges to the degrees of the vertices is twice the number of edges. ■

Proposition 14.9: *If G is a directed graph with m edges and vertex set V , then*

$$\sum_{v \in V} \text{indeg}(v) = \sum_{v \in V} \text{outdeg}(v) = m.$$

Justification: In a directed graph, an edge (u, v) contributes one unit to the out-degree of its origin u and one unit to the in-degree of its destination v . Thus, the total contribution of the edges to the out-degrees of the vertices is equal to the number of edges, and similarly for the in-degrees. ■

We next show that a simple graph with n vertices has $O(n^2)$ edges.

Proposition 14.10: *Let G be a simple graph with n vertices and m edges. If G is undirected, then $m \leq n(n-1)/2$, and if G is directed, then $m \leq n(n-1)$.*

Justification: Suppose that G is undirected. Since no two edges can have the same endpoints and there are no self-loops, the maximum degree of a vertex in G is $n-1$ in this case. Thus, by Proposition 14.8, $2m \leq n(n-1)$. Now suppose that G is directed. Since no two edges can have the same origin and destination, and there are no self-loops, the maximum in-degree of a vertex in G is $n-1$ in this case. Thus, by Proposition 14.9, $m \leq n(n-1)$. ■

There are a number of simple properties of trees, forests, and connected graphs.

Proposition 14.11: *Let G be an undirected graph with n vertices and m edges.*

- *If G is connected, then $m \geq n-1$.*
- *If G is a tree, then $m = n-1$.*
- *If G is a forest, then $m \leq n-1$.*

14.1.1 The Graph ADT

A graph is a collection of vertices and edges. We model the abstraction as a combination of three data types: Vertex, Edge, and Graph. A Vertex is a lightweight object that stores an arbitrary element provided by the user (e.g., an airport code); we assume the element can be retrieved with the `getElement()` method. An Edge also stores an associated object (e.g., a flight number, travel distance, cost), which is returned by its `getElement()` method.

The primary abstraction for a graph is the Graph ADT. We presume that a graph can be either *undirected* or *directed*, with the designation declared upon construction; recall that a mixed graph can be represented as a directed graph, modeling edge $\{u, v\}$ as a pair of directed edges (u, v) and (v, u) . The Graph ADT includes the following methods:

- `numVertices()`: Returns the number of vertices of the graph.
- `vertices()`: Returns an iteration of all the vertices of the graph.
- `numEdges()`: Returns the number of edges of the graph.
- `edges()`: Returns an iteration of all the edges of the graph.
- `getEdge( $u, v$ )`: Returns the edge from vertex u to vertex v , if one exists; otherwise return null. For an undirected graph, there is no difference between `getEdge( $u, v$ )` and `getEdge( $v, u$ )`.
- `endVertices( $e$ )`: Returns an array containing the two endpoint vertices of edge e . If the graph is directed, the first vertex is the origin and the second is the destination.
- `opposite( $v, e$ )`: For edge e incident to vertex v , returns the other vertex of the edge; an error occurs if e is not incident to v .
- `outDegree( $v$ )`: Returns the number of outgoing edges from vertex v .
- `inDegree( $v$ )`: Returns the number of incoming edges to vertex v . For an undirected graph, this returns the same value as does `outDegree( $v$ )`.
- `outgoingEdges( $v$ )`: Returns an iteration of all outgoing edges from vertex v .
- `incomingEdges( $v$ )`: Returns an iteration of all incoming edges to vertex v . For an undirected graph, this returns the same collection as does `outgoingEdges( $v$ )`.
- `insertVertex( $x$ )`: Creates and returns a new Vertex storing element x .
- `insertEdge( $u, v, x$ )`: Creates and returns a new Edge from vertex u to vertex v , storing element x ; an error occurs if there already exists an edge from u to v .
- `removeVertex( $v$ )`: Removes vertex v and all its incident edges from the graph.
- `removeEdge( $e$ )`: Removes edge e from the graph.

14.2 Data Structures for Graphs

In this section, we introduce four data structures for representing a graph. In each representation, we maintain a collection to store the vertices of a graph. However, the four representations differ greatly in the way they organize the edges.

- In an **edge list**, we maintain an unordered list of all edges. This minimally suffices, but there is no efficient way to locate a particular edge (u, v) , or the set of all edges incident to a vertex v .
- In an **adjacency list**, we additionally maintain, for each vertex, a separate list containing those edges that are incident to the vertex. This organization allows us to more efficiently find all edges incident to a given vertex.
- An **adjacency map** is similar to an adjacency list, but the secondary container of all edges incident to a vertex is organized as a map, rather than as a list, with the adjacent vertex serving as a key. This allows more efficient access to a specific edge (u, v) , for example, in $O(1)$ expected time with hashing.
- An **adjacency matrix** provides worst-case $O(1)$ access to a specific edge (u, v) by maintaining an $n \times n$ matrix, for a graph with n vertices. Each slot is dedicated to storing a reference to the edge (u, v) for a particular pair of vertices u and v ; if no such edge exists, the slot will store null.

A summary of the performance of these structures is given in Table 14.1.

Method	Edge List	Adj. List	Adj. Map	Adj. Matrix
numVertices()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
numEdges()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
vertices()	$O(n)$	$O(n)$	$O(n)$	$O(n)$
edges()	$O(m)$	$O(m)$	$O(m)$	$O(m)$
getEdge(u, v)	$O(m)$	$O(\min(d_u, d_v))$	$O(1)$ exp.	$O(1)$
outDegree(v) inDegree(v)	$O(m)$	$O(1)$	$O(1)$	$O(n)$
outgoingEdges(v) incomingEdges(v)	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n)$
insertVertex(x)	$O(1)$	$O(1)$	$O(1)$	$O(n^2)$
removeVertex(v)	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n^2)$
insertEdge(u, v, x)	$O(1)$	$O(1)$	$O(1)$ exp.	$O(1)$
removeEdge(e)	$O(1)$	$O(1)$	$O(1)$ exp.	$O(1)$

Table 14.1: A summary of the running times for the methods of the graph ADT, using the graph representations discussed in this section. We let n denote the number of vertices, m the number of edges, and d_v the degree of vertex v . Note that the adjacency matrix uses $O(n^2)$ space, while all other structures use $O(n + m)$ space.

14.2.1 Edge List Structure

The **edge list** structure is possibly the simplest, though not the most efficient, representation of a graph G . All vertex objects are stored in an unordered list V , and all edge objects are stored in an unordered list E . We illustrate an example of the edge list structure for a graph G in Figure 14.4.

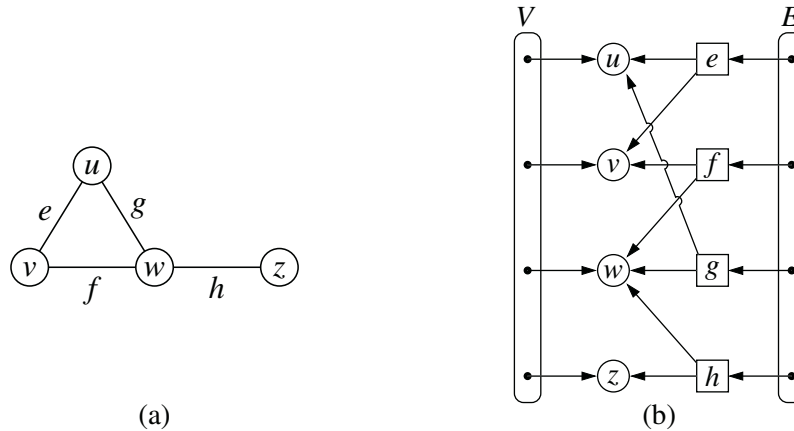


Figure 14.4: (a) A graph G ; (b) schematic representation of the edge list structure for G . Notice that an edge object refers to the two vertex objects that correspond to its endpoints, but that vertices do not refer to incident edges.

To support the many methods of the Graph ADT (Section 14.1), we assume the following additional features of an edge list representation. Collections V and E are represented with doubly linked lists using our `LinkedPositionalList` class from Chapter 7.

Vertex Objects

The vertex object for a vertex v storing element x has instance variables for:

- A reference to element x , to support the `getElement()` method.
- A reference to the position of the vertex instance in the list V , thereby allowing v to be efficiently removed from V if it were removed from the graph.

Edge Objects

The edge object for an edge e storing element x has instance variables for:

- A reference to element x , to support the `getElement()` method.
- References to the vertex objects associated with the endpoint vertices of e . These will allow for constant-time support for methods `endVertices( $e$ )` and `opposite( $v$ ,  $e$ )`.
- A reference to the position of the edge instance in list E , thereby allowing e to be efficiently removed from E if it were removed from the graph.

Performance of the Edge List Structure

The performance of an edge list structure in fulfilling the graph ADT is summarized in Table 14.2. We begin by discussing the space usage, which is $O(n + m)$ for representing a graph with n vertices and m edges. Each individual vertex or edge instance uses $O(1)$ space, and the additional lists V and E use space proportional to their number of entries.

In terms of running time, the edge list structure does as well as one could hope in terms of reporting the number of vertices or edges, or in producing an iteration of those vertices or edges. By querying the respective list V or E , the `numVertices` and `numEdges` methods run in $O(1)$ time, and by iterating through the appropriate list, the methods `vertices` and `edges` run respectively in $O(n)$ and $O(m)$ time.

The most significant limitations of an edge list structure, especially when compared to the other graph representations, are the $O(m)$ running times of methods `getEdge( $u, v$ )`, `outDegree( $v$ )`, and `outgoingEdges( $v$ )` (and corresponding methods `inDegree` and `incomingEdges`). The problem is that with all edges of the graph in an unordered list E , the only way to answer those queries is through an exhaustive inspection of all edges.

Finally, we consider the methods that update the graph. It is easy to add a new vertex or a new edge to the graph in $O(1)$ time. For example, a new edge can be added to the graph by creating an `Edge` instance storing the given element as data, adding that instance to the positional list E , and recording its resulting `Position` within E as an attribute of the edge. That stored position can later be used to locate and remove this edge from E in $O(1)$ time, and thus implement the method `removeEdge( $e$ )`.

It is worth discussing why the `removeVertex( $v$ )` method has a running time of $O(m)$. As stated in the graph ADT, when a vertex v is removed from the graph, all edges incident to v must also be removed (otherwise, we would have a contradiction of edges that refer to vertices that are not part of the graph). To locate the incident edges to the vertex, we must examine all edges of E .

Method	Running Time
<code>numVertices()</code> , <code>numEdges()</code>	$O(1)$
<code>vertices()</code>	$O(n)$
<code>edges()</code>	$O(m)$
<code>getEdge(u, v)</code> , <code>outDegree(v)</code> , <code>outgoingEdges(v)</code>	$O(m)$
<code>insertVertex(x)</code> , <code>insertEdge(u, v, x)</code> , <code>removeEdge(e)</code>	$O(1)$
<code>removeVertex(v)</code>	$O(m)$

Table 14.2: Running times of the methods of a graph implemented with the edge list structure. The space used is $O(n + m)$, where n is the number of vertices and m is the number of edges.

14.2.2 Adjacency List Structure

The adjacency list structure for a graph adds extra information to the edge list structure that supports direct access to the incident edges (and thus to the adjacent vertices) of each vertex. Specifically, for each vertex v , we maintain a collection $I(v)$, called the **incidence collection** of v , whose entries are edges incident to v . In the case of a directed graph, outgoing and incoming edges can be respectively stored in two separate collections, $I_{\text{out}}(v)$ and $I_{\text{in}}(v)$. Traditionally, the incidence collection $I(v)$ for a vertex v is a list, which is why we call this way of representing a graph the **adjacency list** structure.

We require that the primary structure for an adjacency list maintain the collection V of vertices in a way so that we can locate the secondary structure $I(v)$ for a given vertex v in $O(1)$ time. This could be done by using a positional list to represent V , with each Vertex instance maintaining a direct reference to its $I(v)$ incidence collection; we illustrate such an adjacency list structure of a graph in Figure 14.5. If vertices can be uniquely numbered from 0 to $n - 1$, we could instead use a primary array-based structure to access the appropriate secondary lists.

The primary benefit of an adjacency list is that the collection $I(v)$ (or more specifically, $I_{\text{out}}(v)$) contains exactly those edges that should be reported by the method `outgoingEdges(v)`. Therefore, we can implement this method by iterating the edges of $I(v)$ in $O(\deg(v))$ time, where $\deg(v)$ is the degree of vertex v . This is the best possible outcome for any graph representation, because there are $\deg(v)$ edges to be reported.

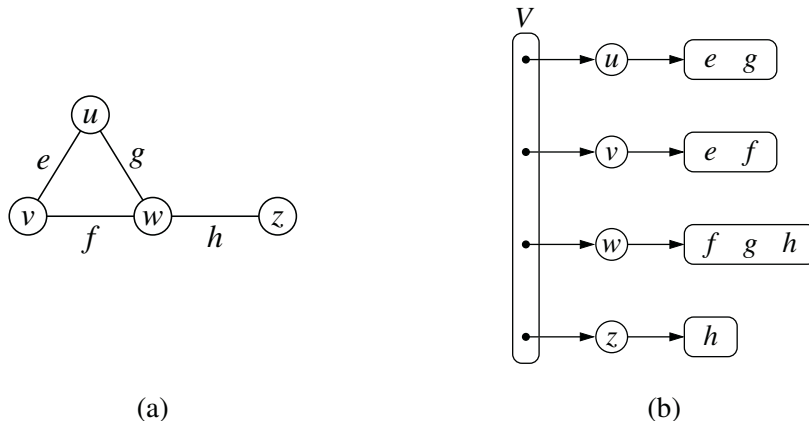


Figure 14.5: (a) An undirected graph G ; (b) a schematic representation of the adjacency list structure for G . Collection V is the primary list of vertices, and each vertex has an associated list of incident edges. Although not diagrammed as such, we presume that each edge of the graph is represented with a unique Edge instance that maintains references to its endpoint vertices, and that E is a list of all edges.

Performance of the Adjacency List Structure

Table 14.3 summarizes the performance of the adjacency list structure implementation of a graph, assuming that the primary collection V and E , and all secondary collections $I(v)$ are implemented with doubly linked lists.

Asymptotically, the space requirements for an adjacency list are the same as an edge list structure, using $O(n + m)$ space for a graph with n vertices and m edges. It is clear that the primary lists of vertices and edges use $O(n + m)$ space. In addition, the sum of the lengths of all secondary lists is $O(m)$, for reasons that were formalized in Propositions 14.8 and 14.9. In short, an undirected edge (u, v) is referenced in both $I(u)$ and $I(v)$, but its presence in the graph results in only a constant amount of additional space.

We have already noted that the `outgoingEdges( $v$ )` method can be achieved in $O(\deg(v))$ time based on use of $I(v)$. For a directed graph, this is more specifically $O(\text{outdeg}(v))$ based on use of $I_{\text{out}}(v)$. The `outDegree( $v$ )` method of the graph ADT can run in $O(1)$ time, assuming collection $I(v)$ can report its size in similar time. To locate a specific edge for implementing `getEdge( $u, v$ )`, we can search through either $I(u)$ and $I(v)$ (or for a directed graph, either $I_{\text{out}}(u)$ or $I_{\text{in}}(v)$). By choosing the smaller of the two, we get $O(\min(\deg(u), \deg(v)))$ running time.

The rest of the bounds in Table 14.3 can be achieved with additional care. To efficiently support deletions of edges, an edge (u, v) would need to maintain a reference to its positions within both $I(u)$ and $I(v)$, so that it could be deleted from those collections in $O(1)$ time. To remove a vertex v , we must also remove any incident edges, but at least we can locate those edges in $O(\deg(v))$ time.

Method	Running Time
<code>numVertices()</code> , <code>numEdges()</code>	$O(1)$
<code>vertices()</code>	$O(n)$
<code>edges()</code>	$O(m)$
<code>getEdge(u, v)</code>	$O(\min(\deg(u), \deg(v)))$
<code>outDegree(v)</code> , <code>inDegree(v)</code>	$O(1)$
<code>outgoingEdges(v)</code> , <code>incomingEdges(v)</code>	$O(\deg(v))$
<code>insertVertex(x)</code> , <code>insertEdge(u, v, x)</code>	$O(1)$
<code>removeEdge(e)</code>	$O(1)$
<code>removeVertex(v)</code>	$O(\deg(v))$

Table 14.3: Running times of the methods of a graph implemented with the adjacency list structure. The space used is $O(n + m)$, where n is the number of vertices and m is the number of edges.

14.2.3 Adjacency Map Structure

In the adjacency list structure, we assume that the secondary incidence collections are implemented as unordered linked lists. Such a collection $I(v)$ uses space proportional to $O(\deg(v))$, allows an edge to be added or removed in $O(1)$ time, and allows an iteration of all edges incident to vertex v in $O(\deg(v))$ time. However, the best implementation of $\text{getEdge}(u, v)$ requires $O(\min(\deg(u), \deg(v)))$ time, because we must search through either $I(u)$ or $I(v)$.

We can improve the performance by using a hash-based map to implement $I(v)$ for each vertex v . Specifically, we let the opposite endpoint of each incident edge serve as a key in the map, with the edge structure serving as the value. We call such a graph representation an **adjacency map**. (See Figure 14.6.) The space usage for an adjacency map remains $O(n + m)$, because $I(v)$ uses $O(\deg(v))$ space for each vertex v , as with the adjacency list.

The advantage of the adjacency map, relative to an adjacency list, is that the $\text{getEdge}(u, v)$ method can be implemented in **expected** $O(1)$ time by searching for vertex u as a key in $I(v)$, or vice versa. This provides a likely improvement over the adjacency list, while retaining the worst-case bound of $O(\min(\deg(u), \deg(v)))$.

In comparing the performance of adjacency map to other representations (see Table 14.1), we find that it essentially achieves optimal running times for all methods, making it an excellent all-purpose choice as a graph representation.

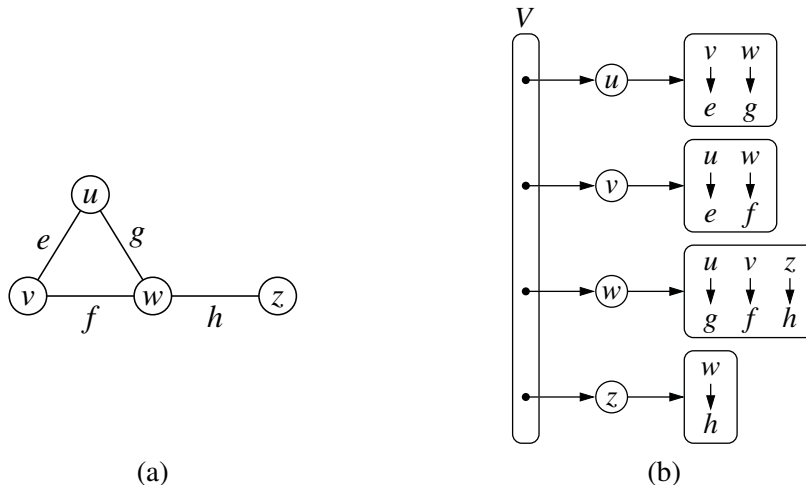


Figure 14.6: (a) An undirected graph G ; (b) a schematic representation of the adjacency map structure for G . Each vertex maintains a secondary map in which neighboring vertices serve as keys, with the connecting edges as associated values. As with the adjacency list, we presume that there is also an overall list E of all Edge instances.

14.2.4 Adjacency Matrix Structure

The **adjacency matrix** structure for a graph G augments the edge list structure with a matrix A (that is, a two-dimensional array, as in Section 3.1.5), which allows us to locate an edge between a given pair of vertices in *worst-case* constant time. In the adjacency matrix representation, we think of the vertices as being the integers in the set $\{0, 1, \dots, n-1\}$ and the edges as being pairs of such integers. This allows us to store references to edges in the cells of a two-dimensional $n \times n$ array A . Specifically, the cell $A[i][j]$ holds a reference to the edge (u, v) , if it exists, where u is the vertex with index i and v is the vertex with index j . If there is no such edge, then $A[i][j] = \text{null}$. We note that array A is symmetric if graph G is undirected, as $A[i][j] = A[j][i]$ for all pairs i and j . (See Figure 14.7.)

The most significant advantage of an adjacency matrix is that any edge (u, v) can be accessed in worst-case $O(1)$ time; recall that the adjacency map supports that operation in $O(1)$ *expected* time. However, several operations are less efficient with an adjacency matrix. For example, to find the edges incident to vertex v , we must presumably examine all n entries in the row associated with v ; recall that an adjacency list or map can locate those edges in optimal $O(\deg(v))$ time. Adding or removing vertices from a graph is problematic, as the matrix must be resized.

Furthermore, the $O(n^2)$ space usage of an adjacency matrix is typically far worse than the $O(n + m)$ space required of the other representations. Although, in the worst case, the number of edges in a **dense** graph will be proportional to n^2 , most real-world graphs are **sparse**. In such cases, use of an adjacency matrix is inefficient. However, if a graph is dense, the constants of proportionality of an adjacency matrix can be smaller than that of an adjacency list or map. In fact, if edges do not have auxiliary data, a boolean adjacency matrix can use one bit per edge slot, such that $A[i][j] = \text{true}$ if and only if associated (u, v) is an edge.

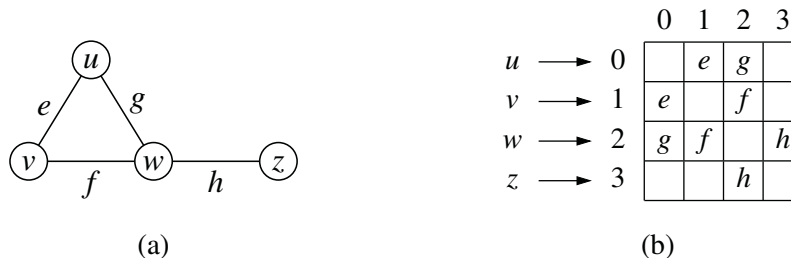


Figure 14.7: (a) An undirected graph G ; (b) a schematic representation of the auxiliary adjacency matrix structure for G , in which n vertices are mapped to indices 0 to $n-1$. Although not diagrammed as such, we presume that there is a unique Edge instance for each edge, and that it maintains references to its endpoint vertices. We also assume that there is a secondary edge list (not pictured), to allow the `edges()` method to run in $O(m)$ time, for a graph with m edges.

14.2.5 Java Implementation

In this section, we provide an implementation of the Graph ADT, based on the *adjacency map* representation, as described in Section 14.2.3. We use positional lists to represent each of the primary lists V and E , as originally described in the edge list representation. Additionally, for each vertex v , we use a hash-based map to represent the secondary incidence map $I(v)$.

To gracefully support both undirected and directed graphs, each vertex maintains two different map references: outgoing and incoming. In the directed case, these are initialized to two distinct map instances, representing $I_{\text{out}}(v)$ and $I_{\text{in}}(v)$, respectively. In the case of an undirected graph, we assign both outgoing and incoming as aliases to a single map instance.

Our implementation is organized as follows. We assume definitions for `Vertex`, `Edge`, and `Graph` interfaces, although for the sake of brevity, we do not include those definitions in the book (they are available online). We then define a concrete `AdjacencyMapGraph` class, with nested classes `InnerVertex` and `InnerEdge` to implement the vertex and edge abstractions. These classes use generic parameters V and E to designate the element type stored respectively at vertices and edges.

We begin in Code Fragment 14.1, with the definitions of the `InnerVertex` and `InnerEdge` classes (although in reality, those definitions should be nested within the following `AdjacencyMapGraph` class). Note well how the `InnerVertex` constructor initializes the outgoing and incoming instance variables depending on whether the overall graph is undirected or directed.

Code Fragments 14.2 and 14.3 contain the core implementation of the class `AdjacencyMapGraph`. A graph instance maintains a boolean variable that designates whether the graph is directed, and it maintains the vertex list and edge list. Although not shown in these code fragments, our implementation includes private `validate` methods that perform type conversions between the public `Vertex` and `Edge` interface types to the concrete `InnerVertex` and `InnerEdge` classes, while also performing some error checking. This design is similar to the `validate` method of the `LinkedPositionalList` class (see Code Fragment 7.10 of Section 7.3.3), which converts an outward `Position` to the underlying `Node` type for that class.

The most complex methods are those that modify the graph. When `insertVertex` is called, we must create a new `InnerVertex` instance, add that vertex to the list of vertices, and record its position within that list (so that we can efficiently delete it from the list if the vertex is removed from the graph). When inserting an edge (u, v) , we must also create a new instance, add it to the edge list, and record its position, yet we must also add the new edge to the outgoing adjacency map for vertex u , and the incoming map for vertex v . Code Fragment 14.3 contains code for `removeVertex` as well; the implementation of `removeEdge` is not included, but is available in the online version of the code.

```

1  /** A vertex of an adjacency map graph representation. */
2  private class InnerVertex<V> implements Vertex<V> {
3      private V element;
4      private Position<Vertex<V>> pos;
5      private Map<Vertex<V>, Edge<E>> outgoing, incoming;
6      /** Constructs a new InnerVertex instance storing the given element. */
7      public InnerVertex(V elem, boolean graphsDirected) {
8          element = elem;
9          outgoing = new ProbeHashMap<>();
10         if (graphsDirected)
11             incoming = new ProbeHashMap<>();
12         else
13             incoming = outgoing;          // if undirected, alias outgoing map
14     }
15     /** Returns the element associated with the vertex. */
16     public V getElement() { return element; }
17     /** Stores the position of this vertex within the graph's vertex list. */
18     public void setPosition(Position<Vertex<V>> p) { pos = p; }
19     /** Returns the position of this vertex within the graph's vertex list. */
20     public Position<Vertex<V>> getPosition() { return pos; }
21     /** Returns reference to the underlying map of outgoing edges. */
22     public Map<Vertex<V>, Edge<E>> getOutgoing() { return outgoing; }
23     /** Returns reference to the underlying map of incoming edges. */
24     public Map<Vertex<V>, Edge<E>> getIncoming() { return incoming; }
25 } //----- end of InnerVertex class -----
26
27 /** An edge between two vertices. */
28 private class InnerEdge<E> implements Edge<E> {
29     private E element;
30     private Position<Edge<E>> pos;
31     private Vertex<V>[] endpoints;
32     /** Constructs InnerEdge instance from u to v, storing the given element. */
33     public InnerEdge(Vertex<V> u, Vertex<V> v, E elem) {
34         element = elem;
35         endpoints = (Vertex<V>[]) new Vertex[ ]{u,v}; // array of length 2
36     }
37     /** Returns the element associated with the edge. */
38     public E getElement() { return element; }
39     /** Returns reference to the endpoint array. */
40     public Vertex<V>[] getEndpoints() { return endpoints; }
41     /** Stores the position of this edge within the graph's vertex list. */
42     public void setPosition(Position<Edge<E>> p) { pos = p; }
43     /** Returns the position of this edge within the graph's vertex list. */
44     public Position<Edge<E>> getPosition() { return pos; }
45 } //----- end of InnerEdge class -----

```

Code Fragment 14.1: InnerVertex and InnerEdge classes (to be nested within the AdjacencyMapGraph class). Interfaces Vertex<V> and Edge<E> are not shown.

```

1  public class AdjacencyMapGraph<V,E> implements Graph<V,E> {
2      // nested InnerVertex and InnerEdge classes defined here...
3      private boolean isDirected;
4      private PositionalList<Vertex<V>> vertices = new LinkedPositionalList<>();
5      private PositionalList<Edge<E>> edges = new LinkedPositionalList<>();
6      /** Constructs an empty graph (either undirected or directed). */
7      public AdjacencyMapGraph(boolean directed) { isDirected = directed; }
8      /** Returns the number of vertices of the graph */
9      public int numVertices() { return vertices.size(); }
10     /** Returns the vertices of the graph as an iterable collection */
11     public Iterable<Vertex<V>> vertices() { return vertices; }
12     /** Returns the number of edges of the graph */
13     public int numEdges() { return edges.size(); }
14     /** Returns the edges of the graph as an iterable collection */
15     public Iterable<Edge<E>> edges() { return edges; }
16     /** Returns the number of edges for which vertex v is the origin. */
17     public int outDegree(Vertex<V> v) {
18         InnerVertex<V> vert = validate(v);
19         return vert.getOutgoing().size();
20     }
21     /** Returns an iterable collection of edges for which vertex v is the origin. */
22     public Iterable<Edge<E>> outgoingEdges(Vertex<V> v) {
23         InnerVertex<V> vert = validate(v);
24         return vert.getOutgoing().values(); // edges are the values in the adjacency map
25     }
26     /** Returns the number of edges for which vertex v is the destination. */
27     public int inDegree(Vertex<V> v) {
28         InnerVertex<V> vert = validate(v);
29         return vert.getIncoming().size();
30     }
31     /** Returns an iterable collection of edges for which vertex v is the destination. */
32     public Iterable<Edge<E>> incomingEdges(Vertex<V> v) {
33         InnerVertex<V> vert = validate(v);
34         return vert.getIncoming().values(); // edges are the values in the adjacency map
35     }
36     public Edge<E> getEdge(Vertex<V> u, Vertex<V> v) {
37         /** Returns the edge from u to v, or null if they are not adjacent. */
38         InnerVertex<V> origin = validate(u);
39         return origin.getOutgoing().get(v); // will be null if no edge from u to v
40     }
41     /** Returns the vertices of edge e as an array of length two. */
42     public Vertex<V>[] endVertices(Edge<E> e) {
43         InnerEdge<E> edge = validate(e);
44         return edge.getEndpoints();
45     }

```

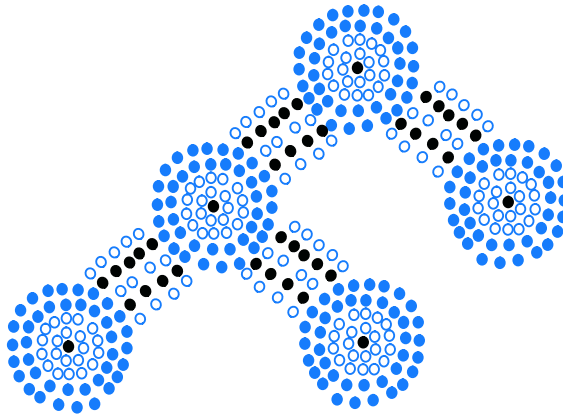
Code Fragment 14.2: AdjacencyMapGraph class definition. (Continues in Code Fragment 14.3.) The validate(v) and validate(e) methods are available online.

```

46  /** Returns the vertex that is opposite vertex v on edge e. */
47  public Vertex<V> opposite(Vertex<V> v, Edge<E> e)
48                                     throws IllegalArgumentException {
49      InnerEdge<E> edge = validate(e);
50      Vertex<V>[] endpoints = edge.getEndpoints();
51      if (endpoints[0] == v)
52          return endpoints[1];
53      else if (endpoints[1] == v)
54          return endpoints[0];
55      else
56          throw new IllegalArgumentException("v is not incident to this edge");
57  }
58  /** Inserts and returns a new vertex with the given element. */
59  public Vertex<V> insertVertex(V element) {
60      InnerVertex<V> v = new InnerVertex<>(element, isDirected);
61      v.setPosition(vertices.addLast(v));
62      return v;
63  }
64  /** Inserts and returns a new edge between u and v, storing given element. */
65  public Edge<E> insertEdge(Vertex<V> u, Vertex<V> v, E element)
66                                     throws IllegalArgumentException {
67      if (getEdge(u,v) == null) {
68          InnerEdge<E> e = new InnerEdge<>(u, v, element);
69          e.setPosition(edges.addLast(e));
70          InnerVertex<V> origin = validate(u);
71          InnerVertex<V> dest = validate(v);
72          origin.getOutgoing().put(v, e);
73          dest.getIncoming().put(u, e);
74          return e;
75      } else
76          throw new IllegalArgumentException("Edge from u to v exists");
77  }
78  /** Removes a vertex and all its incident edges from the graph. */
79  public void removeVertex(Vertex<V> v) {
80      InnerVertex<V> vert = validate(v);
81      // remove all incident edges from the graph
82      for (Edge<E> e : vert.getOutgoing().values())
83          removeEdge(e);
84      for (Edge<E> e : vert.getIncoming().values())
85          removeEdge(e);
86      // remove this vertex from the list of vertices
87      vertices.remove(vert.getPosition());
88  }
89  }

```

Code Fragment 14.3: AdjacencyMapGraph class definition (continued from Code Fragment 14.2). We omit the removeEdge method, for brevity.



Contents

8.1	General Trees	308
8.1.1	Tree Definitions and Properties	309
8.1.2	The Tree Abstract Data Type	312
8.1.3	Computing Depth and Height	314
8.2	Binary Trees	317
8.2.1	The Binary Tree Abstract Data Type	319
8.2.2	Properties of Binary Trees	321
8.3	Implementing Trees	323
8.3.1	Linked Structure for Binary Trees	323
8.3.2	Array-Based Representation of a Binary Tree	331
8.3.3	Linked Structure for General Trees	333
8.4	Tree Traversal Algorithms	334
8.4.1	Preorder and Postorder Traversals of General Trees	334
8.4.2	Breadth-First Tree Traversal	336
8.4.3	Inorder Traversal of a Binary Tree	337
8.4.4	Implementing Tree Traversals in Java	339
8.4.5	Applications of Tree Traversals	343
8.4.6	Euler Tours	348
8.5	Exercises	350

8.1 General Trees

Productivity experts say that breakthroughs come by thinking “nonlinearly.” In this chapter, we will discuss one of the most important nonlinear data structures in computing—*trees*. Tree structures are indeed a breakthrough in data organization, for they allow us to implement a host of algorithms much faster than when using linear data structures, such as arrays or linked lists. Trees also provide a natural organization for data, and consequently have become ubiquitous structures in file systems, graphical user interfaces, databases, websites, and many other computer systems.

It is not always clear what productivity experts mean by “nonlinear” thinking, but when we say that trees are “nonlinear,” we are referring to an organizational relationship that is richer than the simple “before” and “after” relationships between objects in sequences. The relationships in a tree are *hierarchical*, with some objects being “above” and some “below” others. Actually, the main terminology for tree data structures comes from family trees, with the terms “parent,” “child,” “ancestor,” and “descendant” being the most common words used to describe relationships. We show an example of a family tree in Figure 8.1.

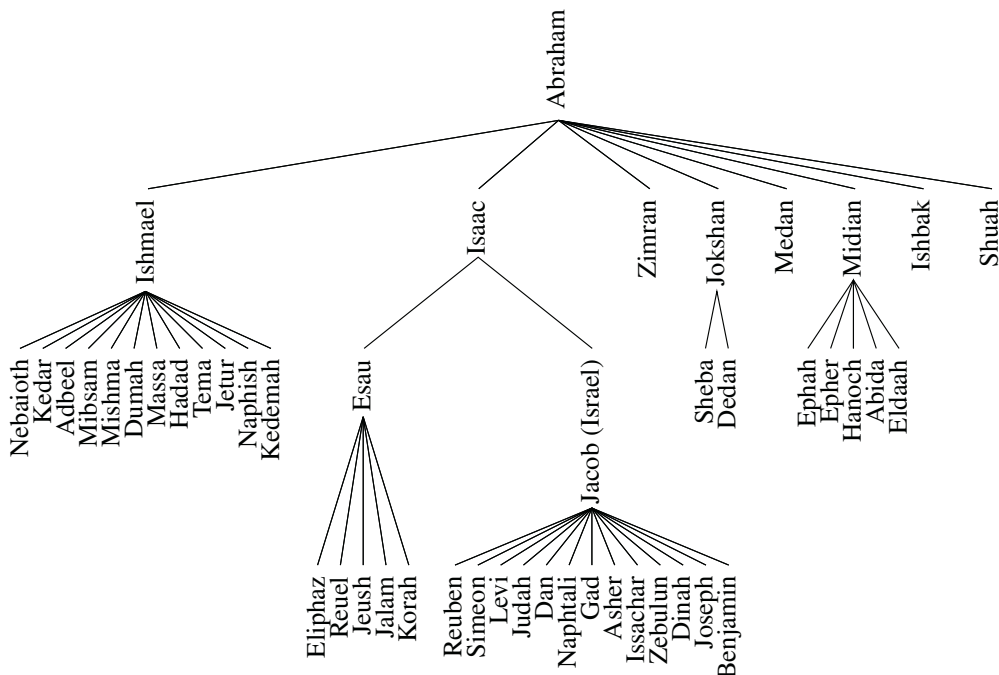


Figure 8.1: A family tree showing some descendants of Abraham, as recorded in Genesis, chapters 25–36.

8.1.1 Tree Definitions and Properties

A **tree** is an abstract data type that stores elements hierarchically. With the exception of the top element, each element in a tree has a **parent** element and zero or more **children** elements. A tree is usually visualized by placing elements inside ovals or rectangles, and by drawing the connections between parents and children with straight lines. (See Figure 8.2.) We typically call the top element the **root** of the tree, but it is drawn as the highest element, with the other elements being connected below (just the opposite of a botanical tree).

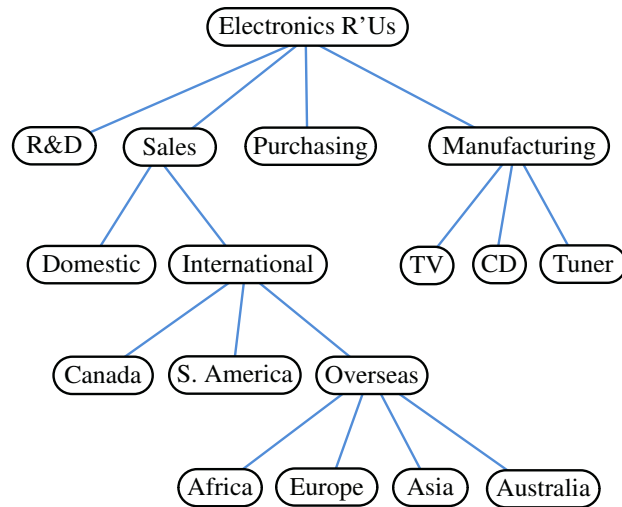


Figure 8.2: A tree with 17 nodes representing the organization of a fictitious corporation. The root stores *Electronics R'Us*. The children of the root store *R&D*, *Sales*, *Purchasing*, and *Manufacturing*. The internal nodes store *Sales*, *International*, *Overseas*, *Electronics R'Us*, and *Manufacturing*.

Formal Tree Definition

Formally, we define a **tree** T as a set of **nodes** storing elements such that the nodes have a **parent-child** relationship that satisfies the following properties:

- If T is nonempty, it has a special node, called the **root** of T , that has no parent.
- Each node v of T different from the root has a unique **parent** node w ; every node with parent w is a **child** of w .

Note that according to our definition, a tree can be empty, meaning that it does not have any nodes. This convention also allows us to define a tree recursively such that a tree T is either empty or consists of a node r , called the root of T , and a (possibly empty) set of subtrees whose roots are the children of r .

Other Node Relationships

Two nodes that are children of the same parent are **siblings**. A node v is **external** if v has no children. A node v is **internal** if it has one or more children. External nodes are also known as **leaves**.

Example 8.1: In Section 5.1.4, we discussed the hierarchical relationship between files and directories in a computer's file system, although at the time we did not emphasize the nomenclature of a file system as a tree. In Figure 8.3, we revisit an earlier example. We see that the internal nodes of the tree are associated with directories and the leaves are associated with regular files. In the Unix and Linux operating systems, the root of the tree is appropriately called the “root directory,” and is represented by the symbol “/.”

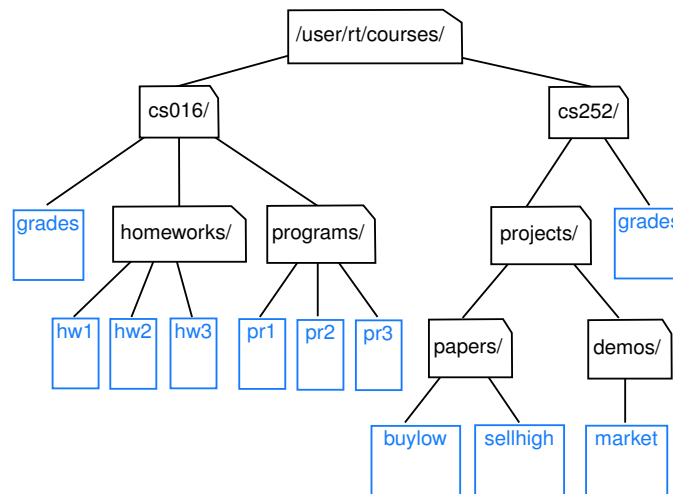


Figure 8.3: Tree representing a portion of a file system.

A node u is an **ancestor** of a node v if $u = v$ or u is an ancestor of the parent of v . Conversely, we say that a node v is a **descendant** of a node u if u is an ancestor of v . For example, in Figure 8.3, $cs252/$ is an ancestor of $papers/$, and $pr3$ is a descendant of $cs016/$. The **subtree** of T **rooted** at a node v is the tree consisting of all the descendants of v in T (including v itself). In Figure 8.3, the subtree rooted at $cs016/$ consists of the nodes $cs016/$, $grades$, $homeworks/$, $programs/$, $hw1$, $hw2$, $hw3$, $pr1$, $pr2$, and $pr3$.

Edges and Paths in Trees

An **edge** of tree T is a pair of nodes (u, v) such that u is the parent of v , or vice versa. A **path** of T is a sequence of nodes such that any two consecutive nodes in the sequence form an edge. For example, the tree in Figure 8.3 contains the path $(cs252/, projects/, demos/, market)$.

Ordered Trees

A tree is **ordered** if there is a meaningful linear order among the children of each node; that is, we purposefully identify the children of a node as being the first, second, third, and so on. Such an order is usually visualized by arranging siblings left to right, according to their order.

Example 8.2: *The components of a structured document, such as a book, are hierarchically organized as a tree whose internal nodes are parts, chapters, and sections, and whose leaves are paragraphs, tables, figures, and so on. (See Figure 8.4.) The root of the tree corresponds to the book itself. We could, in fact, consider expanding the tree further to show paragraphs consisting of sentences, sentences consisting of words, and words consisting of characters. Such a tree is an example of an ordered tree, because there is a well-defined order among the children of each node.*

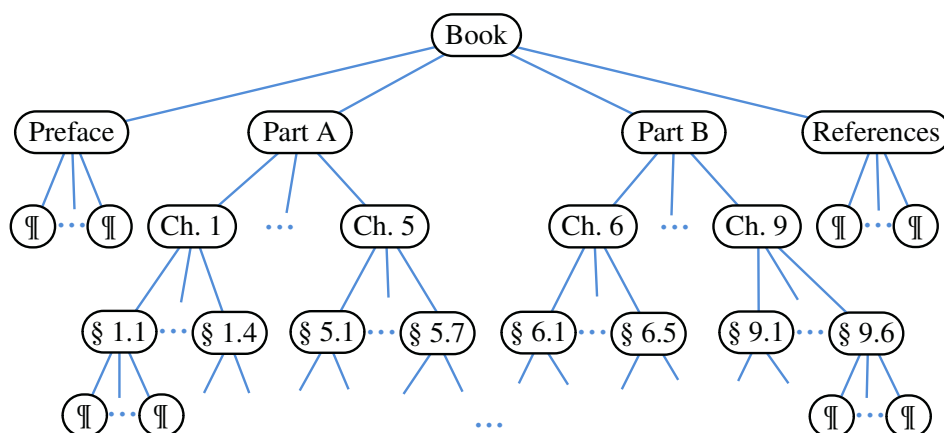


Figure 8.4: An ordered tree associated with a book.

Let's look back at the other examples of trees that we have described thus far, and consider whether the order of children is significant. A family tree that describes generational relationships, as in Figure 8.1, is often modeled as an ordered tree, with siblings ordered according to their birth.

In contrast, an organizational chart for a company, as in Figure 8.2, is typically considered an unordered tree. Likewise, when using a tree to describe an inheritance hierarchy, as in Figure 2.7, there is no particular significance to the order among the subclasses of a parent class. Finally, we consider the use of a tree in modeling a computer's file system, as in Figure 8.3. Although an operating system often displays entries of a directory in a particular order (e.g., alphabetical, chronological), such an order is not typically inherent to the file system's representation.

8.1.2 The Tree Abstract Data Type

As we did with positional lists in Section 7.3, we define a tree ADT using the concept of a *position* as an abstraction for a node of a tree. An element is stored at each position, and positions satisfy parent-child relationships that define the tree structure. A position object for a tree supports the method:

`getElement()`: Returns the element stored at this position.

The tree ADT then supports the following *accessor methods*, allowing a user to navigate the various positions of a tree T :

`root()`: Returns the position of the root of the tree (or null if empty).

`parent( $p$ )`: Returns the position of the parent of position p (or null if p is the root).

`children( $p$ )`: Returns an iterable collection containing the children of position p (if any).

`numChildren( $p$ )`: Returns the number of children of position p .

If a tree T is ordered, then `children( $p$ )` reports the children of p in order.

In addition to the above fundamental accessor methods, a tree supports the following *query methods*:

`isInternal( $p$ )`: Returns true if position p has at least one child.

`isExternal( $p$ )`: Returns true if position p does not have any children.

`isRoot( $p$ )`: Returns true if position p is the root of the tree.

These methods make programming with trees easier and more readable, since we can use them in the conditionals of **if** statements and **while** loops.

Trees support a number of more general methods, unrelated to the specific structure of the tree. These include:

`size()`: Returns the number of positions (and hence elements) that are contained in the tree.

`isEmpty()`: Returns true if the tree does not contain any positions (and thus no elements).

`iterator()`: Returns an iterator for all elements in the tree (so that the tree itself is iterable).

`positions()`: Returns an iterable collection of all positions of the tree.

If an invalid position is sent as a parameter to any method of a tree, then an `IllegalArgumentException` is thrown.

We do not define any methods for creating or modifying trees at this point. We prefer to describe different tree update methods in conjunction with specific implementations of the tree interface, and specific applications of trees.

A Tree Interface in Java

In Code Fragment 8.1, we formalize the Tree ADT by defining the Tree interface in Java. We rely upon the same definition of the Position interface as introduced for positional lists in Section 7.3.2. Note well that we declare the Tree interface to formally extend Java's Iterable interface (and we include a declaration of the required iterator() method).

```

1  /** An interface for a tree where nodes can have an arbitrary number of children. */
2  public interface Tree<E> extends Iterable<E> {
3      Position<E> root();
4      Position<E> parent(Position<E> p) throws IllegalArgumentException;
5      Iterable<Position<E>> children(Position<E> p)
6          throws IllegalArgumentException;
7      int numChildren(Position<E> p) throws IllegalArgumentException;
8      boolean isInternal(Position<E> p) throws IllegalArgumentException;
9      boolean isExternal(Position<E> p) throws IllegalArgumentException;
10     boolean isRoot(Position<E> p) throws IllegalArgumentException;
11     int size();
12     boolean isEmpty();
13     Iterator<E> iterator();
14     Iterable<Position<E>> positions();
15 }

```

Code Fragment 8.1: Definition of the Tree interface.

An AbstractTree Base Class in Java

In Section 2.3, we discussed the role of interfaces and abstract classes in Java. While an interface is a type definition that includes public declarations of various methods, an interface cannot include definitions for any of those methods. In contrast, an *abstract class* may define concrete implementations for some of its methods, while leaving other abstract methods without definition.

An abstract class is designed to serve as a base class, through inheritance, for one or more concrete implementations of an interface. When some of the functionality of an interface is implemented in an abstract class, less work remains to complete a concrete implementation. The standard Java libraries include many such abstract classes, including several within the Java Collections Framework. To make their purpose clear, those classes are conventionally named beginning with the word Abstract. For example, there is an AbstractCollection class that implements some of the functionality of the Collection interface, an AbstractQueue class that implements some of the functionality of the Queue interface, and an AbstractList class that implements some of the functionality of the List interface.

In the case of our Tree interface, we will define an AbstractTree base class, demonstrating how many tree-based algorithms can be described independently of the low-level representation of a tree data structure. In fact, if a concrete implementation provides three fundamental methods—`root()`, `parent(p)`, and `children(p)`—all other behaviors of the Tree interface can be derived within the AbstractTree base class.

Code Fragment 8.2 presents an initial implementation of an AbstractTree base class that provides the most trivial methods of the Tree interface. We will defer until Section 8.4 a discussion of general tree-traversal algorithms that can be used to produce the `positions()` iteration within the AbstractTree class. As with our positional list ADT in Chapter 7, the iteration of *positions* of the tree can easily be adapted to produce an iteration of the *elements* of a tree, or even to determine the size of a tree (although our concrete tree implementations will provide more direct means for reporting the size).

```

1  /** An abstract base class providing some functionality of the Tree interface. */
2  public abstract class AbstractTree<E> implements Tree<E> {
3      public boolean isInternal(Position<E> p) { return numChildren(p) > 0; }
4      public boolean isExternal(Position<E> p) { return numChildren(p) == 0; }
5      public boolean isRoot(Position<E> p) { return p == root(); }
6      public boolean isEmpty() { return size() == 0; }
7  }
```

Code Fragment 8.2: An initial implementation of the AbstractTree base class. (We add additional functionality to this class as the chapter continues.)

8.1.3 Computing Depth and Height

Let *p* be a position within tree *T*. The **depth** of *p* is the number of ancestors of *p*, other than *p* itself. For example, in the tree of Figure 8.2, the node storing *International* has depth 2. Note that this definition implies that the depth of the root of *T* is 0. The depth of *p* can also be recursively defined as follows:

- If *p* is the root, then the depth of *p* is 0.
- Otherwise, the depth of *p* is one plus the depth of the parent of *p*.

Based on this definition, we present a simple recursive algorithm, `depth`, in Code Fragment 8.3, for computing the depth of a position *p* in tree *T*. This method calls itself recursively on the parent of *p*, and adds 1 to the value returned.

The running time of `depth(p)` for position *p* is $O(d_p + 1)$, where d_p denotes the depth of *p* in the tree, because the algorithm performs a constant-time recursive step for each ancestor of *p*. Thus, algorithm `depth(p)` runs in $O(n)$ worst-case time, where *n* is the total number of positions of *T*, because a position of *T* may have depth *n* − 1 if all nodes form a single branch. Although such a running time is a function of the input size, it is more informative to characterize the running time in terms of the parameter d_p , as this parameter may be much smaller than *n*.

```

1  /** Returns the number of levels separating Position p from the root. */
2  public int depth(Position<E> p) {
3      if (isRoot(p))
4          return 0;
5      else
6          return 1 + depth(parent(p));
7  }

```

Code Fragment 8.3: Method `depth`, as implemented within the `AbstractTree` class.

Height

We next define the **height** of a tree to be equal to the maximum of the depths of its positions (or zero, if the tree is empty). For example, the tree of Figure 8.2 has height 4, as the node storing *Africa* (and its siblings) has depth 4. It is easy to see that the position with maximum depth must be a leaf.

In Code Fragment 8.4, we present a method that computes the height of a tree based on this definition. Unfortunately, such an approach is not very efficient, and so name the algorithm `heightBad` and declare it as a private method of the `AbstractTree` class (so that it cannot be used by others).

```

1  /** Returns the height of the tree. */
2  private int heightBad() {                                     // works, but quadratic worst-case time
3      int h = 0;
4      for (Position<E> p : positions())
5          if (isExternal(p))                                   // only consider leaf positions
6              h = Math.max(h, depth(p));
7      return h;
8  }

```

Code Fragment 8.4: Method `heightBad` of the `AbstractTree` class. Note that this method calls the `depth` method from Code Fragment 8.3.

Although we have not yet defined the `positions()` method, we will see that it can be implemented such that the entire iteration runs in $O(n)$ time, where n is the number of positions of T . Because `heightBad` calls algorithm `depth(p)` on each leaf of T , its running time is $O(n + \sum_{p \in L} (d_p + 1))$, where L is the set of leaf positions of T . In the worst case, the sum $\sum_{p \in L} (d_p + 1)$ is proportional to n^2 . (See Exercise C-8.31.) Thus, algorithm `heightBad` runs in $O(n^2)$ worst-case time.

We can compute the height of a tree more efficiently, in $O(n)$ worst-case time, by considering a recursive definition. To do this, we will parameterize a function based on a position within the tree, and calculate the height of the subtree rooted at that position. Formally, we define the **height** of a position p in a tree T as follows:

- If p is a leaf, then the height of p is 0.
- Otherwise, the height of p is one more than the maximum of the heights of p 's children.

The following proposition relates our original definition of the height of a tree to the height of the *root* position using this recursive formula.

Proposition 8.3: *The height of the root of a nonempty tree T , according to the recursive definition, equals the maximum depth among all leaves of tree T .*

We leave the justification of this proposition as Exercise R-8.3.

An implementation of a recursive algorithm to compute the height of a subtree rooted at a given position p is presented in Code Fragment 8.5. The overall height of a nonempty tree can be computed by sending the root of the tree as a parameter.

```

1  /** Returns the height of the subtree rooted at Position p. */
2  public int height(Position<E> p) {
3      int h = 0;                                // base case if p is external
4      for (Position<E> c : children(p))
5          h = Math.max(h, 1 + height(c));
6      return h;
7  }
```

Code Fragment 8.5: Method height for computing the height of a subtree rooted at a position p of an AbstractTree.

It is important to understand why method height is more efficient than method heightBad. The algorithm is recursive, and it progresses in a top-down fashion. If the method is initially called on the root of T , it will eventually be called once for each position of T . This is because the root eventually invokes the recursion on each of its children, which in turn invokes the recursion on each of their children, and so on.

We can determine the running time of the recursive height algorithm by summing, over all the positions, the amount of time spent on the nonrecursive part of each call. (Review Section 5.2 for analyses of recursive processes.) In our implementation, there is a constant amount of work per position, plus the overhead of computing the maximum over the iteration of children. Although we do not yet have a concrete implementation of children(p), we assume that such an iteration is executed in $O(c_p + 1)$ time, where c_p denotes the number of children of p . Algorithm height(p) spends $O(c_p + 1)$ time at each position p to compute the maximum, and its overall running time is $O(\sum_p (c_p + 1)) = O(n + \sum_p c_p)$. In order to complete the analysis, we make use of the following property.

Proposition 8.4: *Let T be a tree with n positions, and let c_p denote the number of children of a position p of T . Then, summing over the positions of T , $\sum_p c_p = n - 1$.*

Justification: Each position of T , with the exception of the root, is a child of another position, and thus contributes one unit to the above sum. ■

By Proposition 8.4, the running time of algorithm height, when called on the root of T , is $O(n)$, where n is the number of positions of T .

8.3 Implementing Trees

The `AbstractTree` and `AbstractBinaryTree` classes that we have defined thus far in this chapter are both *abstract base classes*. Although they provide a great deal of support, neither of them can be directly instantiated. We have not yet defined key implementation details for how a tree will be represented internally, and how we can effectively navigate between parents and children.

There are several choices for the internal representation of trees. We describe the most common representations in this section. We begin with the case of a *binary tree*, since its shape is more strictly defined.

8.3.1 Linked Structure for Binary Trees

A natural way to realize a binary tree T is to use a *linked structure*, with a node (see Figure 8.9a) that maintains references to the element stored at a position p and to the nodes associated with the children and parent of p . If p is the root of T , then the parent field of p is null. Likewise, if p does not have a left child (respectively, right child), the associated field is null. The tree itself maintains an instance variable storing a reference to the root node (if any), and a variable, called *size*, that represents the overall number of nodes of T . We show such a linked structure representation of a binary tree in Figure 8.9b.

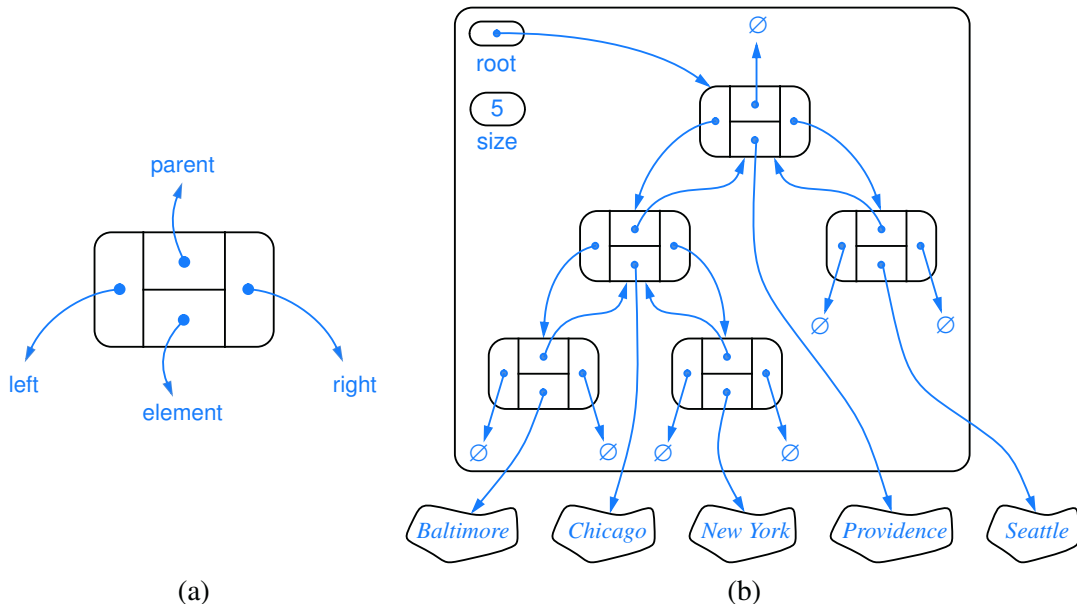


Figure 8.9: A linked structure for representing: (a) a single node; (b) a binary tree.

Operations for Updating a Linked Binary Tree

The `Tree` and `BinaryTree` interfaces define a variety of methods for inspecting an existing tree, yet they do not declare any update methods. Presuming that a newly constructed tree is empty, we would like to have means for changing the structure of content of a tree.

Although the principle of encapsulation suggests that the outward behaviors of an abstract data type need not depend on the internal representation, the *efficiency* of the operations depends greatly upon the representation. We therefore prefer to have each concrete implementation of a tree class support the most suitable behaviors for updating a tree. In the case of a linked binary tree, we suggest that the following update methods be supported:

- `addRoot(e)`: Creates a root for an empty tree, storing *e* as the element, and returns the position of that root; an error occurs if the tree is not empty.
- `addLeft(p, e)`: Creates a left child of position *p*, storing element *e*, and returns the position of the new node; an error occurs if *p* already has a left child.
- `addRight(p, e)`: Creates a right child of position *p*, storing element *e*, and returns the position of the new node; an error occurs if *p* already has a right child.
- `set(p, e)`: Replaces the element stored at position *p* with element *e*, and returns the previously stored element.
- `attach(p, T1, T2)`: Attaches the internal structure of trees *T*₁ and *T*₂ as the respective left and right subtrees of leaf position *p* and resets *T*₁ and *T*₂ to empty trees; an error condition occurs if *p* is not a leaf.
- `remove(p)`: Removes the node at position *p*, replacing it with its child (if any), and returns the element that had been stored at *p*; an error occurs if *p* has two children.

We have specifically chosen this collection of operations because each can be implemented in $O(1)$ worst-case time with our linked representation. The most complex of these are `attach` and `remove`, due to the case analyses involving the various parent-child relationships and boundary conditions, yet there remains only a constant number of operations to perform. (The implementation of both methods could be greatly simplified if we used a tree representation with a sentinel node, akin to our treatment of positional lists; see Exercise C-8.38.)

Java Implementation of a Linked Binary Tree Structure

We now present a concrete implementation of a `LinkedBinaryTree` class that implements the binary tree ADT, and supports the update methods described on the previous page. The new class formally extends the `AbstractBinaryTree` base class, inheriting several concrete implementations of methods from that class (as well as the formal designation that it implements the `BinaryTree` interface).

The low-level details of our linked tree implementation are reminiscent of techniques used when implementing the `LinkedPositionalList` class in Section 7.3.3. We define a nonpublic nested `Node` class to represent a node, and to serve as a `Position` for the public interface. As was portrayed in Figure 8.9, a node maintains a reference to an element, as well as references to its parent, its left child, and its right child (any of which might be null). The tree instance maintains a reference to the root node (possibly null), and a count of the number of nodes in the tree.

We also provide a `validate` utility that is called anytime a `Position` is received as a parameter, to ensure that it is a valid node. In the case of a linked tree, we adopt a convention in which we set a node's parent pointer to itself when it is removed from a tree, so that we can later recognize it as an invalid position.

The entire `LinkedBinaryTree` class is presented in Code Fragments 8.8–8.11. We provide the following guide to that code:

- Code Fragment 8.8 contains the definition of the nested `Node` class, which implements the `Position` interface. It also defines a method, `createNode`, that returns a new node instance. Such a design uses what is known as the **factory method pattern**, allowing us to later subclass our tree in order to use a specialized node type. (See Section 11.2.1.) Code Fragment 8.8 concludes with the declaration of the instance variables of the outer `LinkedBinaryTree` class and its constructor.
- Code Fragment 8.9 includes the protected `validate(p)` method, followed by the accessors `size`, `root`, `left`, and `right`. We note that all other methods of the `Tree` and `BinaryTree` interfaces are derived from these four concrete methods, via the `AbstractTree` and `AbstractBinaryTree` base classes.
- Code Fragments 8.10 and 8.11 provide the six update methods for a linked binary tree, as described on the preceding page. We note that the three methods—`addRoot`, `addLeft`, and `addRight`—each rely on use of the factory method, `createNode`, to produce a new node instance.

The `remove` method, given at the end of Code Fragment 8.11, intentionally sets the parent field of a deleted node to refer to itself, in accordance with our conventional representation of a defunct node (as detected within the `validate` method). It resets all other fields to null, to aid in garbage collection.

```

1  /** Concrete implementation of a binary tree using a node-based, linked structure. */
2  public class LinkedBinaryTree<E> extends AbstractBinaryTree<E> {
3
4      //----- nested Node class -----
5      protected static class Node<E> implements Position<E> {
6          private E element;                // an element stored at this node
7          private Node<E> parent;            // a reference to the parent node (if any)
8          private Node<E> left;              // a reference to the left child (if any)
9          private Node<E> right;            // a reference to the right child (if any)
10         /** Constructs a node with the given element and neighbors. */
11         public Node(E e, Node<E> above, Node<E> leftChild, Node<E> rightChild) {
12             element = e;
13             parent = above;
14             left = leftChild;
15             right = rightChild;
16         }
17         // accessor methods
18         public E getElement() { return element; }
19         public Node<E> getParent() { return parent; }
20         public Node<E> getLeft() { return left; }
21         public Node<E> getRight() { return right; }
22         // update methods
23         public void setElement(E e) { element = e; }
24         public void setParent(Node<E> parentNode) { parent = parentNode; }
25         public void setLeft(Node<E> leftChild) { left = leftChild; }
26         public void setRight(Node<E> rightChild) { right = rightChild; }
27     } //----- end of nested Node class -----
28
29     /** Factory function to create a new node storing element e. */
30     protected Node<E> createNode(E e, Node<E> parent,
31                                   Node<E> left, Node<E> right) {
32         return new Node<E>(e, parent, left, right);
33     }
34
35     // LinkedBinaryTree instance variables
36     protected Node<E> root = null;        // root of the tree
37     private int size = 0;                  // number of nodes in the tree
38
39     // constructor
40     public LinkedBinaryTree() { }          // constructs an empty binary tree

```

Code Fragment 8.8: An implementation of the `LinkedBinaryTree` class.
(Continues in Code Fragments 8.9–8.11.)

```

41 // nonpublic utility
42 /** Validates the position and returns it as a node. */
43 protected Node<E> validate(Position<E> p) throws IllegalArgumentException {
44     if (!(p instanceof Node))
45         throw new IllegalArgumentException("Not valid position type");
46     Node<E> node = (Node<E>) p; // safe cast
47     if (node.getParent() == node) // our convention for defunct node
48         throw new IllegalArgumentException("p is no longer in the tree");
49     return node;
50 }
51
52 // accessor methods (not already implemented in AbstractBinaryTree)
53 /** Returns the number of nodes in the tree. */
54 public int size() {
55     return size;
56 }
57
58 /** Returns the root Position of the tree (or null if tree is empty). */
59 public Position<E> root() {
60     return root;
61 }
62
63 /** Returns the Position of p's parent (or null if p is root). */
64 public Position<E> parent(Position<E> p) throws IllegalArgumentException {
65     Node<E> node = validate(p);
66     return node.getParent();
67 }
68
69 /** Returns the Position of p's left child (or null if no child exists). */
70 public Position<E> left(Position<E> p) throws IllegalArgumentException {
71     Node<E> node = validate(p);
72     return node.getLeft();
73 }
74
75 /** Returns the Position of p's right child (or null if no child exists). */
76 public Position<E> right(Position<E> p) throws IllegalArgumentException {
77     Node<E> node = validate(p);
78     return node.getRight();
79 }

```

Code Fragment 8.9: An implementation of the `LinkedBinaryTree` class.
(Continued from Code Fragment 8.8; continues in Code Fragments 8.10 and 8.11.)

```

80 // update methods supported by this class
81 /** Places element e at the root of an empty tree and returns its new Position. */
82 public Position<E> addRoot(E e) throws IllegalStateException {
83     if (!isEmpty()) throw new IllegalStateException("Tree is not empty");
84     root = createNode(e, null, null, null);
85     size = 1;
86     return root;
87 }
88
89 /** Creates a new left child of Position p storing element e; returns its Position. */
90 public Position<E> addLeft(Position<E> p, E e)
91     throws IllegalArgumentException {
92     Node<E> parent = validate(p);
93     if (parent.getLeft() != null)
94         throw new IllegalArgumentException("p already has a left child");
95     Node<E> child = createNode(e, parent, null, null);
96     parent.setLeft(child);
97     size++;
98     return child;
99 }
100
101 /** Creates a new right child of Position p storing element e; returns its Position. */
102 public Position<E> addRight(Position<E> p, E e)
103     throws IllegalArgumentException {
104     Node<E> parent = validate(p);
105     if (parent.getRight() != null)
106         throw new IllegalArgumentException("p already has a right child");
107     Node<E> child = createNode(e, parent, null, null);
108     parent.setRight(child);
109     size++;
110     return child;
111 }
112
113 /** Replaces the element at Position p with e and returns the replaced element. */
114 public E set(Position<E> p, E e) throws IllegalArgumentException {
115     Node<E> node = validate(p);
116     E temp = node.getElement();
117     node.setElement(e);
118     return temp;
119 }

```

Code Fragment 8.10: An implementation of the `LinkedBinaryTree` class.
(Continued from Code Fragments 8.8 and 8.9; continues in Code Fragment 8.11.)

```

120  /** Attaches trees t1 and t2 as left and right subtrees of external p. */
121  public void attach(Position<E> p, LinkedBinaryTree<E> t1,
122                  LinkedBinaryTree<E> t2) throws IllegalArgumentException {
123      Node<E> node = validate(p);
124      if (isInternal(p)) throw new IllegalArgumentException("p must be a leaf");
125      size += t1.size() + t2.size();
126      if (!t1.isEmpty()) { // attach t1 as left subtree of node
127          t1.root.setParent(node);
128          node.setLeft(t1.root);
129          t1.root = null;
130          t1.size = 0;
131      }
132      if (!t2.isEmpty()) { // attach t2 as right subtree of node
133          t2.root.setParent(node);
134          node.setRight(t2.root);
135          t2.root = null;
136          t2.size = 0;
137      }
138  }
139  /** Removes the node at Position p and replaces it with its child, if any. */
140  public E remove(Position<E> p) throws IllegalArgumentException {
141      Node<E> node = validate(p);
142      if (numChildren(p) == 2)
143          throw new IllegalArgumentException("p has two children");
144      Node<E> child = (node.getLeft() != null ? node.getLeft() : node.getRight());
145      if (child != null)
146          child.setParent(node.getParent()); // child's grandparent becomes its parent
147      if (node == root)
148          root = child; // child becomes root
149      else {
150          Node<E> parent = node.getParent();
151          if (node == parent.getLeft())
152              parent.setLeft(child);
153          else
154              parent.setRight(child);
155      }
156      size--;
157      E temp = node.getElement();
158      node.setElement(null); // help garbage collection
159      node.setLeft(null);
160      node.setRight(null);
161      node.setParent(null); // our convention for defunct node
162      return temp;
163  }
164  } //----- end of LinkedBinaryTree class -----

```

Code Fragment 8.11: An implementation of the `LinkedBinaryTree` class.
(Continued from Code Fragments 8.8–8.10.)

Performance of the Linked Binary Tree Implementation

To summarize the efficiencies of the linked structure representation, we analyze the running times of the `LinkedBinaryTree` methods, including derived methods that are inherited from the `AbstractTree` and `AbstractBinaryTree` classes:

- The `size` method, implemented in `LinkedBinaryTree`, uses an instance variable storing the number of nodes of a tree and therefore takes $O(1)$ time. Method `isEmpty`, inherited from `AbstractTree`, relies on a single call to `size` and thus takes $O(1)$ time.
- The accessor methods `root`, `left`, `right`, and `parent` are implemented directly in `LinkedBinaryTree` and take $O(1)$ time each. The `sibling`, `children`, and `numChildren` methods are derived in `AbstractBinaryTree` using on a constant number of calls to these other accessors, so they run in $O(1)$ time as well.
- The `isInternal` and `isExternal` methods, inherited from the `AbstractTree` class, rely on a call to `numChildren`, and thus run in $O(1)$ time as well. The `isRoot` method, also implemented in `AbstractTree`, relies on a comparison to the result of the `root` method and runs in $O(1)$ time.
- The update method, `set`, clearly runs in $O(1)$ time. More significantly, all of the methods `addRoot`, `addLeft`, `addRight`, `attach`, and `remove` run in $O(1)$ time, as each involves relinking only a constant number of parent-child relationships per operation.
- Methods `depth` and `height` were each analyzed in Section 8.1.3. The `depth` method at position p runs in $O(d_p + 1)$ time where d_p is its depth; the `height` method on the root of the tree runs in $O(n)$ time.

The overall space requirement of this data structure is $O(n)$, for a tree with n nodes, as there is an instance of the `Node` class for every node, in addition to the top-level `size` and `root` fields. Table 8.1 summarizes the performance of the linked structure implementation of a binary tree.

Method	Running Time
<code>size</code> , <code>isEmpty</code>	$O(1)$
<code>root</code> , <code>parent</code> , <code>left</code> , <code>right</code> , <code>sibling</code> , <code>children</code> , <code>numChildren</code>	$O(1)$
<code>isInternal</code> , <code>isExternal</code> , <code>isRoot</code>	$O(1)$
<code>addRoot</code> , <code>addLeft</code> , <code>addRight</code> , <code>set</code> , <code>attach</code> , <code>remove</code>	$O(1)$
<code>depth</code> (p)	$O(d_p + 1)$
<code>height</code>	$O(n)$

Table 8.1: Running times for the methods of an n -node binary tree implemented with a linked structure. The space usage is $O(n)$.

8.3.2 Array-Based Representation of a Binary Tree

An alternative representation of a binary tree T is based on a way of numbering the positions of T . For every position p of T , let $f(p)$ be the integer defined as follows.

- If p is the root of T , then $f(p) = 0$.
- If p is the left child of position q , then $f(p) = 2f(q) + 1$.
- If p is the right child of position q , then $f(p) = 2f(q) + 2$.

The numbering function f is known as a **level numbering** of the positions in a binary tree T , for it numbers the positions on each level of T in increasing order from left to right. (See Figure 8.10.) Note well that the level numbering is based on *potential* positions within a tree, not the actual shape of a specific tree, so they are not necessarily consecutive. For example, in Figure 8.10(b), there are no nodes with level numbering 13 or 14, because the node with level numbering 6 has no children.

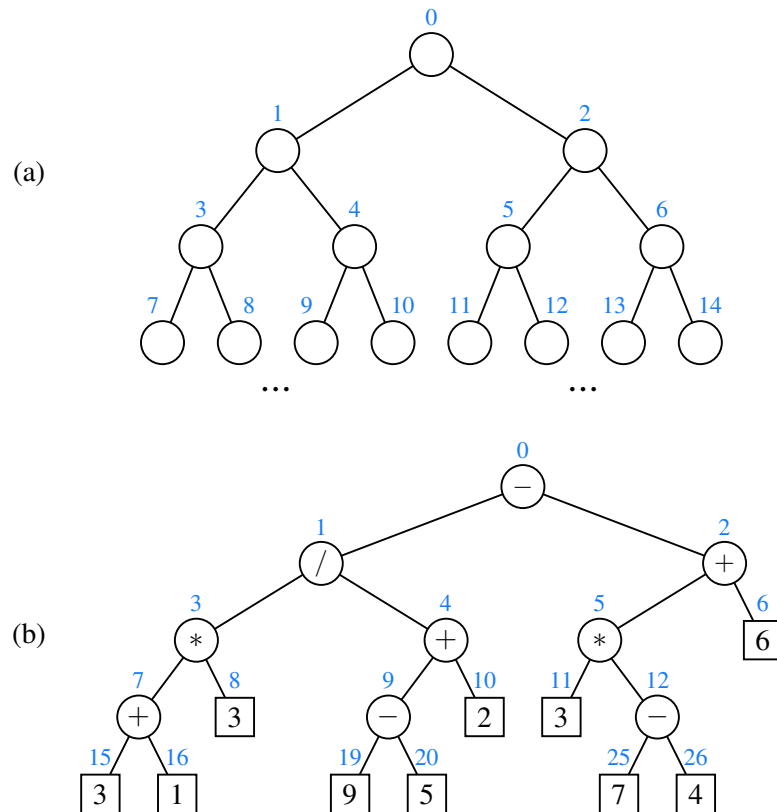


Figure 8.10: Binary tree level numbering: (a) general scheme; (b) an example.

The level numbering function f suggests a representation of a binary tree T by means of an array-based structure A , with the element at position p of T stored at index $f(p)$ of the array. We show an example of an array-based representation of a binary tree in Figure 8.11.

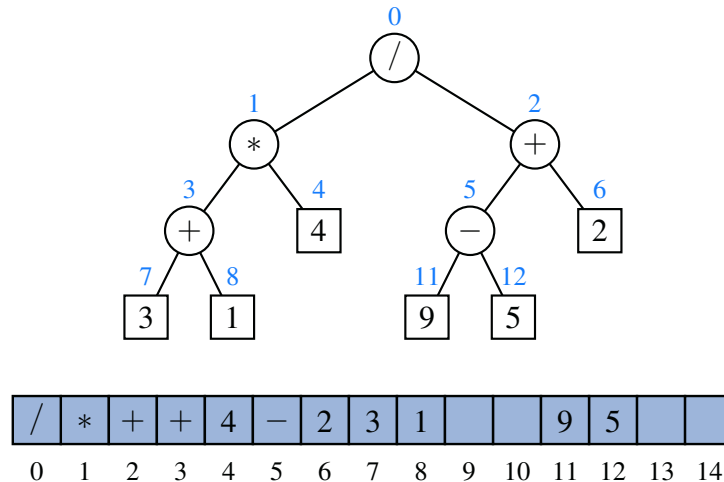


Figure 8.11: Representation of a binary tree by means of an array.

One advantage of an array-based representation of a binary tree is that a position p can be represented by the single integer $f(p)$, and that position-based methods such as `root`, `parent`, `left`, and `right` can be implemented using simple arithmetic operations on the number $f(p)$. Based on our formula for the level numbering, the left child of p has index $2f(p) + 1$, the right child of p has index $2f(p) + 2$, and the parent of p has index $\lfloor (f(p) - 1)/2 \rfloor$. We leave the details of a complete array-based implementation as Exercise R-8.16.

The space usage of an array-based representation depends greatly on the shape of the tree. Let n be the number of nodes of T , and let f_M be the maximum value of $f(p)$ over all the nodes of T . The array A requires length $N = 1 + f_M$, since elements range from $A[0]$ to $A[f_M]$. Note that A may have a number of empty cells that do not refer to existing positions of T . In fact, in the worst case, $N = 2^n - 1$, the justification of which is left as an exercise (R-8.14). In Section 9.3, we will see a class of binary trees, called “heaps” for which $N = n$. Thus, in spite of the worst-case space usage, there are applications for which the array representation of a binary tree is space efficient. Still, for general binary trees, the exponential worst-case space requirement of this representation is prohibitive.

Another drawback of an array representation is that many update operations for trees cannot be efficiently supported. For example, removing a node and promoting its child takes $O(n)$ time because it is not just the child that moves locations within the array, but all descendants of that child.

8.3.3 Linked Structure for General Trees

When representing a binary tree with a linked structure, each node explicitly maintains fields left and right as references to individual children. For a general tree, there is no a priori limit on the number of children that a node may have. A natural way to realize a general tree T as a linked structure is to have each node store a single *container* of references to its children. For example, a children field of a node can be an array or list of references to the children of the node (if any). Such a linked representation is schematically illustrated in Figure 8.12.

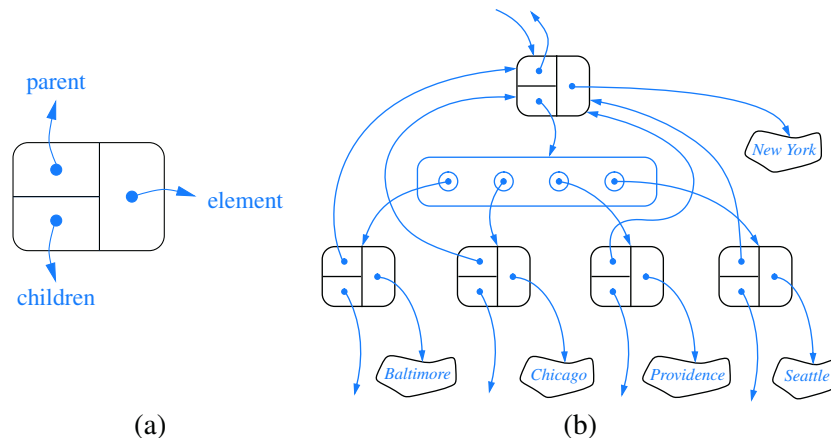


Figure 8.12: The linked structure for a general tree: (a) the structure of a node; (b) a larger portion of the data structure associated with a node and its children.

Table 8.2 summarizes the performance of the implementation of a general tree using a linked structure. The analysis is left as an exercise (R-8.13), but we note that, by using a collection to store the children of each position p , we can implement $\text{children}(p)$ by simply iterating that collection.

Method	Running Time
size, isEmpty	$O(1)$
root, parent, isRoot, isInternal, isExternal	$O(1)$
$\text{numChildren}(p)$	$O(1)$
$\text{children}(p)$	$O(c_p + 1)$
$\text{depth}(p)$	$O(d_p + 1)$
height	$O(n)$

Table 8.2: Running times of the accessor methods of an n -node general tree implemented with a linked structure. We let c_p denote the number of children of a position p , and d_p its depth. The space usage is $O(n)$.

8.4 Tree Traversal Algorithms

A **traversal** of a tree T is a systematic way of accessing, or “visiting,” all the positions of T . The specific action associated with the “visit” of a position p depends on the application of this traversal, and could involve anything from incrementing a counter to performing some complex computation for p . In this section, we describe several common traversal schemes for trees, implement them in the context of our various tree classes, and discuss several common applications of tree traversals.

8.4.1 Preorder and Postorder Traversals of General Trees

In a **preorder traversal** of a tree T , the root of T is visited first and then the subtrees rooted at its children are traversed recursively. If the tree is ordered, then the subtrees are traversed according to the order of the children. The pseudocode for the preorder traversal of the subtree rooted at a position p is shown in Code Fragment 8.12.

Algorithm preorder(p):

```

perform the “visit” action for position  $p$     { this happens before any recursion }
for each child  $c$  in children( $p$ ) do
    preorder( $c$ )                                { recursively traverse the subtree rooted at  $c$  }
```

Code Fragment 8.12: Algorithm preorder for performing the preorder traversal of a subtree rooted at position p of a tree.

Figure 8.13 portrays the order in which positions of a sample tree are visited during an application of the preorder traversal algorithm.

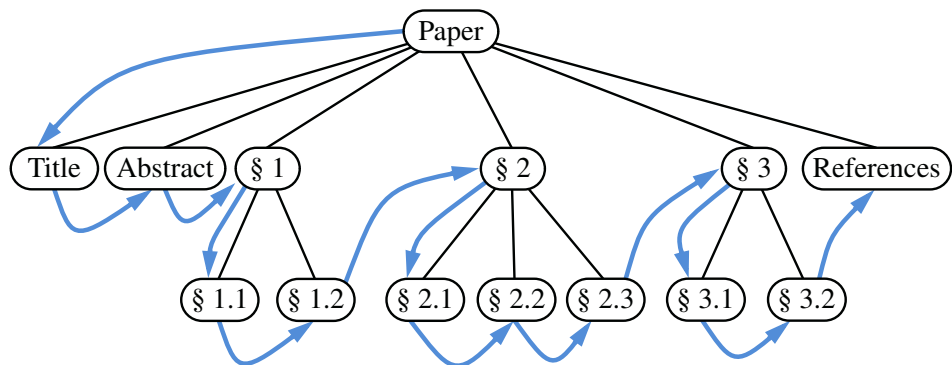


Figure 8.13: Preorder traversal of an ordered tree, where the children of each position are ordered from left to right.

Postorder Traversal

Another important tree traversal algorithm is the *postorder traversal*. In some sense, this algorithm can be viewed as the opposite of the preorder traversal, because it recursively traverses the subtrees rooted at the children of the root first, and then visits the root (hence, the name “postorder”). Pseudocode for the postorder traversal is given in Code Fragment 8.13, and an example of a postorder traversal is portrayed in Figure 8.14.

Algorithm postorder(p):

```

for each child  $c$  in children( $p$ ) do
    postorder( $c$ )           { recursively traverse the subtree rooted at  $c$  }
    perform the “visit” action for position  $p$  { this happens after any recursion }

```

Code Fragment 8.13: Algorithm postorder for performing the postorder traversal of a subtree rooted at position p of a tree.

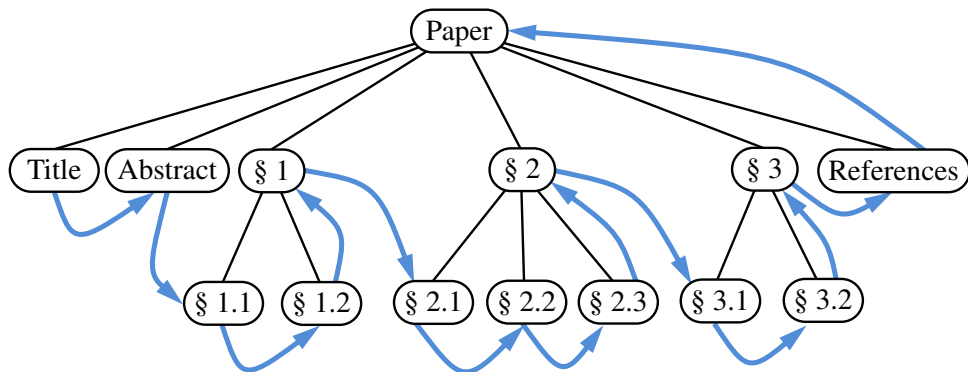


Figure 8.14: Postorder traversal of the ordered tree of Figure 8.13.

Running-Time Analysis

Both preorder and postorder traversal algorithms are efficient ways to access all the positions of a tree. The analysis of either of these traversal algorithms is similar to that of algorithm height, given in Code Fragment 8.5 of Section 8.1.3. At each position p , the nonrecursive part of the traversal algorithm requires time $O(c_p + 1)$, where c_p is the number of children of p , under the assumption that the “visit” itself takes $O(1)$ time. By Proposition 8.4, the overall running time for the traversal of tree T is $O(n)$, where n is the number of positions in the tree. This running time is asymptotically optimal since the traversal must visit all n positions of the tree.

8.4.2 Breadth-First Tree Traversal

Although the preorder and postorder traversals are common ways of visiting the positions of a tree, another approach is to traverse a tree so that we visit all the positions at depth d before we visit the positions at depth $d + 1$. Such an algorithm is known as a *breadth-first traversal*.

A breadth-first traversal is a common approach used in software for playing games. A *game tree* represents the possible choices of moves that might be made by a player (or computer) during a game, with the root of the tree being the initial configuration for the game. For example, Figure 8.15 displays a partial game tree for Tic-Tac-Toe.

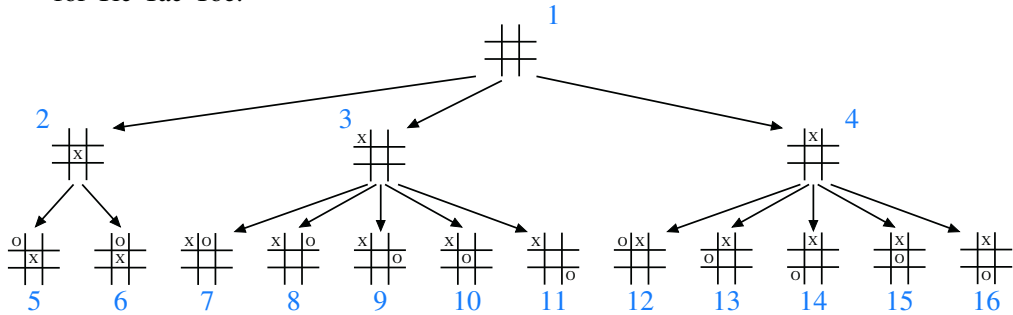


Figure 8.15: Partial game tree for Tic-Tac-Toe when ignoring symmetries; annotations denote the order in which positions are visited in a breadth-first tree traversal.

A breadth-first traversal of such a game tree is often performed because a computer may be unable to explore a complete game tree in a limited amount of time. So the computer will consider all moves, then responses to those moves, going as deep as computational time allows.

Pseudocode for a breadth-first traversal is given in Code Fragment 8.14. The process is not recursive, since we are not traversing entire subtrees at once. We use a queue to produce a FIFO (i.e., first-in first-out) semantics for the order in which we visit nodes. The overall running time is $O(n)$, due to the n calls to enqueue and n calls to dequeue.

Algorithm breadthfirst():

Initialize queue Q to contain root()

while Q not empty **do**

$p = Q.dequeue()$ { p is the oldest entry in the queue }

 perform the “visit” action for position p

for each child c in children(p) **do**

$Q.enqueue(c)$ { add p ’s children to the end of the queue for later visits }

Code Fragment 8.14: Algorithm for performing a breadth-first traversal of a tree.

8.4.3 Inorder Traversal of a Binary Tree

The standard preorder, postorder, and breadth-first traversals that were introduced for general trees can be directly applied to binary trees. In this section, we will introduce another common traversal algorithm specifically for a binary tree.

During an **inorder traversal**, we visit a position between the recursive traversals of its left and right subtrees. The inorder traversal of a binary tree T can be informally viewed as visiting the nodes of T “from left to right.” Indeed, for every position p , the inorder traversal visits p after all the positions in the left subtree of p and before all the positions in the right subtree of p . Pseudocode for the inorder traversal algorithm is given in Code Fragment 8.15, and an example of an inorder traversal is portrayed in Figure 8.16.

Algorithm `inorder( $p$ ):`

```

if  $p$  has a left child  $lc$  then
    inorder( $lc$ )                                { recursively traverse the left subtree of  $p$  }
    perform the “visit” action for position  $p$ 
if  $p$  has a right child  $rc$  then
    inorder( $rc$ )                                { recursively traverse the right subtree of  $p$  }
```

Code Fragment 8.15: Algorithm `inorder` for performing an inorder traversal of a subtree rooted at position p of a binary tree.

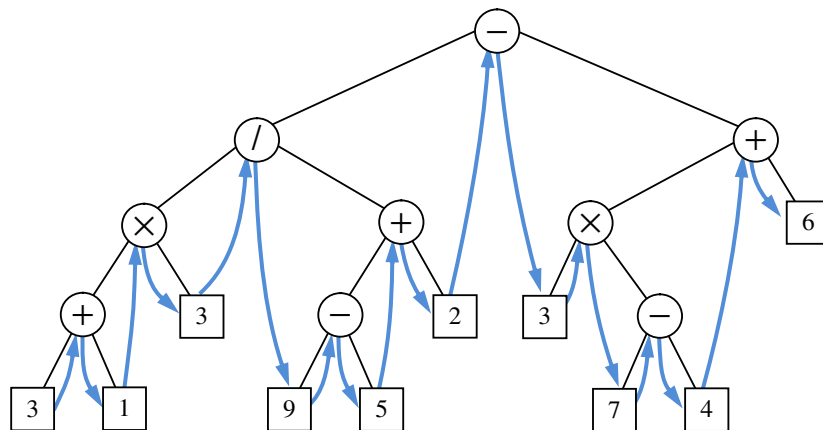


Figure 8.16: Inorder traversal of a binary tree.

The inorder traversal algorithm has several important applications. When using a binary tree to represent an arithmetic expression, as in Figure 8.16, the inorder traversal visits positions in a consistent order with the standard representation of the expression, as in $3 + 1 \times 3 / 9 - 5 + 2 \dots$ (albeit without parentheses).

Binary Search Trees

An important application of the inorder traversal algorithm arises when we store an ordered sequence of elements in a binary tree, defining a structure we call a **binary search tree**. Let S be a set whose unique elements have an order relation. For example, S could be a set of integers. A binary search tree for S is a proper binary tree T such that, for each internal position p of T :

- Position p stores an element of S , denoted as $e(p)$.
- Elements stored in the left subtree of p (if any) are less than $e(p)$.
- Elements stored in the right subtree of p (if any) are greater than $e(p)$.

An example of a binary search tree is shown in Figure 8.17. The above properties assure that an inorder traversal of a binary search tree T visits the elements in nondecreasing order.

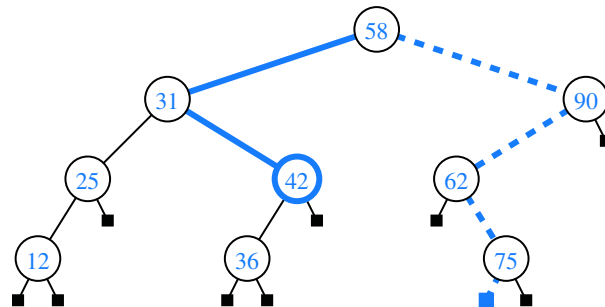


Figure 8.17: A binary search tree storing integers. The solid path is traversed when searching (successfully) for 42. The dashed path is traversed when searching (unsuccessfully) for 70.

We can use a binary search tree T for set S to find whether a given search value v is in S , by traversing a path down the tree T , starting at the root. At each internal position p encountered, we compare our search value v with the element $e(p)$ stored at p . If $v < e(p)$, then the search continues in the left subtree of p . If $v = e(p)$, then the search terminates successfully. If $v > e(p)$, then the search continues in the right subtree of p . Finally, if we reach a leaf, the search terminates unsuccessfully. In other words, a binary search tree can be viewed as a binary decision tree (recall Example 8.5), where the question asked at each internal node is whether the element at that node is less than, equal to, or larger than the element being searched for. We illustrate several examples of the search operation in Figure 8.17.

Note that the running time of searching in a binary search tree T is proportional to the height of T . Recall from Proposition 8.7 that the height of a binary tree with n nodes can be as small as $\log(n+1) - 1$ or as large as $n - 1$. Thus, binary search trees are most efficient when they have small height. Chapter 11 is devoted to the study of search trees.

8.4.4 Implementing Tree Traversals in Java

When first defining the tree ADT in Section 8.1.2, we stated that tree T must include the following supporting methods:

`iterator()`: Returns an iterator for all elements in the tree.

`positions()`: Returns an iterable collection of all positions of the tree.

At that time, we did not make any assumption about the order in which these iterations report their results. In this section, we will demonstrate how any of the tree traversal algorithms we have introduced can be used to produce these iterations as concrete implementations within the `AbstractTree` or `AbstractBinaryTree` base classes.

First, we note that an iteration of all *elements* of a tree can easily be produced if we have an iteration of all *positions* of that tree. Code Fragment 8.16 provides an implementation of the `iterator()` method by adapting an iteration produced by the `positions()` method. In fact, this is the identical approach we used in Code Fragment 7.14 of Section 7.4.2 for the `LinkedPositionalList` class.

```

1  //----- nested ElementIterator class -----
2  /* This class adapts the iteration produced by positions() to return elements. */
3  private class ElementIterator implements Iterator<E> {
4      Iterator<Position<E>> posIterator = positions().iterator();
5      public boolean hasNext() { return posIterator.hasNext(); }
6      public E next() { return posIterator.next().getElement(); } // return element!
7      public void remove() { posIterator.remove(); }
8  }
9
10 /** Returns an iterator of the elements stored in the tree. */
11 public Iterator<E> iterator() { return new ElementIterator(); }

```

Code Fragment 8.16: Iterating all elements of an `AbstractTree` instance, based upon an iteration of the positions of the tree.

To implement the `positions()` method, we have a choice of tree traversal algorithms. Given that there are advantages to each of those traversal orders, we provide public implementations of each strategy that can be called directly by a user of our class. We can then trivially adapt one of those as a default order for the `positions` method of the `AbstractTree` class. For example, on the following page we will define a public method, `preorder()`, that returns an iteration of the positions of a tree in preorder; Code Fragment 8.17 demonstrates how the `positions()` method can be trivially defined to rely on that order.

```
public Iterable<Position<E>> positions() { return preorder(); }
```

Code Fragment 8.17: Defining `preorder` as the default traversal algorithm for the public `positions` method of an abstract tree.

Preorder Traversals

We begin by considering the *preorder traversal* algorithm. Our goal is to provide a public method `preorder()`, as part of the `AbstractTree` class, which returns an iterable container of the positions of the tree in preorder. For ease of implementation, we choose to produce a *snapshot iterator*, as defined in Section 7.4.2, returning a list of all positions. (Exercise C-8.47 explores the goal of implementing a *lazy iterator* that reports positions in preorder.)

We begin by defining a private utility method, `preorderSubtree`, given in Code Fragment 8.18, which allows us to parameterize the recursive process with a specific position of the tree that serves as the root of a subtree to traverse. (We also pass a list as a parameter that serves as a buffer to which “visited” positions are added.)

```

1  /** Adds positions of the subtree rooted at Position p to the given snapshot. */
2  private void preorderSubtree(Position<E> p, List<Position<E>> snapshot) {
3      snapshot.add(p);      // for preorder, we add position p before exploring subtrees
4      for (Position<E> c : children(p))
5          preorderSubtree(c, snapshot);
6  }
```

Code Fragment 8.18: A recursive subroutine for performing a preorder traversal of the subtree rooted at position p of a tree. This code should be included within the body of the `AbstractTree` class.

The `preorderSubtree` method follows the high-level algorithm originally described as pseudocode in Code Fragment 8.12. It has an implicit base case, as the **for** loop body never executes if a position has no children.

The public `preorder` method, shown in Code Fragment 8.19, has the responsibility of creating an empty list for the snapshot buffer, and invoking the recursive method at the root of the tree (assuming the tree is nonempty). We rely on a `java.util.ArrayList` instance as an `Iterable` instance for the snapshot buffer.

```

1  /** Returns an iterable collection of positions of the tree, reported in preorder. */
2  public Iterable<Position<E>> preorder() {
3      List<Position<E>> snapshot = new ArrayList<>();
4      if (!isEmpty())
5          preorderSubtree(root(), snapshot);    // fill the snapshot recursively
6      return snapshot;
7  }
```

Code Fragment 8.19: A public method that performs a preorder traversal of an entire tree. This code should be included within the body of the `AbstractTree` class.

Postorder Traversal

We implement a *postorder traversal* using a similar design as we used for a pre-order traversal. The only difference is that a “visited” position is not added to a postorder snapshot until *after* all of its subtrees have been traversed. Both the recursive utility and the top-level public method are given in Code Fragment 8.20.

```

1  /** Adds positions of the subtree rooted at Position p to the given snapshot. */
2  private void postorderSubtree(Position<E> p, List<Position<E>> snapshot) {
3      for (Position<E> c : children(p))
4          postorderSubtree(c, snapshot);
5      snapshot.add(p);      // for postorder, we add position p after exploring subtrees
6  }
7  /** Returns an iterable collection of positions of the tree, reported in postorder. */
8  public Iterable<Position<E>> postorder() {
9      List<Position<E>> snapshot = new ArrayList<>();
10     if (!isEmpty())
11         postorderSubtree(root(), snapshot);    // fill the snapshot recursively
12     return snapshot;
13 }
```

Code Fragment 8.20: Support for performing a postorder traversal of a tree. This code should be included within the body of the AbstractTree class.

Breadth-First Traversal

On the following page, we will provide an implementation of the breadth-first traversal algorithm in the context of our AbstractTree class (Code Fragment 8.21). Recall that the breadth-first traversal algorithm is not recursive; it relies on a queue of positions to manage the traversal process. We will use the LinkedQueue class from Section 6.2.3, although any implementation of the queue ADT would suffice.

Inorder Traversal for Binary Trees

The preorder, postorder, and breadth-first traversal algorithms are applicable to all trees. The inorder traversal algorithm, because it explicitly relies on the notion of a left and right child of a node, only applies to binary trees. We therefore include its definition within the body of the AbstractBinaryTree class. We use a similar design to our preorder and postorder traversals, with a private recursive utility for traversing subtrees. (See Code Fragment 8.22.)

For many applications of binary trees (for example, see Chapter 11), an inorder traversal is the most natural order. Therefore, Code Fragment 8.22 makes it the default for the AbstractBinaryTree class by overriding the positions method that was inherited from the AbstractTree class. Because the iterator() method relies on positions(), it will also use inorder when reporting the elements of a binary tree.

```

1  /** Returns an iterable collection of positions of the tree in breadth-first order. */
2  public Iterable<Position<E>> breadthfirst() {
3      List<Position<E>> snapshot = new ArrayList<>();
4      if (!isEmpty()) {
5          Queue<Position<E>> fringe = new LinkedList<>();
6          fringe.enqueue(root());           // start with the root
7          while (!fringe.isEmpty()) {
8              Position<E> p = fringe.dequeue(); // remove from front of the queue
9              snapshot.add(p);                 // report this position
10             for (Position<E> c : children(p))
11                 fringe.enqueue(c);           // add children to back of queue
12         }
13     }
14     return snapshot;
15 }

```

Code Fragment 8.21: An implementation of a breadth-first traversal of a tree. This code should be included within the body of the `AbstractTree` class.

```

1  /** Adds positions of the subtree rooted at Position p to the given snapshot. */
2  private void inorderSubtree(Position<E> p, List<Position<E>> snapshot) {
3      if (left(p) != null)
4          inorderSubtree(left(p), snapshot);
5      snapshot.add(p);
6      if (right(p) != null)
7          inorderSubtree(right(p), snapshot);
8  }
9  /** Returns an iterable collection of positions of the tree, reported in inorder. */
10 public Iterable<Position<E>> inorder() {
11     List<Position<E>> snapshot = new ArrayList<>();
12     if (!isEmpty())
13         inorderSubtree(root(), snapshot); // fill the snapshot recursively
14     return snapshot;
15 }
16 /** Overrides positions to make inorder the default order for binary trees. */
17 public Iterable<Position<E>> positions() {
18     return inorder();
19 }

```

Code Fragment 8.22: Support for performing an inorder traversal of a binary tree, and for making that order the default traversal for binary trees. This code should be included within the body of the `AbstractBinaryTree` class.

8.4.5 Applications of Tree Traversals

In this section, we demonstrate several representative applications of tree traversals, including some customizations of the standard traversal algorithms.

Table of Contents

When using a tree to represent the hierarchical structure of a document, a preorder traversal of the tree can be used to produce a table of contents for the document. For example, the table of contents associated with the tree from Figure 8.13 is displayed in Figure 8.18. Part (a) of that figure gives a simple presentation with one element per line; part (b) shows a more attractive presentation, produced by indenting each element based on its depth within the tree.

Paper	Paper
Title	Title
Abstract	Abstract
§1	§1
§1.1	§1.1
§1.2	§1.2
§2	§2
§2.1	§2.1
...	...
(a)	(b)

Figure 8.18: Table of contents for a document represented by the tree in Figure 8.13: (a) without indentation; (b) with indentation based on depth within the tree.

The unindented version of the table of contents can be produced with the following code, given a tree *T* supporting the `preorder()` method:

```
for (Position<E> p : T.preorder())
    System.out.println(p.getElement());
```

To produce the presentation of Figure 8.18(b), we indent each element with a number of spaces equal to twice the element's depth in the tree (hence, the root element was unindented). If we assume that method, `spaces(n)`, produces a string of *n* spaces, we could replace the body of the above loop with the statement `System.out.println(spaces(2*T.depth(p)) + p.getElement())`. Unfortunately, although the work to produce the preorder traversal runs in $O(n)$ time, based on the analysis of Section 8.4.1, the calls to `depth` incur a hidden cost. Making a call to `depth` from every position of the tree results in $O(n^2)$ worst-case time, as noted when analyzing the algorithm `heightBad` in Section 8.1.3.

A preferred approach to producing an indented table of contents is to redesign a top-down recursion that includes the current depth as an additional parameter. Such an implementation is provided in Code Fragment 8.23. This implementation runs in worst-case $O(n)$ time (except, technically, the time it takes to print strings of increasing lengths).

```

1  /** Prints preorder representation of subtree of T rooted at p having depth d. */
2  public static <E> void printPreorderIndent(Tree<E> T, Position<E> p, int d) {
3      System.out.println(spaces(2*d) + p.getElement());    // indent based on d
4      for (Position<E> c : T.children(p))
5          printPreorderIndent(T, c, d+1);                  // child depth is d+1
6  }
```

Code Fragment 8.23: Efficient recursion for printing indented version of a preorder traversal. To print an entire tree T , the recursion should be started with form `printPreorderIndent(T, T.root(), 0)`.

In the example of Figure 8.18, we were fortunate in that the numbering was embedded within the elements of the tree. More generally, we might be interested in using a preorder traversal to display the structure of a tree, with indentation and also explicit numbering that was not present in the tree. For example, we might display the tree from Figure 8.2 beginning as:

```

Electronics R'Us
  1 R&D
  2 Sales
    2.1 Domestic
    2.2 International
      2.2.1 Canada
      2.2.2 S. America
```

This is more challenging, because the numbers used as labels are implicit in the structure of the tree. A label depends on the path from the root to the current position. To accomplish our goal, we add an additional parameter to the recursive signature. We send a list of integers representing the labels leading to a particular position. For example, when visiting the node *Domestic* above, we will send the list of values $\{2, 1\}$ that comprise its label.

At the implementation level, we wish to avoid the inefficiency of duplicating such lists when sending a new parameter from one level of the recursion to the next. A standard solution is to pass the same list instance throughout the recursion. At one level of the recursion, a new entry is temporarily added to the end of the list before making further recursive calls. In order to “leave no trace,” the extraneous entry must later be removed from the list by the same recursive call that added it. An implementation based on this approach is given in Code Fragment 8.24.

```

1  /** Prints labeled representation of subtree of T rooted at p having depth d. */
2  public static <E>
3  void printPreorderLabeled(Tree<E> T, Position<E> p, ArrayList<Integer> path) {
4      int d = path.size();           // depth equals the length of the path
5      System.out.print(spaces(2*d)); // print indentation, then label
6      for (int j=0; j < d; j++) System.out.print(path.get(j) + (j == d-1 ? " " : "."));
7      System.out.println(p.getElement());
8      path.add(1);                   // add path entry for first child
9      for (Position<E> c : T.children(p)) {
10         printPreorderLabeled(T, c, path);
11         path.set(d, 1 + path.get(d)); // increment last entry of path
12     }
13     path.remove(d);                 // restore path to its incoming state
14 }

```

Code Fragment 8.24: Efficient recursion for printing an indented and *labeled* presentation of a preorder traversal.

Computing Disk Space

In Example 8.1, we considered the use of a tree as a model for a file-system structure, with internal positions representing directories and leaves representing files. In fact, when introducing the use of recursion back in Chapter 5, we specifically examined the topic of file systems (see Section 5.1.4). Although we did not explicitly model it as a tree at that time, we gave an implementation of an algorithm for computing the disk usage (Code Fragment 5.5).

The recursive computation of disk space is emblematic of a *postorder* traversal, as we cannot effectively compute the total space used by a directory until *after* we know the space that is used by its children directories. Unfortunately, the formal implementation of postorder, as given in Code Fragment 8.20, does not suffice for this purpose. We would like to have a mechanism for children to return information to the parent as part of the traversal process. A custom solution to the disk space problem, with each level of recursion providing a return value to the (parent) caller, is provided in Code Fragment 8.25.

```

1  /** Returns total disk space for subtree of T rooted at p. */
2  public static int diskSpace(Tree<Integer> T, Position<Integer> p) {
3      int subtotal = p.getElement(); // we assume element represents space usage
4      for (Position<Integer> c : T.children(p))
5          subtotal += diskSpace(T, c);
6      return subtotal;
7  }

```

Code Fragment 8.25: Recursive computation of disk space for a tree. We assume that each tree element reports the local space used at that position.

Parenthetic Representations of a Tree

It is not possible to reconstruct a general tree, given only the preorder sequence of elements, as in Figure 8.18a. Some additional context is necessary for the structure of the tree to be well defined. The use of indentation or numbered labels provides such context, with a very human-friendly presentation. However, there are more concise string representations of trees that are computer-friendly.

In this section, we explore one such representation. The *parenthetic string representation* $P(T)$ of tree T is recursively defined. If T consists of a single position p , then $P(T) = p.getElement()$. Otherwise, it is defined recursively as,

$$P(T) = p.getElement() + "(" + P(T_1) + ", " + \dots + ", " + P(T_k) + ")"$$

where p is the root of T and $T_1, T_2, \dots, T_k$ are the subtrees rooted at the children of p , which are given in order if T is an ordered tree. We are using “+” here to denote string concatenation. As an example, the parenthetic representation of the tree of Figure 8.2 would appear as follows (line breaks are cosmetic):

```
Electronics R'Us (R&D, Sales (Domestic, International (Canada,
S. America, Overseas (Africa, Europe, Asia, Australia))),
Purchasing, Manufacturing (TV, CD, Tuner))
```

Although the parenthetic representation is essentially a preorder traversal, we cannot easily produce the additional punctuation using the formal implementation of preorder. The opening parenthesis must be produced just before the loop over a position's children, the separating commas between children, and the closing parenthesis just after the loop completes. The Java method `parenthesize`, shown in Code Fragment 8.26, is a custom traversal that prints such a parenthetic string representation of a tree T .

```
1  /** Prints parenthesized representation of subtree of T rooted at p. */
2  public static <E> void parenthesize(Tree<E> T, Position<E> p) {
3      System.out.print(p.getElement());
4      if (T.isInternal(p)) {
5          boolean firstTime = true;
6          for (Position<E> c : T.children(p)) {
7              System.out.print( (firstTime ? " (" : ", ")); // determine proper punctuation
8              firstTime = false;                             // any future passes will get comma
9              parenthesize(T, c);                             // recur on child
10         }
11         System.out.print(")");
12     }
13 }
```

Code Fragment 8.26: Method that prints parenthetic string representation of a tree.

Using Inorder Traversal for Tree Drawing

An inorder traversal can be applied to the problem of computing a graphical layout of a binary tree, as shown in Figure 8.19. We assume the convention, common to computer graphics, that x -coordinates increase left to right and y -coordinates increase top to bottom, so that the origin is in the upper left corner of the drawing.

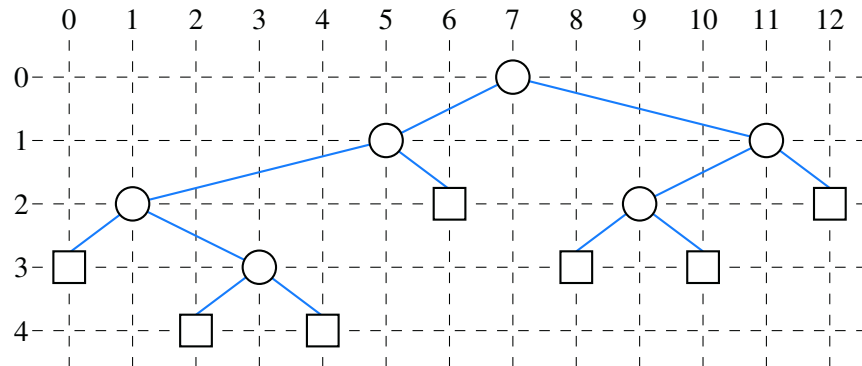


Figure 8.19: An inorder drawing of a binary tree.

The geometry is determined by an algorithm that assigns x - and y -coordinates to each position p of a binary tree T using the following two rules:

- $x(p)$ is the number of positions visited before p in an inorder traversal of T .
- $y(p)$ is the depth of p in T .

Code Fragment 8.27 provides an implementation of a recursive method that assigns x - and y -coordinates to positions of a tree in this manner. Depth information is passed from one level of the recursion to another, as done in our earlier example for indentation. To maintain an accurate value for the x -coordinate as the traversal proceeds, the method must be provided with the value of x that should be assigned to the leftmost node of the current subtree, and it must return to its parent a revised value of x that is appropriate for the first node drawn to the right of the subtree.

```

1  public static <E> int layout(BinaryTree<E> T, Position<E> p, int d, int x) {
2      if (T.left(p) != null)
3          x = layout(T, T.left(p), d+1, x);           // resulting x will be increased
4      p.getElement().setX(x++);                     // post-increment x
5      p.getElement().setY(d);
6      if (T.right(p) != null)
7          x = layout(T, T.right(p), d+1, x);         // resulting x will be increased
8      return x;
9  }
```

Code Fragment 8.27: Recursive method for computing coordinates at which to draw positions of a binary tree. We assume that the element type for the tree supports `setX` and `setY` methods. The initial call should be `layout(T, T.root(), 0, 0)`.

8.4.6 Euler Tours

The various applications described in Section 8.4.5 demonstrate the great power of recursive tree traversals, but they also show that not every application strictly fits the mold of a preorder, postorder, or inorder traversal. We can unify the tree-traversal algorithms into a single framework known as an **Euler tour traversal**. The Euler tour traversal of a tree T can be informally defined as a “walk” around T , where we start by going from the root toward its leftmost child, viewing the edges of T as being “walls” that we always keep to our left. (See Figure 8.20.)

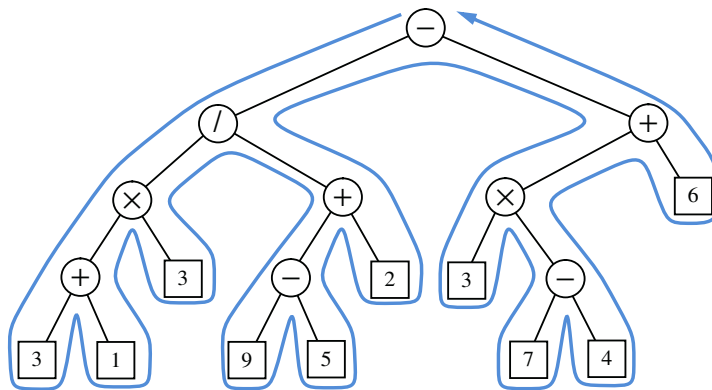


Figure 8.20: Euler tour traversal of a tree.

The complexity of the walk is $O(n)$, for a tree with n nodes, because it progresses exactly two times along each of the $n - 1$ edges of the tree—once going downward along the edge, and later going upward along the edge. To unify the concept of preorder and postorder traversals, we can view there being two notable “visits” to each position p :

- A “pre visit” occurs when first reaching the position, that is, when the walk passes immediately *left* of the node in our visualization.
- A “post visit” occurs when the walk later proceeds upward from that position, that is, when the walk passes to the *right* of the node in our visualization.

The process of an Euler tour can be naturally viewed as recursive. In between the “pre visit” and “post visit” of a given position will be a recursive tour of each of its subtrees. Looking at Figure 8.20 as an example, there is a contiguous portion of the entire tour that is itself an Euler tour of the subtree of the node with element “/”. That tour contains two contiguous subtours, one traversing that position’s left subtree and another traversing the right subtree.

In the special case of a binary tree, we can designate the time when the walk passes immediately *below* a node as an “in visit” event. This will be just after the tour of its left subtree (if any), but before the tour of its right subtree (if any).

The pseudocode for an Euler tour traversal of a subtree rooted at a position p is shown in Code Fragment 8.28.

Algorithm eulerTour(T, p):

```
perform the "pre visit" action for position  $p$ 
for each child  $c$  in  $T.children(p)$  do
    eulerTour( $T, c$ )           { recursively tour the subtree rooted at  $c$  }
perform the "post visit" action for position  $p$ 
```

Code Fragment 8.28: Algorithm eulerTour for performing an Euler tour traversal of a subtree rooted at position p of a tree.

The Euler tour traversal extends the preorder and postorder traversals, but it can also perform other kinds of traversals. For example, suppose we wish to compute the number of descendants of each position p in an n -node binary tree. We start an Euler tour by initializing a counter to 0, and then increment the counter during the "pre visit" for each position. To determine the number of descendants of a position p , we compute the difference between the values of the counter from when the pre-visit occurs and when the post-visit occurs, and add 1 (for p). This simple rule gives us the number of descendants of p , because each node in the subtree rooted at p is counted between p 's visit on the left and p 's visit on the right. Therefore, we have an $O(n)$ -time method for computing the number of descendants of each node.

For the case of a binary tree, we can customize the algorithm to include an explicit "in visit" action, as shown in Code Fragment 8.29.

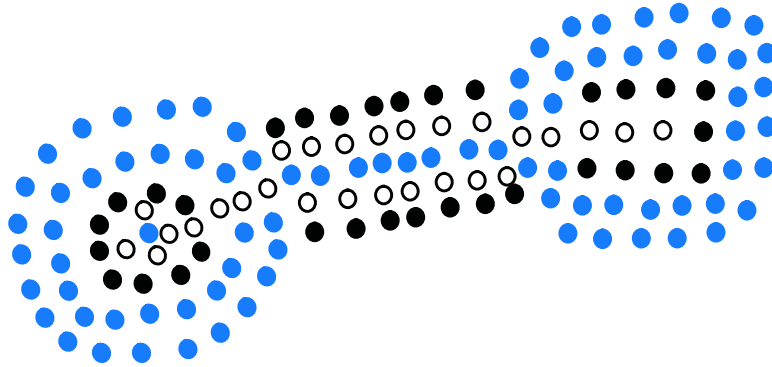
Algorithm eulerTourBinary(T, p):

```
perform the "pre visit" action for position  $p$ 
if  $p$  has a left child  $lc$  then
    eulerTourBinary( $T, lc$ )           { recursively tour the left subtree of  $p$  }
perform the "in visit" action for position  $p$ 
if  $p$  has a right child  $rc$  then
    eulerTourBinary( $T, rc$ )           { recursively tour the right subtree of  $p$  }
perform the "post visit" action for position  $p$ 
```

Code Fragment 8.29: Algorithm eulerTourBinary for performing an Euler tour traversal of a subtree rooted at position p of a binary tree.

For example, a binary Euler tour can produce a traditional parenthesized arithmetic expression, such as " $((((3+1) \times 3) / ((9-5)+2)) - ((3 \times (7-4)) + 6))$ " for the tree in Figure 8.20, as follows:

- "Pre visit" action: if the position is internal, print "(".
- "In visit" action: print the value or operator stored at the position.
- "Post visit" action: if the position is internal, print ")".



Contents

6.1	Stacks	226
6.1.1	The Stack Abstract Data Type	227
6.1.2	A Simple Array-Based Stack Implementation	230
6.1.3	Implementing a Stack with a Singly Linked List	233
6.1.4	Reversing an Array Using a Stack	234
6.1.5	Matching Parentheses and HTML Tags	235
6.2	Queues	238
6.2.1	The Queue Abstract Data Type	239
6.2.2	Array-Based Queue Implementation	241
6.2.3	Implementing a Queue with a Singly Linked List	245
6.2.4	A Circular Queue	246
6.3	Double-Ended Queues	248
6.3.1	The Deque Abstract Data Type	248
6.3.2	Implementing a Deque	250
6.3.3	Dequeues in the Java Collections Framework	251
6.4	Exercises	252

6.1 Stacks

A *stack* is a collection of objects that are inserted and removed according to the *last-in, first-out (LIFO)* principle. A user may insert objects into a stack at any time, but may only access or remove the most recently inserted object that remains (at the so-called “top” of the stack). The name “stack” is derived from the metaphor of a stack of plates in a spring-loaded, cafeteria plate dispenser. In this case, the fundamental operations involve the “pushing” and “popping” of plates on the stack. When we need a new plate from the dispenser, we “pop” the top plate off the stack, and when we add a plate, we “push” it down on the stack to become the new top plate. Perhaps an even more amusing example is a PEZ[®] candy dispenser, which stores mint candies in a spring-loaded container that “pops” out the topmost candy in the stack when the top of the dispenser is lifted (see Figure 6.1).

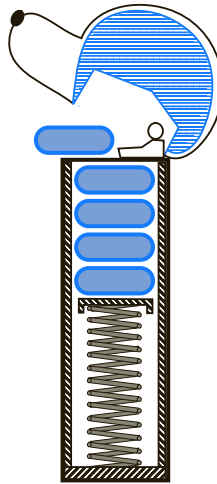


Figure 6.1: A schematic drawing of a PEZ[®] dispenser; a physical implementation of the stack ADT. (PEZ[®] is a registered trademark of PEZ Candy, Inc.)

Stacks are a fundamental data structure. They are used in many applications, including the following.

Example 6.1: Internet Web browsers store the addresses of recently visited sites on a stack. Each time a user visits a new site, that site’s address is “pushed” onto the stack of addresses. The browser then allows the user to “pop” back to previously visited sites using the “back” button.

Example 6.2: Text editors usually provide an “undo” mechanism that cancels recent editing operations and reverts to former states of a document. This undo operation can be accomplished by keeping text changes in a stack.

6.1.1 The Stack Abstract Data Type

Stacks are the simplest of all data structures, yet they are also among the most important, as they are used in a host of different applications, and as a tool for many more sophisticated data structures and algorithms. Formally, a stack is an abstract data type (ADT) that supports the following two update methods:

push(*e*): Adds element *e* to the top of the stack.

pop(): Removes and returns the top element from the stack (or null if the stack is empty).

Additionally, a stack supports the following accessor methods for convenience:

top(): Returns the top element of the stack, without removing it (or null if the stack is empty).

size(): Returns the number of elements in the stack.

isEmpty(): Returns a boolean indicating whether the stack is empty.

By convention, we assume that elements added to the stack can have arbitrary type and that a newly created stack is empty.

Example 6.3: *The following table shows a series of stack operations and their effects on an initially empty stack *S* of integers.*

Method	Return Value	Stack Contents
push(5)	—	(5)
push(3)	—	(5, 3)
size()	2	(5, 3)
pop()	3	(5)
isEmpty()	false	(5)
pop()	5	()
isEmpty()	true	()
pop()	null	()
push(7)	—	(7)
push(9)	—	(7, 9)
top()	9	(7, 9)
push(4)	—	(7, 9, 4)
size()	3	(7, 9, 4)
pop()	4	(7, 9)
push(6)	—	(7, 9, 6)
push(8)	—	(7, 9, 6, 8)
pop()	8	(7, 9, 6)

A Stack Interface in Java

In order to formalize our abstraction of a stack, we define what is known as its **application programming interface** (API) in the form of a Java **interface**, which describes the names of the methods that the ADT supports and how they are to be declared and used. This interface is defined in Code Fragment 6.1.

We rely on Java’s **generics framework** (described in Section 2.5.2), allowing the elements stored in the stack to belong to any object type $\langle E \rangle$. For example, a variable representing a stack of integers could be declared with type `Stack<Integer>`. The formal type parameter is used as the parameter type for the push method, and the return type for both pop and top.

Recall, from the discussion of Java interfaces in Section 2.3.1, that the interface serves as a type definition but that it cannot be directly instantiated. For the ADT to be of any use, we must provide one or more concrete classes that implement the methods of the interface associated with that ADT. In the following subsections, we will give two such implementations of the Stack interface: one that uses an array for storage and another that uses a linked list.

The java.util.Stack Class

Because of the importance of the stack ADT, Java has included, since its original version, a concrete class named `java.util.Stack` that implements the LIFO semantics of a stack. However, Java’s `Stack` class remains only for historic reasons, and its interface is not consistent with most other data structures in the Java library. In fact, the current documentation for the `Stack` class recommends that it not be used, as LIFO functionality (and more) is provided by a more general data structure known as a double-ended queue (which we describe in Section 6.3).

For the sake of comparison, Table 6.1 provides a side-by-side comparison of the interface for our stack ADT and the `java.util.Stack` class. In addition to some differences in method names, we note that methods `pop` and `peek` of the `java.util.Stack` class throw a custom `EmptyStackException` if called when the stack is empty (whereas `null` is returned in our abstraction).

Our Stack ADT	Class <code>java.util.Stack</code>	
<code>size()</code>	<code>size()</code>	
<code>isEmpty()</code>	<code>empty()</code>	⇐
<code>push(<i>e</i>)</code>	<code>push(<i>e</i>)</code>	
<code>pop()</code>	<code>pop()</code>	
<code>top()</code>	<code>peek()</code>	⇐

Table 6.1: Methods of our stack ADT and corresponding methods of the class `java.util.Stack`, with differences highlighted in the right margin.

```
1  /**
2   * A collection of objects that are inserted and removed according to the last-in
3   * first-out principle. Although similar in purpose, this interface differs from
4   * java.util.Stack.
5   *
6   * @author Michael T. Goodrich
7   * @author Roberto Tamassia
8   * @author Michael H. Goldwasser
9   */
10 public interface Stack<E> {
11
12     /**
13      * Returns the number of elements in the stack.
14      * @return number of elements in the stack
15      */
16     int size();
17
18     /**
19      * Tests whether the stack is empty.
20      * @return true if the stack is empty, false otherwise
21      */
22     boolean isEmpty();
23
24     /**
25      * Inserts an element at the top of the stack.
26      * @param e the element to be inserted
27      */
28     void push(E e);
29
30     /**
31      * Returns, but does not remove, the element at the top of the stack.
32      * @return top element in the stack (or null if empty)
33      */
34     E top();
35
36     /**
37      * Removes and returns the top element from the stack.
38      * @return element removed (or null if empty)
39      */
40     E pop();
41 }
```

Code Fragment 6.1: Interface Stack documented with comments in Javadoc style (Section 1.9.4). Note also the use of the generic parameterized type, E, which allows a stack to contain elements of any specified (reference) type.

6.1.2 A Simple Array-Based Stack Implementation

As our first implementation of the stack ADT, we store elements in an array, named `data`, with capacity N for some fixed N . We oriented the stack so that the bottom element of the stack is always stored in cell `data[0]`, and the top element of the stack in cell `data[t]` for index t that is equal to one less than the current size of the stack. (See Figure 6.2.)

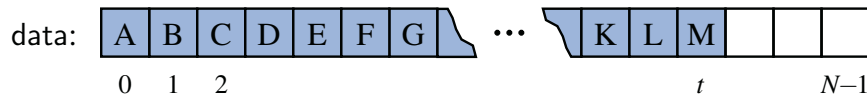


Figure 6.2: Representing a stack with an array; the top element is in cell `data[t]`.

Recalling that arrays start at index 0 in Java, when the stack holds elements from `data[0]` to `data[t]` inclusive, it has size $t + 1$. By convention, when the stack is empty it will have t equal to -1 (and thus has size $t + 1$, which is 0). A complete Java implementation based on this strategy is given in Code Fragment 6.2 (with Javadoc comments omitted due to space considerations).

```

1 public class ArrayStack<E> implements Stack<E> {
2     public static final int CAPACITY=1000; // default array capacity
3     private E[] data; // generic array used for storage
4     private int t = -1; // index of the top element in stack
5     public ArrayStack() { this(CAPACITY); } // constructs stack with default capacity
6     public ArrayStack(int capacity) { // constructs stack with given capacity
7         data = (E[]) new Object[capacity]; // safe cast; compiler may give warning
8     }
9     public int size() { return (t + 1); }
10    public boolean isEmpty() { return (t == -1); }
11    public void push(E e) throws IllegalStateException {
12        if (size() == data.length) throw new IllegalStateException("Stack is full");
13        data[++t] = e; // increment t before storing new item
14    }
15    public E top() {
16        if (isEmpty()) return null;
17        return data[t];
18    }
19    public E pop() {
20        if (isEmpty()) return null;
21        E answer = data[t];
22        data[t] = null; // dereference to help garbage collection
23        t--;
24        return answer;
25    }
26 }

```

Code Fragment 6.2: Array-based implementation of the Stack interface.

A Drawback of This Array-Based Stack Implementation

The array implementation of a stack is simple and efficient. Nevertheless, this implementation has one negative aspect—it relies on a fixed-capacity array, which limits the ultimate size of the stack.

For convenience, we allow the user of a stack to specify the capacity as a parameter to the constructor (and offer a default constructor that uses capacity of 1,000). In cases where a user has a good estimate on the number of items needing to go in the stack, the array-based implementation is hard to beat. However, if the estimate is wrong, there can be grave consequences. If the application needs much less space than the reserved capacity, memory is wasted. Worse yet, if an attempt is made to push an item onto a stack that has already reached its maximum capacity, the implementation of Code Fragment 6.2 throws an `IllegalStateException`, refusing to store the new element. Thus, even with its simplicity and efficiency, the array-based stack implementation is not necessarily ideal.

Fortunately, we will later demonstrate two approaches for implementing a stack without such a size limitation and with space always proportional to the actual number of elements stored in the stack. One approach, given in the next subsection uses a singly linked list for storage; in Section 7.2.1, we will provide a more advanced array-based approach that overcomes the limit of a fixed capacity.

Analyzing the Array-Based Stack Implementation

The correctness of the methods in the array-based implementation follows from our definition of index t . Note well that when pushing an element, t is incremented before placing the new element, so that it uses the first available cell.

Table 6.2 shows the running times for methods of this array-based stack implementation. Each method executes a constant number of statements involving arithmetic operations, comparisons, and assignments, or calls to `size` and `isEmpty`, which both run in constant time. Thus, in this implementation of the stack ADT, each method runs in constant time, that is, they each run in $O(1)$ time.

Method	Running Time
size	$O(1)$
isEmpty	$O(1)$
top	$O(1)$
push	$O(1)$
pop	$O(1)$

Table 6.2: Performance of a stack realized by an array. The space usage is $O(N)$, where N is the size of the array, determined at the time the stack is instantiated, and independent from the number $n \leq N$ of elements that are actually in the stack.

Garbage Collection in Java

We wish to draw attention to one interesting aspect involving the implementation of the pop method in Code Fragment 6.2. We set a local variable, `answer`, to reference the element that is being popped, and then we intentionally reset `data[t]` to **null** at line 22, before decrementing `t`. The assignment to **null** was not technically required, as our stack would still operate correctly without it.

Our reason for returning the cell to a null reference is to assist Java's *garbage collection* mechanism, which searches memory for objects that are no longer actively referenced and reclaims their space for future use. (For more details, see Section 15.1.3.) If we continued to store a reference to the popped element in our array, the stack class would ignore it (eventually overwriting the reference if more elements get added to the stack). But, if there were no other active references to the element in the user's application, that spurious reference in the stack's array would stop Java's garbage collector from reclaiming the element.

Sample Usage

We conclude this section by providing a demonstration of code that creates and uses an instance of the `ArrayStack` class. In this example, we declare the parameterized type of the stack as the `Integer` wrapper class. This causes the signature of the push method to accept an `Integer` instance as a parameter, and for the return type of both `top` and `pop` to be an `Integer`. Of course, with Java's autoboxing and unboxing (see Section 1.3), a primitive **int** can be sent as a parameter to `push`.

```
Stack<Integer> S = new ArrayStack<>(); // contents: ()
S.push(5); // contents: (5)
S.push(3); // contents: (5, 3)
System.out.println(S.size()); // contents: (5, 3) outputs 2
System.out.println(S.pop()); // contents: (5) outputs 3
System.out.println(S.isEmpty()); // contents: (5) outputs false
System.out.println(S.pop()); // contents: () outputs 5
System.out.println(S.isEmpty()); // contents: () outputs true
System.out.println(S.pop()); // contents: () outputs null
S.push(7); // contents: (7)
S.push(9); // contents: (7, 9)
System.out.println(S.top()); // contents: (7, 9) outputs 9
S.push(4); // contents: (7, 9, 4)
System.out.println(S.size()); // contents: (7, 9, 4) outputs 3
System.out.println(S.pop()); // contents: (7, 9) outputs 4
S.push(6); // contents: (7, 9, 6)
S.push(8); // contents: (7, 9, 6, 8)
System.out.println(S.pop()); // contents: (7, 9, 6) outputs 8
```

Code Fragment 6.3: Sample usage of our `ArrayStack` class.

6.1.3 Implementing a Stack with a Singly Linked List

In this section, we demonstrate how the Stack interface can be easily implemented using a singly linked list for storage. Unlike our array-based implementation, the linked-list approach has memory usage that is always proportional to the number of actual elements currently in the stack, and without an arbitrary capacity limit.

In designing such an implementation, we need to decide if the top of the stack is at the front or back of the list. There is clearly a best choice here, however, since we can insert and delete elements in constant time only at the front. With the top of the stack stored at the front of the list, all methods execute in constant time.

The Adapter Pattern

The *adapter* design pattern applies to any context where we effectively want to modify an existing class so that its methods match those of a related, but different, class or interface. One general way to apply the adapter pattern is to define a new class in such a way that it contains an instance of the existing class as a hidden field, and then to implement each method of the new class using methods of this hidden instance variable. By applying the adapter pattern in this way, we have created a new class that performs some of the same functions as an existing class, but repackaged in a more convenient way.

In the context of the stack ADT, we can adapt our SinglyLinkedList class of Section 3.2.1 to define a new LinkedStack class, shown in Code Fragment 6.4. This class declares a SinglyLinkedList named list as a private field, and uses the following correspondences:

<i>Stack Method</i>	<i>Singly Linked List Method</i>
size()	list.size()
isEmpty()	list.isEmpty()
push(<i>e</i>)	list.addFirst(<i>e</i>)
pop()	list.removeFirst()
top()	list.first()

```

1 public class LinkedStack<E> implements Stack<E> {
2     private SinglyLinkedList<E> list = new SinglyLinkedList<>(); // an empty list
3     public LinkedStack() { } // new stack relies on the initially empty list
4     public int size() { return list.size(); }
5     public boolean isEmpty() { return list.isEmpty(); }
6     public void push(E element) { list.addFirst(element); }
7     public E top() { return list.first(); }
8     public E pop() { return list.removeFirst(); }
9 }

```

Code Fragment 6.4: Implementation of a Stack using a SinglyLinkedList as storage.

6.1.4 Reversing an Array Using a Stack

As a consequence of the LIFO protocol, a stack can be used as a general toll to reverse a data sequence. For example, if the values 1, 2, and 3 are pushed onto a stack in that order, they will be popped from the stack in the order 3, 2, and then 1.

We demonstrate this concept by revisiting the problem of reversing the elements of an array. (We provided a recursive algorithm for this task in Section 5.3.1.) We create an empty stack for auxiliary storage, push all of the array elements onto the stack, and then pop those elements off of the stack while overwriting the cells of the array from beginning to end. In Code Fragment 6.5, we give a Java implementation of this algorithm. We show an example use of this method in Code Fragment 6.6.

```

1  /** A generic method for reversing an array. */
2  public static <E> void reverse(E[] a) {
3      Stack<E> buffer = new ArrayStack<>(a.length);
4      for (int i=0; i < a.length; i++)
5          buffer.push(a[i]);
6      for (int i=0; i < a.length; i++)
7          a[i] = buffer.pop();
8  }
```

Code Fragment 6.5: A generic method that reverses the elements in an array with objects of type E, using a stack declared with the interface Stack<E> as its type.

```

1  /** Tester routine for reversing arrays */
2  public static void main(String args[] ) {
3      Integer[] a = {4, 8, 15, 16, 23, 42}; // autoboxing allows this
4      String[] s = {"Jack", "Kate", "Hurley", "Jin", "Michael"};
5      System.out.println("a = " + Arrays.toString(a));
6      System.out.println("s = " + Arrays.toString(s));
7      System.out.println("Reversing...");
8      reverse(a);
9      reverse(s);
10     System.out.println("a = " + Arrays.toString(a));
11     System.out.println("s = " + Arrays.toString(s));
12 }
```

The output from this method is the following:

```

a = [4, 8, 15, 16, 23, 42]
s = [Jack, Kate, Hurley, Jin, Michael]
Reversing...
a = [42, 23, 16, 15, 8, 4]
s = [Michael, Jin, Hurley, Kate, Jack]
```

Code Fragment 6.6: A test of the reverse method using two arrays.

6.1.5 Matching Parentheses and HTML Tags

In this subsection, we explore two related applications of stacks, both of which involve testing for pairs of matching delimiters. In our first application, we consider arithmetic expressions that may contain various pairs of grouping symbols, such as

- Parentheses: “(” and “)”
- Braces: “{” and “}”
- Brackets: “[” and “]”

Each opening symbol must match its corresponding closing symbol. For example, a left bracket, “[,” must match a corresponding right bracket, “],” as in the following expression

$$[(5 + x) - (y + z)].$$

The following examples further illustrate this concept:

- Correct: `()(()){}([()])`
- Correct: `((())(()){}([()]){})`
- Incorrect: `)((()){}([()])`
- Incorrect: `{[()]}`
- Incorrect: `(`

We leave the precise definition of a matching group of symbols to Exercise R-6.6.

An Algorithm for Matching Delimiters

An important task when processing arithmetic expressions is to make sure their delimiting symbols match up correctly. We can use a stack to perform this task with a single left-to-right scan of the original string.

Each time we encounter an opening symbol, we push that symbol onto the stack, and each time we encounter a closing symbol, we pop a symbol from the stack (assuming it is not empty) and check that these two symbols form a valid pair. If we reach the end of the expression and the stack is empty, then the original expression was properly matched. Otherwise, there must be an opening delimiter on the stack without a matching symbol. If the length of the original expression is n , the algorithm will make at most n calls to push and n calls to pop. Code Fragment 6.7 presents a Java implementation of such an algorithm. It specifically checks for delimiter pairs `()`, `{ }`, and `[ ]`, but could easily be changed to accommodate further symbols. Specifically, we define two fixed strings, `"( { [ "` and `) } ] "`, that are intentionally coordinated to reflect the symbol pairs. When examining a character of the expression string, we call the `indexOf` method of the `String` class on these special strings to determine if the character matches a delimiter and, if so, which one. Method `indexOf` returns the index at which a given character is first found in a string (or `-1` if the character is not found).

```

1  /** Tests if delimiters in the given expression are properly matched. */
2  public static boolean isMatched(String expression) {
3      final String opening  = "{[";           // opening delimiters
4      final String closing  = "}]";           // respective closing delimiters
5      Stack<Character> buffer = new LinkedStack<>();
6      for (char c : expression.toCharArray()) {
7          if (opening.indexOf(c) != -1)        // this is a left delimiter
8              buffer.push(c);
9          else if (closing.indexOf(c) != -1) {  // this is a right delimiter
10             if (buffer.isEmpty())             // nothing to match with
11                 return false;
12             if (closing.indexOf(c) != opening.indexOf(buffer.pop()))
13                 return false;                // mismatched delimiter
14         }
15     }
16     return buffer.isEmpty();                 // were all opening delimiters matched?
17 }

```

Code Fragment 6.7: Method for matching delimiters in an arithmetic expression.

Matching Tags in a Markup Language

Another application of matching delimiters is in the validation of markup languages such as HTML or XML. HTML is the standard format for hyperlinked documents on the Internet and XML is an extensible markup language used for a variety of structured data sets. We show a sample HTML document in Figure 6.3.

```

<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine. The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even as
a stowaway now felt that he
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>

```

(a)

The Little Boat

The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage.

1. Will the salesman die?
2. What color is the boat?
3. And what about Naomi?

(b)

Figure 6.3: Illustrating (a) an HTML document and (b) its rendering.

In an HTML document, portions of text are delimited by *HTML tags*. A simple opening HTML tag has the form “<name>” and the corresponding closing tag has the form “</name>”. For example, we see the <body> tag on the first line of Figure 6.3a, and the matching </body> tag at the close of that document. Other commonly used HTML tags that are used in this example include:

- <body>: document body
- <h1>: section header
- <center>: center justify
- <p>: paragraph
- : numbered (ordered) list
- : list item

Ideally, an HTML document should have matching tags, although most browsers tolerate a certain number of mismatching tags. In Code Fragment 6.8, we give a Java method that matches tags in a string representing an HTML document.

We make a left-to-right pass through the raw string, using index *j* to track our progress. The indexOf method of the String class, which optionally accepts a starting index as a second parameter, locates the '<' and '>' characters that define the tags. Method substring, also of the String class, returns the substring starting at a given index and optionally ending right before another given index. Opening tags are pushed onto the stack, and matched against closing tags as they are popped from the stack, just as we did when matching delimiters in Code Fragment 6.7.

```

1  /** Tests if every opening tag has a matching closing tag in HTML string. */
2  public static boolean isHTMLMatched(String html) {
3      Stack<String> buffer = new LinkedStack<>();
4      int j = html.indexOf('<');           // find first '<' character (if any)
5      while (j != -1) {
6          int k = html.indexOf('>', j+1); // find next '>' character
7          if (k == -1)
8              return false;             // invalid tag
9          String tag = html.substring(j+1, k); // strip away < >
10         if (!tag.startsWith("/"))       // this is an opening tag
11             buffer.push(tag);
12         else {                           // this is a closing tag
13             if (buffer.isEmpty())
14                 return false;           // no tag to match
15             if (!tag.substring(1).equals(buffer.pop()))
16                 return false;          // mismatched tag
17         }
18         j = html.indexOf('<', k+1);      // find next '<' character (if any)
19     }
20     return buffer.isEmpty();             // were all opening tags matched?
21 }
```

Code Fragment 6.8: Method for testing if an HTML document has matching tags.

6.2 Queues

Another fundamental data structure is the *queue*. It is a close “cousin” of the stack, but a queue is a collection of objects that are inserted and removed according to the *first-in, first-out (FIFO)* principle. That is, elements can be inserted at any time, but only the element that has been in the queue the longest can be next removed.

We usually say that elements enter a queue at the back and are removed from the front. A metaphor for this terminology is a line of people waiting to get on an amusement park ride. People waiting for such a ride enter at the back of the line and get on the ride from the front of the line. There are many other applications of queues (see Figure 6.4). Stores, theaters, reservation centers, and other similar services typically process customer requests according to the FIFO principle. A queue would therefore be a logical choice for a data structure to handle calls to a customer service center, or a wait-list at a restaurant. FIFO queues are also used by many computing devices, such as a networked printer, or a Web server responding to requests.

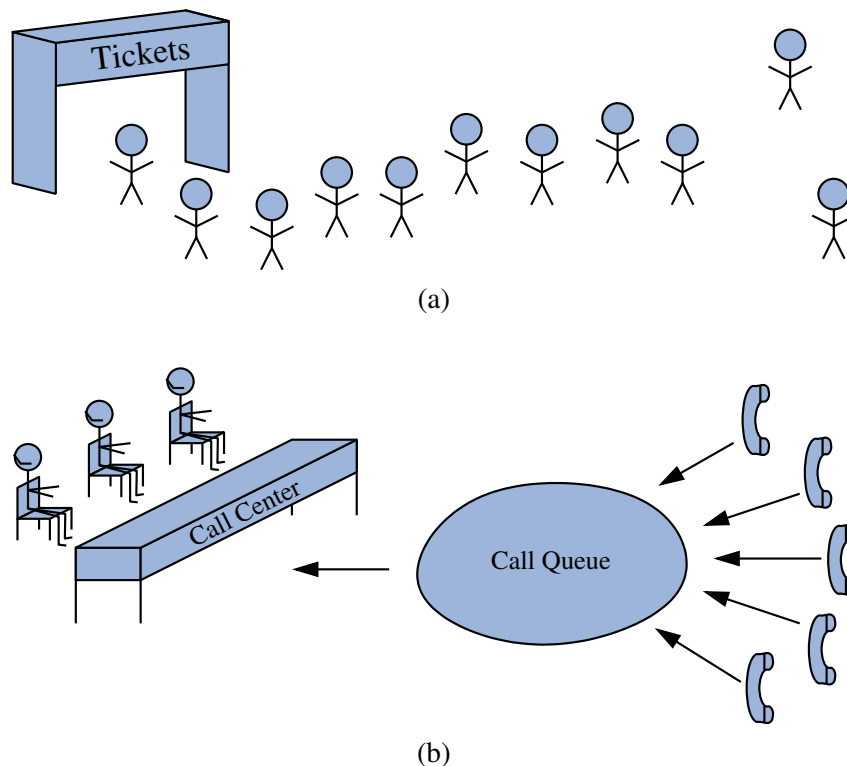


Figure 6.4: Real-world examples of a first-in, first-out queue. (a) People waiting in line to purchase tickets; (b) phone calls being routed to a customer service center.

6.2.1 The Queue Abstract Data Type

Formally, the queue abstract data type defines a collection that keeps objects in a sequence, where element access and deletion are restricted to the *first* element in the queue, and element insertion is restricted to the back of the sequence. This restriction enforces the rule that items are inserted and deleted in a queue according to the first-in, first-out (FIFO) principle. The *queue* abstract data type (ADT) supports the following two update methods:

`enqueue(e)`: Adds element *e* to the back of queue.

`dequeue()`: Removes and returns the first element from the queue (or null if the queue is empty).

The queue ADT also includes the following accessor methods (with first being analogous to the stack's top method):

`first()`: Returns the first element of the queue, without removing it (or null if the queue is empty).

`size()`: Returns the number of elements in the queue.

`isEmpty()`: Returns a boolean indicating whether the queue is empty.

By convention, we assume that elements added to the queue can have arbitrary type and that a newly created queue is empty. We formalize the queue ADT with the Java interface shown in Code Fragment 6.9.

```
1 public interface Queue<E> {  
2     /** Returns the number of elements in the queue. */  
3     int size();  
4     /** Tests whether the queue is empty. */  
5     boolean isEmpty();  
6     /** Inserts an element at the rear of the queue. */  
7     void enqueue(E e);  
8     /** Returns, but does not remove, the first element of the queue (null if empty). */  
9     E first();  
10    /** Removes and returns the first element of the queue (null if empty). */  
11    E dequeue();  
12 }
```

Code Fragment 6.9: A Queue interface defining the queue ADT, with a standard FIFO protocol for insertions and removals.

Example 6.4: The following table shows a series of queue operations and their effects on an initially empty queue Q of integers.

Method	Return Value	first $\leftarrow Q \leftarrow$ last
enqueue(5)	—	(5)
enqueue(3)	—	(5, 3)
size()	2	(5, 3)
dequeue()	5	(3)
isEmpty()	false	(3)
dequeue()	3	()
isEmpty()	true	()
dequeue()	null	()
enqueue(7)	—	(7)
enqueue(9)	—	(7, 9)
first()	7	(7, 9)
enqueue(4)	—	(7, 9, 4)

The java.util.Queue Interface in Java

Java provides a type of queue interface, `java.util.Queue`, which has functionality similar to the traditional queue ADT, given above, but the documentation for the `java.util.Queue` interface does not insist that it support only the FIFO principle. When supporting the FIFO principle, the methods of the `java.util.Queue` interface have the equivalences with the queue ADT shown in Table 6.3.

The `java.util.Queue` interface supports two styles for most operations, which vary in the way that they treat exceptional cases. When a queue is empty, the `remove()` and `element()` methods throw a `NoSuchElementException`, while the corresponding methods `poll()` and `peek()` return **null**. For implementations with a bounded capacity, the `add` method will throw an `IllegalStateException` when full, while the `offer` method ignores the new element and returns **false** to signal that the element was not accepted.

Our Queue ADT	Interface <code>java.util.Queue</code>	
	throws exceptions	returns special value
enqueue(e)	add(e)	offer(e)
dequeue()	remove()	poll()
first()	element()	peek()
size()	size()	
isEmpty()	isEmpty()	

Table 6.3: Methods of the queue ADT and corresponding methods of the interface `java.util.Queue`, when supporting the FIFO principle.

6.2.2 Array-Based Queue Implementation

In Section 6.1.2, we implemented the LIFO semantics of the Stack ADT using an array (albeit, with a fixed capacity), such that every operation executes in constant time. In this section, we will consider how to use an array to efficiently support the FIFO semantics of the Queue ADT.

Let's assume that as elements are inserted into a queue, we store them in an array such that the first element is at index 0, the second element at index 1, and so on. (See Figure 6.5.)

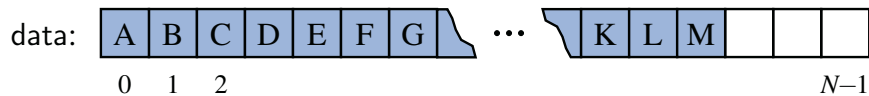


Figure 6.5: Using an array to store elements of a queue, such that the first element inserted, “A”, is at cell 0, the second element inserted, “B”, at cell 1, and so on.

With such a convention, the question is how we should implement the dequeue operation. The element to be removed is stored at index 0 of the array. One strategy is to execute a loop to shift all other elements of the queue one cell to the left, so that the front of the queue is again aligned with cell 0 of the array. Unfortunately, the use of such a loop would result in an $O(n)$ running time for the dequeue method.

We can improve on the above strategy by avoiding the loop entirely. We will replace a dequeued element in the array with a null reference, and maintain an explicit variable f to represent the index of the element that is currently at the front of the queue. Such an algorithm for dequeue would run in $O(1)$ time. After several dequeue operations, this approach might lead to the configuration portrayed in Figure 6.6.

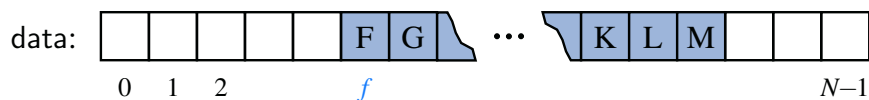


Figure 6.6: Allowing the front of the queue to drift away from index 0. In this representation, index f denotes the location of the front of the queue.

However, there remains a challenge with the revised approach. With an array of capacity N , we should be able to store up to N elements before reaching any exceptional case. If we repeatedly let the front of the queue drift rightward over time, the back of the queue would reach the end of the underlying array even when there are fewer than N elements currently in the queue. We must decide how to store additional elements in such a configuration.

Using an Array Circularly

In developing a robust queue implementation, we allow both the front and back of the queue to drift rightward, with the contents of the queue “wrapping around” the end of an array, as necessary. Assuming that the array has fixed length N , new elements are enqueued toward the “end” of the current queue, progressing from the front to index $N - 1$ and continuing at index 0, then 1. Figure 6.7 illustrates such a queue with first element F and last element R.

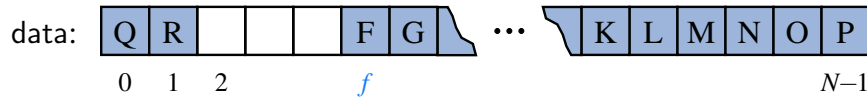


Figure 6.7: Modeling a queue with a circular array that wraps around the end.

Implementing such a circular view is relatively easy with the *modulo* operator, denoted with the symbol `%` in Java. Recall that the modulo operator is computed by taking the remainder after an integral division. For example, 14 divided by 3 has a quotient of 4 with remainder 2, that is, $\frac{14}{3} = 4\frac{2}{3}$. So in Java, `14 / 3` evaluates to the quotient 4, while `14 % 3` evaluates to the remainder 2.

The modulo operator is ideal for treating an array circularly. When we dequeue an element and want to “advance” the front index, we use the arithmetic $f = (f + 1) \% N$. As a concrete example, if we have an array of length 10, and a front index 7, we can advance the front by formally computing $(7 + 1) \% 10$, which is simply 8, as 8 divided by 10 is 0 with a remainder of 8. Similarly, advancing index 8 results in index 9. But when we advance from index 9 (the last one in the array), we compute $(9 + 1) \% 10$, which evaluates to index 0 (as 10 divided by 10 has a remainder of zero).

A Java Queue Implementation

A complete implementation of a queue ADT using an array in circular fashion is presented in Code Fragment 6.10. Internally, the queue class maintains the following three instance variables:

- data:** a reference to the underlying array.
- f:** an integer that represents the index, within array `data`, of the first element of the queue (assuming the queue is not empty).
- sz:** an integer representing the current number of elements stored in the queue (not to be confused with the length of the array).

We allow the user to specify the capacity of the queue as an optional parameter to the constructor.

The implementations of methods `size` and `isEmpty` are trivial, given the `sz` field, and the implementation of `first` is simple, given index `f`. A discussion of update methods `enqueue` and `dequeue` follows the presentation of the code.

```

1  /** Implementation of the queue ADT using a fixed-length array. */
2  public class ArrayQueue<E> implements Queue<E> {
3      // instance variables
4      private E[] data;                // generic array used for storage
5      private int f = 0;                // index of the front element
6      private int sz = 0;                // current number of elements
7
8      // constructors
9      public ArrayQueue() {this(CAPACITY);} // constructs queue with default capacity
10     public ArrayQueue(int capacity) {      // constructs queue with given capacity
11         data = (E[]) new Object[capacity]; // safe cast; compiler may give warning
12     }
13
14     // methods
15     /** Returns the number of elements in the queue. */
16     public int size() { return sz; }
17
18     /** Tests whether the queue is empty. */
19     public boolean isEmpty() { return (sz == 0); }
20
21     /** Inserts an element at the rear of the queue. */
22     public void enqueue(E e) throws IllegalStateException {
23         if (sz == data.length) throw new IllegalStateException("Queue is full");
24         int avail = (f + sz) % data.length; // use modular arithmetic
25         data[avail] = e;
26         sz++;
27     }
28
29     /** Returns, but does not remove, the first element of the queue (null if empty). */
30     public E first() {
31         if (isEmpty()) return null;
32         return data[f];
33     }
34
35     /** Removes and returns the first element of the queue (null if empty). */
36     public E dequeue() {
37         if (isEmpty()) return null;
38         E answer = data[f];
39         data[f] = null; // dereference to help garbage collection
40         f = (f + 1) % data.length;
41         sz--;
42         return answer;
43     }

```

Code Fragment 6.10: Array-based implementation of a queue.

Adding and Removing Elements

The goal of the enqueue method is to add a new element to the back of the queue. We need to determine the proper index at which to place the new element. Although we do not explicitly maintain an instance variable for the back of the queue, we compute the index of the next opening based on the formula:

$$\text{avail} = (f + \text{sz}) \% \text{data.length};$$

Note that we are using the size of the queue as it exists *prior* to the addition of the new element. As a sanity check, for a queue with capacity 10, current size 3, and first element at index 5, its three elements are stored at indices 5, 6, and 7, and the next element should be added at index 8, computed as $(5+3) \% 10$. As a case with wraparound, if the queue has capacity 10, current size 3, and first element at index 8, its three elements are stored at indices 8, 9, and 0, and the next element should be added at index 1, computed as $(8+3) \% 10$.

When the dequeue method is called, the current value of f designates the index of the value that is to be removed and returned. We keep a local reference to the element that will be returned, before setting its cell of the array back to **null**, to aid the garbage collector. Then the index f is updated to reflect the removal of the first element, and the presumed promotion of the second element to become the new first. In most cases, we simply want to increment the index by one, but because of the possibility of a wraparound configuration, we rely on modular arithmetic, computing $f = (f+1) \% \text{data.length}$, as originally described on page 242.

Analyzing the Efficiency of an Array-Based Queue

Table 6.4 shows the running times of methods in a realization of a queue by an array. As with our array-based stack implementation, each of the queue methods in the array realization executes a constant number of statements involving arithmetic operations, comparisons, and assignments. Thus, each method in this implementation runs in $O(1)$ time.

Method	Running Time
size	$O(1)$
isEmpty	$O(1)$
first	$O(1)$
enqueue	$O(1)$
dequeue	$O(1)$

Table 6.4: Performance of a queue realized by an array. The space usage is $O(N)$, where N is the size of the array, determined at the time the queue is created, and independent from the number $n < N$ of elements that are actually in the queue.

6.2.3 Implementing a Queue with a Singly Linked List

As we did for the stack ADT, we can easily adapt a singly linked list to implement the queue ADT while supporting worst-case $O(1)$ -time for all operations, and without any artificial limit on the capacity. The natural orientation for a queue is to align the front of the queue with the front of the list, and the back of the queue with the tail of the list, because the only update operation that singly linked lists support at the back end is an insertion. Our Java implementation of a `LinkedList` class is given in Code 6.11.

```
1  /** Realization of a FIFO queue as an adaptation of a SinglyLinkedList. */
2  public class LinkedList<E> implements Queue<E> {
3      private SinglyLinkedList<E> list = new SinglyLinkedList<>(); // an empty list
4      public LinkedList() { } // new queue relies on the initially empty list
5      public int size() { return list.size(); }
6      public boolean isEmpty() { return list.isEmpty(); }
7      public void enqueue(E element) { list.addLast(element); }
8      public E first() { return list.first(); }
9      public E dequeue() { return list.removeFirst(); }
10 }
```

Code Fragment 6.11: Implementation of a Queue using a SinglyLinkedList.

Analyzing the Efficiency of a Linked Queue

Although we had not yet introduced asymptotic analysis when we presented our SinglyLinkedList implementation in Chapter 3, it is clear upon reexamination that each method of that class runs in $O(1)$ worst-case time. Therefore, each method of our LinkedList adaptation also runs in $O(1)$ worst-case time.

We also avoid the need to specify a maximum size for the queue, as was done in the array-based queue implementation. However, this benefit comes with some expense. Because each node stores a next reference, in addition to the element reference, a linked list uses more space per element than a properly sized array of references.

Also, although all methods execute in constant time for both implementations, it seems clear that the operations involving linked lists have a large number of primitive operations per call. For example, adding an element to an array-based queue consists primarily of calculating an index with modular arithmetic, storing the element in the array cell, and incrementing the size counter. For a linked list, an insertion includes the instantiation and initialization of a new node, relinking an existing node to the new node, and incrementing the size counter. In practice, this makes the linked-list method more expensive than the array-based method.

6.2.4 A Circular Queue

In Section 3.3, we implemented a *circularly linked list* class that supports all behaviors of a singly linked list, and an additional `rotate()` method that efficiently moves the first element to the end of the list. We can generalize the Queue interface to define a new CircularQueue interface with such a behavior, as shown in Code Fragment 6.12.

```
1 public interface CircularQueue<E> extends Queue<E> {  
2     /**  
3     * Rotates the front element of the queue to the back of the queue.  
4     * This does nothing if the queue is empty.  
5     */  
6     void rotate();  
7 }
```

Code Fragment 6.12: A Java interface, CircularQueue, that extends the Queue ADT with a new `rotate()` method.

This interface can easily be implemented by adapting the CircularlyLinkedList class of Section 3.3 to produce a new LinkedCircularQueue class. This class has an advantage over the traditional LinkedQueue, because a call to `Q.rotate()` is implemented more efficiently than the combination of calls, `Q.enqueue(Q.dequeue())`, because no nodes are created, destroyed, or relinked by the implementation of a rotate operation on a circularly linked list.

A circular queue is an excellent abstraction for applications in which elements are cyclically arranged, such as for multiplayer, turn-based games, or round-robin scheduling of computing processes. In the remainder of this section, we provide a demonstration of the use of a circular queue.

The Josephus Problem

In the children’s game “hot potato,” a group of n children sit in a circle passing an object, called the “potato,” around the circle. The potato begins with a starting child in the circle, and the children continue passing the potato until a leader rings a bell, at which point the child holding the potato must leave the game after handing the potato to the next child in the circle. After the selected child leaves, the other children close up the circle. This process is then continued until there is only one child remaining, who is declared the winner. If the leader always uses the strategy of ringing the bell so that every k^{th} person is removed from the circle, for some fixed value k , then determining the winner for a given list of children is known as the *Josephus problem* (named after an ancient story with far more severe consequences than in the children’s game).

Solving the Josephus Problem Using a Queue

We can solve the Josephus problem for a collection of n elements using a circular queue, by associating the potato with the element at the front of the queue and storing elements in the queue according to their order around the circle. Thus, passing the potato is equivalent to rotating the first element to the back of the queue. After this process has been performed $k - 1$ times, we remove the front element by dequeuing it from the queue and discarding it. We show a complete Java program for solving the Josephus problem using this approach in Code Fragment 6.13, which describes a solution that runs in $O(nk)$ time. (We can solve this problem faster using techniques beyond the scope of this book.)

```

1  public class Josephus {
2      /** Computes the winner of the Josephus problem using a circular queue. */
3      public static <E> E Josephus(CircularQueue<E> queue, int k) {
4          if (queue.isEmpty()) return null;
5          while (queue.size() > 1) {
6              for (int i=0; i < k-1; i++) // skip past k-1 elements
7                  queue.rotate();
8              E e = queue.dequeue(); // remove the front element from the collection
9              System.out.println("    " + e + " is out");
10         }
11         return queue.dequeue(); // the winner
12     }
13
14     /** Builds a circular queue from an array of objects. */
15     public static <E> CircularQueue<E> buildQueue(E a[ ]) {
16         CircularQueue<E> queue = new LinkedCircularQueue<>();
17         for (int i=0; i<a.length; i++)
18             queue.enqueue(a[i]);
19         return queue;
20     }
21
22     /** Tester method */
23     public static void main(String[ ] args) {
24         String[ ] a1 = {"Alice", "Bob", "Cindy", "Doug", "Ed", "Fred"};
25         String[ ] a2 = {"Gene", "Hope", "Irene", "Jack", "Kim", "Lance"};
26         String[ ] a3 = {"Mike", "Roberto"};
27         System.out.println("First winner is " + Josephus(buildQueue(a1), 3));
28         System.out.println("Second winner is " + Josephus(buildQueue(a2), 10));
29         System.out.println("Third winner is " + Josephus(buildQueue(a3), 7));
30     }
31 }

```

Code Fragment 6.13: A complete Java program for solving the Josephus problem using a circular queue.

6.3 Double-Ended Queues

We next consider a queue-like data structure that supports insertion and deletion at both the front and the back of the queue. Such a structure is called a *double-ended queue*, or *deque*, which is usually pronounced “deck” to avoid confusion with the dequeue method of the regular queue ADT, which is pronounced like the abbreviation “D.Q.”

The deque abstract data type is more general than both the stack and the queue ADTs. The extra generality can be useful in some applications. For example, we described a restaurant using a queue to maintain a waitlist. Occasionally, the first person might be removed from the queue only to find that a table was not available; typically, the restaurant will reinsert the person at the *first* position in the queue. It may also be that a customer at the end of the queue may grow impatient and leave the restaurant. (We will need an even more general data structure if we want to model customers leaving the queue from other positions.)

6.3.1 The Deque Abstract Data Type

The deque abstract data type is richer than both the stack and the queue ADTs. To provide a symmetrical abstraction, the deque ADT is defined to support the following update methods:

- `addFirst(e)`: Insert a new element *e* at the front of the deque.
- `addLast(e)`: Insert a new element *e* at the back of the deque.
- `removeFirst()`: Remove and return the first element of the deque (or null if the deque is empty).
- `removeLast()`: Remove and return the last element of the deque (or null if the deque is empty).

Additionally, the deque ADT will include the following accessors:

- `first()`: Returns the first element of the deque, without removing it (or null if the deque is empty).
- `last()`: Returns the last element of the deque, without removing it (or null if the deque is empty).
- `size()`: Returns the number of elements in the deque.
- `isEmpty()`: Returns a boolean indicating whether the deque is empty.

We formalize the deque ADT with the Java interface shown in Code Fragment 6.14.

```

1  /**
2   * Interface for a double-ended queue: a collection of elements that can be inserted
3   * and removed at both ends; this interface is a simplified version of java.util.Deque.
4   */
5  public interface Deque<E> {
6      /** Returns the number of elements in the deque. */
7      int size();
8      /** Tests whether the deque is empty. */
9      boolean isEmpty();
10     /** Returns, but does not remove, the first element of the deque (null if empty). */
11     E first();
12     /** Returns, but does not remove, the last element of the deque (null if empty). */
13     E last();
14     /** Inserts an element at the front of the deque. */
15     void addFirst(E e);
16     /** Inserts an element at the back of the deque. */
17     void addLast(E e);
18     /** Removes and returns the first element of the deque (null if empty). */
19     E removeFirst();
20     /** Removes and returns the last element of the deque (null if empty). */
21     E removeLast();
22 }

```

Code Fragment 6.14: A Java interface, Deque, describing the double-ended queue ADT. Note the use of the generic parameterized type, E, allowing a deque to contain elements of any specified class.

Example 6.5: The following table shows a series of operations and their effects on an initially empty deque *D* of integers.

Method	Return Value	<i>D</i>
addLast(5)	—	(5)
addFirst(3)	—	(3, 5)
addFirst(7)	—	(7, 3, 5)
first()	7	(7, 3, 5)
removeLast()	5	(7, 3)
size()	2	(7, 3)
removeLast()	3	(7)
removeFirst()	7	()
addFirst(6)	—	(6)
last()	6	(6)
addFirst(8)	—	(8, 6)
isEmpty()	false	(8, 6)
last()	6	(8, 6)

6.3.2 Implementing a Deque

We can implement the deque ADT efficiently using either an array or a linked list for storing elements.

Implementing a Deque with a Circular Array

If using an array, we recommend a representation similar to the `ArrayQueue` class, treating the array in circular fashion and storing the index of the first element and the current size of the deque as fields; the index of the last element can be calculated, as needed, using modular arithmetic.

One extra concern is avoiding use of negative values with the modulo operator. When removing the first element, the front index is advanced in circular fashion, with the assignment $f = (f+1) \% N$. But when an element is inserted at the front, the first index must effectively be decremented in circular fashion and it is a mistake to assign $f = (f-1) \% N$. The problem is that when f is 0, the goal should be to “decrement” it to the other end of the array, and thus to index $N-1$. However, a calculation such as $-1 \% 10$ in Java results in the value -1 . A standard way to decrement an index circularly is instead to assign $f = (f-1+N) \% N$. Adding the additional term of N before the modulus is calculated assures that the result is a positive value. We leave details of this approach to Exercise P-6.40.

Implementing a Deque with a Doubly Linked List

Because the deque requires insertion and removal at both ends, a doubly linked list is most appropriate for implementing all operations efficiently. In fact, the `DoublyLinkedList` class from Section 3.4.1 already implements the entire `Deque` interface; we simply need to add the declaration “**implements** `Deque<E>`” to that class definition in order to use it as a deque.

Performance of the Deque Operations

Table 6.5 shows the running times of methods for a deque implemented with a doubly linked list. Note that every method runs in $O(1)$ time.

Method	Running Time
size, isEmpty	$O(1)$
first, last	$O(1)$
addFirst, addLast	$O(1)$
removeFirst, removeLast	$O(1)$

Table 6.5: Performance of a deque realized by either a circular array or a doubly linked list. The space usage for the array-based implementation is $O(N)$, where N is the size of the array, while the space usage of the doubly linked list is $O(n)$ where $n < N$ is the actual number of elements in the deque.

6.3.3 Deques in the Java Collections Framework

The Java Collections Framework includes its own definition of a deque, as the `java.util.Deque` interface, as well as several implementations of the interface including one based on use of a circular array (`java.util.ArrayDeque`) and one based on use of a doubly linked list (`java.util.LinkedList`). So, if we need to use a deque and would rather not implement one from scratch, we can simply use one of those built-in classes.

As is the case with the `java.util.Queue` class (see page 240), the `java.util.Deque` provides duplicative methods that use different techniques to signal exceptional cases. A summary of those methods is given in Table 6.6.

Our Deque ADT	Interface <code>java.util.Deque</code>	
	throws exceptions	returns special value
<code>first()</code>	<code>getFirst()</code>	<code>peekFirst()</code>
<code>last()</code>	<code>getLast()</code>	<code>peekLast()</code>
<code>addFirst(<i>e</i>)</code>	<code>addFirst(<i>e</i>)</code>	<code>offerFirst(<i>e</i>)</code>
<code>addLast(<i>e</i>)</code>	<code>addLast(<i>e</i>)</code>	<code>offerLast(<i>e</i>)</code>
<code>removeFirst()</code>	<code>removeFirst()</code>	<code>pollFirst()</code>
<code>removeLast()</code>	<code>removeLast()</code>	<code>pollLast()</code>
<code>size()</code>	<code>size()</code>	
<code>isEmpty()</code>	<code>isEmpty()</code>	

Table 6.6: Methods of our deque ADT and the corresponding methods of the `java.util.Deque` interface.

When attempting to access or remove the first or last element of an *empty* deque, the methods in the middle column of Table 6.6—that is, `getFirst()`, `getLast()`, `removeFirst()`, and `removeLast()`—throw a `NoSuchElementException`. The methods in the rightmost column—that is, `peekFirst()`, `peekLast()`, `pollFirst()`, and `pollLast()`—simply return the null reference when a deque is empty. In similar manner, when attempting to add an element to an end of a deque with a capacity limit, the `addFirst` and `addLast` methods throw an exception, while the `offerFirst` and `offerLast` methods return false.

The methods that handle bad situations more gracefully (i.e., without throwing exceptions) are useful in applications, known as producer-consumer scenarios, in which it is common for one component of software to look for an element that may have been placed in a queue by another program, or in which it is common to try to insert an item into a fixed-sized buffer that might be full. However, having methods return null when empty are not appropriate for applications in which null might serve as an actual element of a queue.

6.4 Exercises

Reinforcement

- R-6.1 Suppose an initially empty stack S has performed a total of 25 push operations, 12 top operations, and 10 pop operations, 3 of which returned null to indicate an empty stack. What is the current size of S ?
- R-6.2 Had the stack of the previous problem been an instance of the `ArrayStack` class, from Code Fragment 6.2, what would be the final value of the instance variable `t`?
- R-6.3 What values are returned during the following series of stack operations, if executed upon an initially empty stack? `push(5)`, `push(3)`, `pop()`, `push(2)`, `push(8)`, `pop()`, `pop()`, `push(9)`, `push(1)`, `pop()`, `push(7)`, `push(6)`, `pop()`, `pop()`, `push(4)`, `pop()`, `pop()`.
- R-6.4 Implement a method with signature `transfer( $S$ ,  $T$ )` that transfers all elements from stack S onto stack T , so that the element that starts at the top of S is the first to be inserted onto T , and the element at the bottom of S ends up at the top of T .
- R-6.5 Give a recursive method for removing all the elements from a stack.
- R-6.6 Give a precise and complete definition of the concept of matching for grouping symbols in an arithmetic expression. Your definition may be recursive.
- R-6.7 Suppose an initially empty queue Q has performed a total of 32 enqueue operations, 10 first operations, and 15 dequeue operations, 5 of which returned null to indicate an empty queue. What is the current size of Q ?
- R-6.8 Had the queue of the previous problem been an instance of the `ArrayQueue` class, from Code Fragment 6.10, with capacity 30 never exceeded, what would be the final value of the instance variable `f`?
- R-6.9 What values are returned during the following sequence of queue operations, if executed on an initially empty queue? `enqueue(5)`, `enqueue(3)`, `dequeue()`, `enqueue(2)`, `enqueue(8)`, `dequeue()`, `dequeue()`, `enqueue(9)`, `enqueue(1)`, `dequeue()`, `enqueue(7)`, `enqueue(6)`, `dequeue()`, `dequeue()`, `enqueue(4)`, `dequeue()`, `dequeue()`.
- R-6.10 Give a simple adapter that implements the stack ADT while using an instance of a deque for storage.
- R-6.11 Give a simple adapter that implements the queue ADT while using an instance of a deque for storage.
- R-6.12 What values are returned during the following sequence of deque ADT operations, on an initially empty deque? `addFirst(3)`, `addLast(8)`, `addLast(9)`, `addFirst(1)`, `last()`, `isEmpty()`, `addFirst(2)`, `removeLast()`, `addLast(7)`, `first()`, `last()`, `addLast(4)`, `size()`, `removeFirst()`, `removeFirst()`.

- R-6.13 Suppose you have a deque D containing the numbers $(1, 2, 3, 4, 5, 6, 7, 8)$, in this order. Suppose further that you have an initially empty queue Q . Give a code fragment that uses only D and Q (and no other variables) and results in D storing the elements in the order $(1, 2, 3, 5, 4, 6, 7, 8)$.
- R-6.14 Repeat the previous problem using the deque D and an initially empty stack S .
- R-6.15 Augment the `ArrayQueue` implementation with a new `rotate()` method having semantics identical to the combination, `enqueue(dequeue())`. But, your implementation should be more efficient than making two separate calls (for example, because there is no need to modify the size).

Creativity

- C-6.16 Suppose Alice has picked three distinct integers and placed them into a stack S in random order. Write a short, straightline piece of pseudocode (with no loops or recursion) that uses only one comparison and only one variable x , yet that results in variable x storing the largest of Alice's three integers with probability $2/3$. Argue why your method is correct.
- C-6.17 Show how to use the transfer method, described in Exercise R-6.4, and two temporary stacks, to replace the contents of a given stack S with those same elements, but in reversed order.
- C-6.18 In Code Fragment 6.8 we assume that opening tags in HTML have form `<name>`, as with `<li>`. More generally, HTML allows optional attributes to be expressed as part of an opening tag. The general form used for expressing an attribute is `<name attribute1="value1" attribute2="value2">`; for example, a table can be given a border and additional padding by using an opening tag of `<table border="3" cellpadding="5">`. Modify Code Fragment 6.8 so that it can properly match tags, even when an opening tag may include one or more such attributes.
- C-6.19 **Postfix notation** is an unambiguous way of writing an arithmetic expression without parentheses. It is defined so that if “ $(exp_1) \mathbf{op} (exp_2)$ ” is a normal fully parenthesized expression whose operation is **op**, the postfix version of this is “ $pexp_1 pexp_2 \mathbf{op}$ ”, where $pexp_1$ is the postfix version of exp_1 and $pexp_2$ is the postfix version of exp_2 . The postfix version of a single number or variable is just that number or variable. So, for example, the postfix version of “ $((5 + 2) * (8 - 3)) / 4$ ” is “ $5 2 + 8 3 - * 4 /$ ”. Describe a nonrecursive way of evaluating an expression in postfix notation.
- C-6.20 Suppose you have three nonempty stacks R , S , and T . Describe a sequence of operations that results in S storing all elements originally in T below all of S 's original elements, with both sets of those elements in their original order. The final configuration for R should be the same as its original configuration. For example, if $R = (1, 2, 3)$, $S = (4, 5)$, and $T = (6, 7, 8, 9)$, when ordered from bottom to top, then the final configuration should have $R = (1, 2, 3)$ and $S = (6, 7, 8, 9, 4, 5)$.

- C-6.21** Describe a nonrecursive algorithm for enumerating all permutations of the numbers $\{1, 2, \dots, n\}$ using an explicit stack.
- C-6.22** Alice has three array-based stacks, A , B , and C , such that A has capacity 100, B has capacity 5, and C has capacity 3. Initially, A is full, and B and C are empty. Unfortunately, the person who programmed the class for these stacks made the push and pop methods private. The only method Alice can use is a static method, `dump( $S, T$ )`, which transfers (by iteratively applying the private pop and push methods) elements from stack S to stack T until either S becomes empty or T becomes full. So, for example, starting from our initial configuration and performing `dump( $A, C$ )` results in A now holding 97 elements and C holding 3. Describe a sequence of dump operations that starts from the initial configuration and results in B holding 4 elements at the end.
- C-6.23** Show how to use a stack S and a queue Q to generate all possible subsets of an n -element set T nonrecursively.
- C-6.24** Suppose you have a stack S containing n elements and a queue Q that is initially empty. Describe how you can use Q to scan S to see if it contains a certain element x , with the additional constraint that your algorithm must return the elements back to S in their original order. You may only use S , Q , and a constant number of other primitive variables.
- C-6.25** Describe how to implement the stack ADT using a single queue as an instance variable, and only constant additional local memory within the method bodies. What is the running time of the `push()`, `pop()`, and `top()` methods for your design?
- C-6.26** When implementing the `ArrayQueue` class, we initialized $f = 0$ (at line 5 of Code Fragment 6.10). What would happen had we initialized that field to some other positive value? What if we had initialized it to -1 ?
- C-6.27** Implement the `clone()` method for the `ArrayStack` class. (See Section 3.6 for a discussion of cloning data structures.)
- C-6.28** Implement the `clone()` method for the `ArrayQueue` class. (See Section 3.6 for a discussion of cloning data structures.)
- C-6.29** Implement a method with signature `concatenate(LinkedQueue<E> Q2)` for the `LinkedQueue<E>` class that takes all elements of $Q2$ and appends them to the end of the original queue. The operation should run in $O(1)$ time and should result in $Q2$ being an empty queue.
- C-6.30** Give a pseudocode description for an array-based implementation of the double-ended queue ADT. What is the running time for each operation?
- C-6.31** Describe how to implement the deque ADT using two stacks as the only instance variables. What are the running times of the methods?
- C-6.32** Suppose you have two nonempty stacks S and T and a deque D . Describe how to use D so that S stores all the elements of T below all of its original elements, with both sets of elements still in their original order.

- C-6.33 Alice has two circular queues, C and D , which can store integers. Bob gives Alice 50 odd integers and 50 even integers and insists that she stores all 100 integers in C and D . They then play a game where Bob picks C or D at random and then applies the `rotate()` method to the chosen queue a random number of times. If the last number to be rotated at the end of this game is odd, Bob wins. Otherwise, Alice wins. How can Alice allocate integers to queues to optimize her chances of winning? What is her chance of winning?
- C-6.34 Suppose Bob has four cows that he wants to take across a bridge, but only one yoke, which can hold up to two cows, side by side, tied to the yoke. The yoke is too heavy for him to carry across the bridge, but he can tie (and untie) cows to it in no time at all. Of his four cows, Mazie can cross the bridge in 2 minutes, Daisy can cross it in 4 minutes, Crazy can cross it in 10 minutes, and Lazy can cross it in 20 minutes. Of course, when two cows are tied to the yoke, they must go at the speed of the slower cow. Describe how Bob can get all his cows across the bridge in 34 minutes.

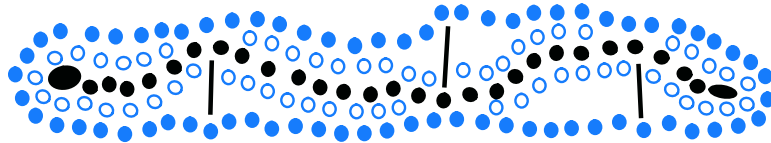
Projects

- P-6.35 Implement a program that can input an expression in postfix notation (see Exercise C-6.19) and output its value.
- P-6.36 When a share of common stock of some company is sold, the *capital gain* (or, sometimes, loss) is the difference between the share's selling price and the price originally paid to buy it. This rule is easy to understand for a single share, but if we sell multiple shares of stock bought over a long period of time, then we must identify the shares actually being sold. A standard accounting principle for identifying which shares of a stock were sold in such a case is to use a FIFO protocol—the shares sold are the ones that have been held the longest (indeed, this is the default method built into several personal finance software packages). For example, suppose we buy 100 shares at \$20 each on day 1, 20 shares at \$24 on day 2, 200 shares at \$36 on day 3, and then sell 150 shares on day 4 at \$30 each. Then applying the FIFO protocol means that of the 150 shares sold, 100 were bought on day 1, 20 were bought on day 2, and 30 were bought on day 3. The capital gain in this case would therefore be $100 \cdot 10 + 20 \cdot 6 + 30 \cdot (-6)$, or \$940. Write a program that takes as input a sequence of transactions of the form “buy x share(s) at \$ y each” or “sell x share(s) at \$ y each,” assuming that the transactions occur on consecutive days and the values x and y are integers. Given this input sequence, the output should be the total capital gain (or loss) for the entire sequence, using the FIFO protocol to identify shares.
- P-6.37 Design an ADT for a two-color, double-stack ADT that consists of two stacks—one “red” and one “blue”—and has as its operations color-coded versions of the regular stack ADT operations. For example, this ADT should support both a `redPush` operation and a `bluePush` operation. Give an efficient implementation of this ADT using a single array whose capacity is set at some value N that is assumed to always be larger than the sizes of the red and blue stacks combined.

- P-6.38 The introduction of Section 6.1 notes that stacks are often used to provide “undo” support in applications like a Web browser or text editor. While support for undo can be implemented with an unbounded stack, many applications provide only **limited** support for such an undo history, with a fixed-capacity stack. When push is invoked with the stack at full capacity, rather than throwing an exception, a more typical semantic is to accept the pushed element at the top while “leaking” the oldest element from the bottom of the stack to make room. Give an implementation of such a LeakyStack abstraction, using a circular array.
- P-6.39 Repeat the previous problem using a singly linked list for storage, and a maximum capacity specified as a parameter to the constructor.
- P-6.40 Give a complete implementation of the Deque ADT using a fixed-capacity array, so that each of the update methods runs in $O(1)$ time.

Chapter Notes

We were introduced to the approach of defining data structures first in terms of their ADTs and then in terms of concrete implementations by the classic books by Aho, Hopcroft, and Ullman [5, 6]. Exercises C-6.22, C-6.33, and C-6.34 are similar to interview questions said to be from a well-known software company. For further study of abstract data types, see Liskov and Guttag [67] and Demurjian [28].



Contents

7.1	The List ADT	258
7.2	Array Lists	260
7.2.1	Dynamic Arrays	263
7.2.2	Implementing a Dynamic Array	264
7.2.3	Amortized Analysis of Dynamic Arrays	265
7.2.4	Java's StringBuilder class	269
7.3	Positional Lists	270
7.3.1	Positions	272
7.3.2	The Positional List Abstract Data Type	272
7.3.3	Doubly Linked List Implementation	276
7.4	Iterators	282
7.4.1	The Iterable Interface and Java's For-Each Loop	283
7.4.2	Implementing Iterators	284
7.5	The Java Collections Framework	288
7.5.1	List Iterators in Java	289
7.5.2	Comparison to Our Positional List ADT	290
7.5.3	List-Based Algorithms in the Java Collections Framework	291
7.6	Sorting a Positional List	293
7.7	Case Study: Maintaining Access Frequencies	294
7.7.1	Using a Sorted List	294
7.7.2	Using a List with the Move-to-Front Heuristic	297
7.8	Exercises	300

7.1 The List ADT

In Chapter 6, we introduced the stack, queue, and deque abstract data types, and discussed how either an array or a linked list could be used for storage in an efficient concrete implementation of each. Each of those ADTs represents a linearly ordered sequence of elements. The deque is the most general of the three, yet even so, it only allows insertions and deletions at the front or back of a sequence.

In this chapter, we explore several abstract data types that represent a linear sequence of elements, but with more general support for adding or removing elements at arbitrary positions. However, designing a single abstraction that is well suited for efficient implementation with either an array or a linked list is challenging, given the very different nature of these two fundamental data structures.

Locations within an array are easily described with an integer *index*. Recall that an index of an element e in a sequence is equal to the number of elements before e in that sequence. By this definition, the first element of a sequence has index 0, and the last has index $n - 1$, assuming that n denotes the total number of elements. The notion of an element's index is well defined for a linked list as well, although we will see that it is not as convenient of a notion, as there is no way to efficiently access an element at a given index without traversing a portion of the linked list that depends upon the magnitude of the index.

With that said, Java defines a general interface, `java.util.List`, that includes the following index-based methods (and more):

- `size()`: Returns the number of elements in the list.
- `isEmpty()`: Returns a boolean indicating whether the list is empty.
- `get( $i$ )`: Returns the element of the list having index i ; an error condition occurs if i is not in range $[0, \text{size}() - 1]$.
- `set( $i, e$ )`: Replaces the element at index i with e , and returns the old element that was replaced; an error condition occurs if i is not in range $[0, \text{size}() - 1]$.
- `add( $i, e$ )`: Inserts a new element e into the list so that it has index i , moving all subsequent elements one index later in the list; an error condition occurs if i is not in range $[0, \text{size}()]$.
- `remove( $i$ )`: Removes and returns the element at index i , moving all subsequent elements one index earlier in the list; an error condition occurs if i is not in range $[0, \text{size}() - 1]$.

We note that the index of an existing element may change over time, as other elements are added or removed in front of it. We also draw attention to the fact that the range of valid indices for the `add` method includes the current size of the list, in which case the new element becomes the last.

Example 7.1 demonstrates a series of operations on a list instance, and Code Fragment 7.1 below provides a formal definition of our simplified version of the List interface; we use an `IndexOutOfBoundsException` to signal an invalid index argument.

Example 7.1: *We demonstrate operations on an initially empty list of characters.*

Method	Return Value	List Contents
<code>add(0, A)</code>	–	(A)
<code>add(0, B)</code>	–	(B, A)
<code>get(1)</code>	A	(B, A)
<code>set(2, C)</code>	“error”	(B, A)
<code>add(2, C)</code>	–	(B, A, C)
<code>add(4, D)</code>	“error”	(B, A, C)
<code>remove(1)</code>	A	(B, C)
<code>add(1, D)</code>	–	(B, D, C)
<code>add(1, E)</code>	–	(B, E, D, C)
<code>get(4)</code>	“error”	(B, E, D, C)
<code>add(4, F)</code>	–	(B, E, D, C, F)
<code>set(2, G)</code>	D	(B, E, G, C, F)
<code>get(2)</code>	G	(B, E, G, C, F)

```

1  /** A simplified version of the java.util.List interface. */
2  public interface List<E> {
3      /** Returns the number of elements in this list. */
4      int size();
5
6      /** Returns whether the list is empty. */
7      boolean isEmpty();
8
9      /** Returns (but does not remove) the element at index i. */
10     E get(int i) throws IndexOutOfBoundsException;
11
12     /** Replaces the element at index i with e, and returns the replaced element. */
13     E set(int i, E e) throws IndexOutOfBoundsException;
14
15     /** Inserts element e to be at index i, shifting all subsequent elements later. */
16     void add(int i, E e) throws IndexOutOfBoundsException;
17
18     /** Removes/returns the element at index i, shifting subsequent elements earlier. */
19     E remove(int i) throws IndexOutOfBoundsException;
20 }

```

Code Fragment 7.1: A simple version of the List interface.

7.2 Array Lists

An obvious choice for implementing the list ADT is to use an array A , where $A[i]$ stores (a reference to) the element with index i . We will begin by assuming that we have a fixed-capacity array, but in Section 7.2.1 describe a more advanced technique that effectively allows an array-based list to have unbounded capacity. Such an unbounded list is known as an **array list** in Java (or a **vector** in C++ and in the earliest versions of Java).

With a representation based on an array A , the $\text{get}(i)$ and $\text{set}(i, e)$ methods are easy to implement by accessing $A[i]$ (assuming i is a legitimate index). Methods $\text{add}(i, e)$ and $\text{remove}(i)$ are more time consuming, as they require shifting elements up or down to maintain our rule of always storing an element whose list index is i at index i of the array. (See Figure 7.1.) Our initial implementation of the `ArrayList` class follows in Code Fragments 7.2 and 7.3.

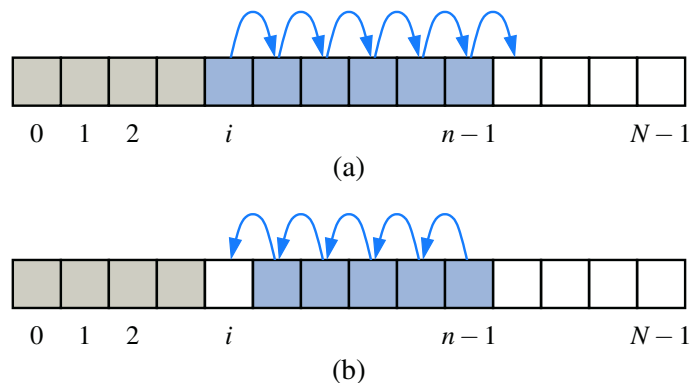


Figure 7.1: Array-based implementation of an array list that is storing n elements: (a) shifting up for an insertion at index i ; (b) shifting down for a removal at index i .

```

1 public class ArrayList<E> implements List<E> {
2     // instance variables
3     public static final int CAPACITY=16; // default array capacity
4     private E[] data; // generic array used for storage
5     private int size = 0; // current number of elements
6     // constructors
7     public ArrayList() { this(CAPACITY); } // constructs list with default capacity
8     public ArrayList(int capacity) { // constructs list with given capacity
9         data = (E[]) new Object[capacity]; // safe cast; compiler may give warning
10    }

```

Code Fragment 7.2: An implementation of a simple `ArrayList` class with bounded capacity. (Continues in Code Fragment 7.3.)

```

11 // public methods
12 /** Returns the number of elements in the array list. */
13 public int size() { return size; }
14 /** Returns whether the array list is empty. */
15 public boolean isEmpty() { return size == 0; }
16 /** Returns (but does not remove) the element at index i. */
17 public E get(int i) throws IndexOutOfBoundsException {
18     checkIndex(i, size);
19     return data[i];
20 }
21 /** Replaces the element at index i with e, and returns the replaced element. */
22 public E set(int i, E e) throws IndexOutOfBoundsException {
23     checkIndex(i, size);
24     E temp = data[i];
25     data[i] = e;
26     return temp;
27 }
28 /** Inserts element e to be at index i, shifting all subsequent elements later. */
29 public void add(int i, E e) throws IndexOutOfBoundsException,
30                 IllegalStateException {
31     checkIndex(i, size + 1);
32     if (size == data.length) // not enough capacity
33         throw new IllegalStateException("Array is full");
34     for (int k=size-1; k >= i; k--) // start by shifting rightmost
35         data[k+1] = data[k];
36     data[i] = e; // ready to place the new element
37     size++;
38 }
39 /** Removes/returns the element at index i, shifting subsequent elements earlier. */
40 public E remove(int i) throws IndexOutOfBoundsException {
41     checkIndex(i, size);
42     E temp = data[i];
43     for (int k=i; k < size-1; k++) // shift elements to fill hole
44         data[k] = data[k+1];
45     data[size-1] = null; // help garbage collection
46     size--;
47     return temp;
48 }
49 // utility method
50 /** Checks whether the given index is in the range [0, n-1]. */
51 protected void checkIndex(int i, int n) throws IndexOutOfBoundsException {
52     if (i < 0 || i >= n)
53         throw new IndexOutOfBoundsException("Illegal index: " + i);
54 }
55 }

```

Code Fragment 7.3: An implementation of a simple ArrayList class with bounded capacity. (Continued from Code Fragment 7.2.)

The Performance of a Simple Array-Based Implementation

Table 7.1 shows the worst-case running times of the methods of an array list with n elements realized by means of an array. Methods `isEmpty`, `size`, `get` and `set` clearly run in $O(1)$ time, but the insertion and removal methods can take much longer than this. In particular, `add( $i$ ,  $e$ )` runs in time $O(n)$. Indeed, the worst case for this operation occurs when i is 0, since all the existing n elements have to be shifted forward. A similar argument applies to method `remove( $i$ )`, which runs in $O(n)$ time, because we have to shift backward $n - 1$ elements in the worst case, when i is 0. In fact, assuming that each possible index is equally likely to be passed as an argument to these operations, their average running time is $O(n)$, for we will have to shift $n/2$ elements on average.

Method	Running Time
<code>size()</code>	$O(1)$
<code>isEmpty()</code>	$O(1)$
<code>get(i)</code>	$O(1)$
<code>set(i, e)</code>	$O(1)$
<code>add(i, e)</code>	$O(n)$
<code>remove(i)</code>	$O(n)$

Table 7.1: Performance of an array list with n elements realized by a fixed-capacity array.

Looking more closely at `add( $i$ ,  $e$ )` and `remove( $i$ )`, we note that they each run in time $O(n - i + 1)$, for only those elements at index i and higher have to be shifted up or down. Thus, inserting or removing an item at the end of an array list, using the methods `add( $n$ ,  $e$ )` and `remove( $n - 1$ )` respectively, takes $O(1)$ time each. Moreover, this observation has an interesting consequence for the adaptation of the array list ADT to the deque ADT from Section 6.3.1. If we do the “obvious” thing and store elements of a deque so that the first element is at index 0 and the last element at index $n - 1$, then methods `addLast` and `removeLast` of the deque each run in $O(1)$ time. However, methods `addFirst` and `removeFirst` of the deque each run in $O(n)$ time.

Actually, with a little effort, we can produce an array-based implementation of the array list ADT that achieves $O(1)$ time for insertions and removals at index 0, as well as insertions and removals at the end of the array list. Achieving this requires that we give up on our rule that an element at index i is stored in the array at index i , however, as we would have to use a circular array approach like the one we used in Section 6.2 to implement a queue. We leave the details of this implementation as Exercise C-7.25.

7.2.1 Dynamic Arrays

The `ArrayList` implementation in Code Fragments 7.2 and 7.3 (as well as those for a stack, queue, and deque from Chapter 6) has a serious limitation; it requires that a fixed maximum capacity be declared, throwing an exception if attempting to add an element once full. This is a major weakness, because if a user is unsure of the maximum size that will be reached for a collection, there is risk that either too large of an array will be requested, causing an inefficient waste of memory, or that too small of an array will be requested, causing a fatal error when exhausting that capacity.

Java's `ArrayList` class provides a more robust abstraction, allowing a user to add elements to the list, with no apparent limit on the overall capacity. To provide this abstraction, Java relies on an algorithmic sleight of hand that is known as a *dynamic array*.

In reality, elements of an `ArrayList` are stored in a traditional array, and the precise size of that traditional array must be internally declared in order for the system to properly allocate a consecutive piece of memory for its storage. For example, Figure 7.2 displays an array with 12 cells that might be stored in memory locations 2146 through 2157 on a computer system.



Figure 7.2: An array of 12 cells, allocated in memory locations 2146 through 2157.

Because the system may allocate neighboring memory locations to store other data, the capacity of an array cannot be increased by expanding into subsequent cells.

The first key to providing the semantics of an unbounded array is that an array list instance maintains an internal array that often has greater capacity than the current length of the list. For example, while a user may have created a list with five elements, the system may have reserved an underlying array capable of storing eight object references (rather than only five). This extra capacity makes it easy to add a new element to the end of the list by using the next available cell of the array.

If a user continues to add elements to a list, all reserved capacity in the underlying array will eventually be exhausted. In that case, the class requests a new, larger array from the system, and copies all references from the smaller array into the beginning of the new array. At that point in time, the old array is no longer needed, so it can be reclaimed by the system. Intuitively, this strategy is much like that of the hermit crab, which moves into a larger shell when it outgrows its previous one.

7.2.2 Implementing a Dynamic Array

We now demonstrate how our original version of the `ArrayList`, from Code Fragments 7.2 and 7.3, can be transformed to a dynamic-array implementation, having unbounded capacity. We rely on the same internal representation, with a traditional array A , that is initialized either to a default capacity or to one specified as a parameter to the constructor.

The key is to provide means to “grow” the array A , when more space is needed. Of course, we cannot actually grow that array, as its capacity is fixed. Instead, when a call to add a new element risks *overflowing* the current array, we perform the following additional steps:

1. Allocate a new array B with larger capacity.
2. Set $B[k] = A[k]$, for $k = 0, \dots, n - 1$, where n denotes current number of items.
3. Set $A = B$, that is, we henceforth use the new array to support the list.
4. Insert the new element in the new array.

An illustration of this process is shown in Figure 7.3.

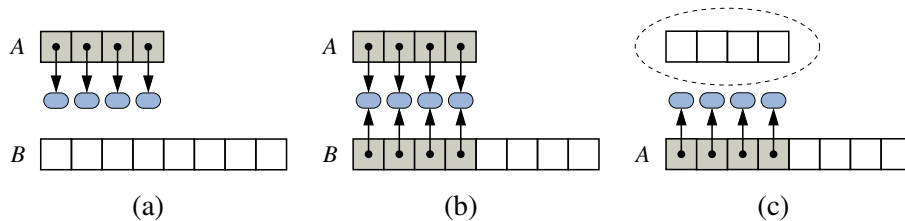


Figure 7.3: An illustration of “growing” a dynamic array: (a) create new array B ; (b) store elements of A in B ; (c) reassign reference A to the new array. Not shown is the future garbage collection of the old array, or the insertion of a new element.

Code Fragment 7.4 provides a concrete implementation of a `resize` method, which should be included as a protected method within the original `ArrayList` class. The instance variable `data` corresponds to array A in the above discussion, and local variable `temp` corresponds to array B .

```

/** Resizes internal array to have given capacity >= size. */
protected void resize(int capacity) {
    E[] temp = (E[]) new Object[capacity]; // safe cast; compiler may give warning
    for (int k=0; k < size; k++)
        temp[k] = data[k];
    data = temp; // start using the new array
}

```

Code Fragment 7.4: An implementation of the `ArrayList.resize` method.

The remaining issue to consider is how large of a new array to create. A commonly used rule is for the new array to have twice the capacity of the existing array that has been filled. In Section 7.2.3, we will provide a mathematical analysis to justify such a choice.

To complete the revision to our original ArrayList implementation, we redesign the add method so that it calls the new resize utility when detecting that the current array is filled (rather than throwing an exception). The revised version appears in Code Fragment 7.5.

```

28  /** Inserts element e to be at index i, shifting all subsequent elements later. */
29  public void add(int i, E e) throws IndexOutOfBoundsException {
30      checkIndex(i, size + 1);
31      if (size == data.length)                // not enough capacity
32          resize(2 * data.length);           // so double the current capacity
...    // rest of method unchanged...

```

Code Fragment 7.5: A revision to the ArrayList.add method, originally from Code Fragment 7.3, which calls the resize method of Code Fragment 7.4 when more capacity is needed.

Finally, we note that our original implementation of the ArrayList class includes two constructors: a default constructor that uses an initial capacity of 16, and a parameterized constructor that allows the caller to specify a capacity value. With the use of dynamic arrays, that capacity is no longer a fixed limit. Still, greater efficiency is achieved when a user selects an initial capacity that matches the actual size of a data set, as this can avoid time spent on intermediate array reallocations and potential space that is wasted by having too large of an array.

7.2.3 Amortized Analysis of Dynamic Arrays

In this section, we will perform a detailed analysis of the running time of operations on dynamic arrays. As a shorthand notation, let us refer to the insertion of an element to be the last element in an array list as a *push* operation.

The strategy of replacing an array with a new, larger array might at first seem slow, because a single push operation may require $\Omega(n)$ time to perform, where n is the current number of elements in the array. (Recall, from Section 4.3.1, that big-Omega notation, describes an asymptotic lower bound on the running time of an algorithm.) However, by doubling the capacity during an array replacement, our new array allows us to add n further elements before the array must be replaced again. In this way, there are many simple push operations for each expensive one (see Figure 7.4). This fact allows us to show that a series of push operations on an initially empty dynamic array is efficient in terms of its total running time.

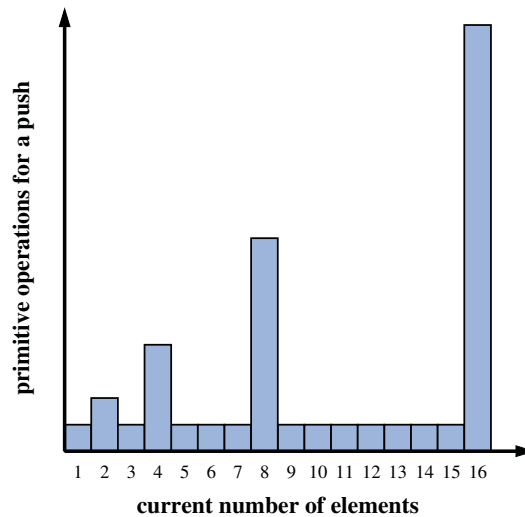


Figure 7.4: Running times of a series of push operations on a dynamic array.

Using an algorithmic design pattern called *amortization*, we show that performing a sequence of push operations on a dynamic array is actually quite efficient. To perform an *amortized analysis*, we use an accounting technique where we view the computer as a coin-operated appliance that requires the payment of one *cyber-dollar* for a constant amount of computing time. When an operation is executed, we should have enough cyber-dollars available in our current “bank account” to pay for that operation’s running time. Thus, the total amount of cyber-dollars spent for any computation will be proportional to the total time spent on that computation. The beauty of using this analysis method is that we can overcharge some operations in order to save up cyber-dollars to pay for others.

Proposition 7.2: *Let L be an initially empty array list with capacity one, implemented by means of a dynamic array that doubles in size when full. The total time to perform a series of n push operations in L is $O(n)$.*

Justification: Let us assume that one cyber-dollar is enough to pay for the execution of each push operation in L , excluding the time spent for growing the array. Also, let us assume that growing the array from size k to size $2k$ requires k cyber-dollars for the time spent initializing the new array. We shall charge each push operation three cyber-dollars. Thus, we overcharge each push operation that does not cause an overflow by two cyber-dollars. Think of the two cyber-dollars profited in an insertion that does not grow the array as being “stored” with the cell in which the element was inserted. An overflow occurs when the array L has 2^i elements, for some integer $i \geq 0$, and the size of the array used by the array representing L is 2^i . Thus, doubling the size of the array will require 2^i cyber-dollars. Fortunately, these cyber-dollars can be found stored in cells 2^{i-1} through $2^i - 1$. (See Figure 7.5.)

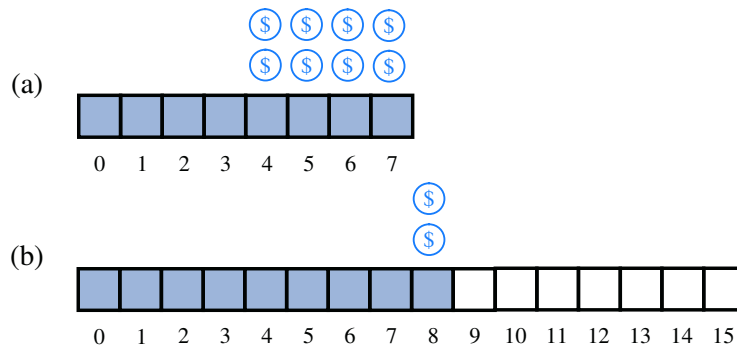


Figure 7.5: Illustration of a series of push operations on a dynamic array: (a) an 8-cell array is full, with two cyber-dollars “stored” at cells 4 through 7; (b) a push operation causes an overflow and a doubling of capacity. Copying the eight old elements to the new array is paid for by the cyber-dollars already stored in the table. Inserting the new element is paid for by one of the cyber-dollars charged to the current push operation, and the two cyber-dollars profited are stored at cell 8.

Note that the previous overflow occurred when the number of elements became larger than 2^{i-1} for the first time, and thus the cyber-dollars stored in cells 2^{i-1} through $2^i - 1$ have not yet been spent. Therefore, we have a valid amortization scheme in which each operation is charged three cyber-dollars and all the computing time is paid for. That is, we can pay for the execution of n push operations using $3n$ cyber-dollars. In other words, the amortized running time of each push operation is $O(1)$; hence, the total running time of n push operations is $O(n)$. ■

Geometric Increase in Capacity

Although the proof of Proposition 7.2 relies on the array being doubled each time it is expanded, the $O(1)$ amortized bound per operation can be proven for any geometrically increasing progression of array sizes. (See Section 2.2.3 for discussion of geometric progressions.) When choosing the geometric base, there exists a trade-off between runtime efficiency and memory usage. If the last insertion causes a resize event, with a base of 2 (i.e., doubling the array), the array essentially ends up twice as large as it needs to be. If we instead increase the array by only 25% of its current size (i.e., a geometric base of 1.25), we do not risk wasting as much memory in the end, but there will be more intermediate resize events along the way. Still it is possible to prove an $O(1)$ amortized bound, using a constant factor greater than the 3 cyber-dollars per operation used in the proof of Proposition 7.2 (see Exercise R-7.7). The key to the performance is that the amount of additional space is proportional to the current size of the array.

Beware of Arithmetic Progression

To avoid reserving too much space at once, it might be tempting to implement a dynamic array with a strategy in which a constant number of additional cells are reserved each time an array is resized. Unfortunately, the overall performance of such a strategy is significantly worse. At an extreme, an increase of only one cell causes each push operation to resize the array, leading to a familiar $1 + 2 + 3 + \dots + n$ summation and $\Omega(n^2)$ overall cost. Using increases of 2 or 3 at a time is slightly better, as portrayed in Figure 7.4, but the overall cost remains quadratic.

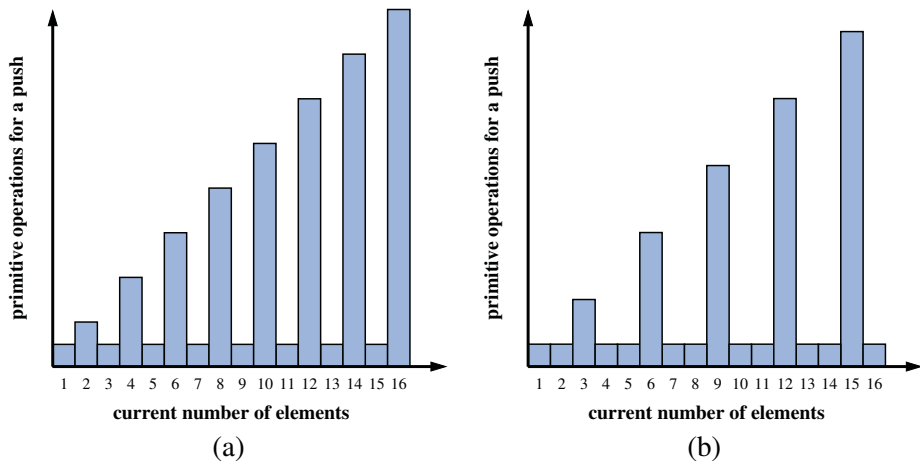


Figure 7.6: Running times of a series of push operations on a dynamic array using arithmetic progression of sizes. Part (a) assumes an increase of 2 in the size of the array, while part (b) assumes an increase of 3.

Using a *fixed* increment for each resize, and thus an arithmetic progression of intermediate array sizes, results in an overall time that is quadratic in the number of operations, as shown in the following proposition. In essence, even an increase in 10,000 cells per resize will become insignificant for large data sets.

Proposition 7.3: *Performing a series of n push operations on an initially empty dynamic array using a fixed increment with each resize takes $\Omega(n^2)$ time.*

Justification: Let $c > 0$ represent the fixed increment in capacity that is used for each resize event. During the series of n push operations, time will have been spent initializing arrays of size $c, 2c, 3c, \dots, mc$ for $m = \lceil n/c \rceil$, and therefore, the overall time is proportional to $c + 2c + 3c + \dots + mc$. By Proposition 4.3, this sum is

$$\sum_{i=1}^m ci = c \cdot \sum_{i=1}^m i = c \frac{m(m+1)}{2} \geq c \frac{\frac{n}{c}(\frac{n}{c} + 1)}{2} \geq \frac{1}{2c} \cdot n^2.$$

Therefore, performing the n push operations takes $\Omega(n^2)$ time. ■

Memory Usage and Shrinking an Array

Another consequence of the rule of a geometric increase in capacity when adding to a dynamic array is that the final array size is guaranteed to be proportional to the overall number of elements. That is, the data structure uses $O(n)$ memory. This is a very desirable property for a data structure.

If a container, such as an array list, provides operations that cause the removal of one or more elements, greater care must be taken to ensure that a dynamic array guarantees $O(n)$ memory usage. The risk is that repeated insertions may cause the underlying array to grow arbitrarily large, and that there will no longer be a proportional relationship between the actual number of elements and the array capacity after many elements are removed.

A robust implementation of such a data structure will shrink the underlying array, on occasion, while maintaining the $O(1)$ amortized bound on individual operations. However, care must be taken to ensure that the structure cannot rapidly oscillate between growing and shrinking the underlying array, in which case the amortized bound would not be achieved. In Exercise C-7.29, we explore a strategy in which the array capacity is halved whenever the number of actual element falls below one-fourth of that capacity, thereby guaranteeing that the array capacity is at most four times the number of elements; we explore the amortized analysis of such a strategy in Exercises C-7.30 and C-7.31.

7.2.4 Java's `StringBuilder` class

Near the beginning of Chapter 4, we described an experiment in which we compared two algorithms for composing a long string (Code Fragment 4.2). The first of those relied on repeated concatenation using the `String` class, and the second relied on use of Java's `StringBuilder` class. We observed the `StringBuilder` was significantly faster, with empirical evidence that suggested a quadratic running time for the algorithm with repeated concatenations, and a linear running time for the algorithm with the `StringBuilder`. We are now able to explain the theoretical underpinning for those observations.

The `StringBuilder` class represents a mutable string by storing characters in a dynamic array. With analysis similar to Proposition 7.2, it guarantees that a series of append operations resulting in a string of length n execute in a combined time of $O(n)$. (Insertions at positions other than the end of a string builder do not carry this guarantee, just as they do not for an `ArrayList`.)

In contrast, the repeated use of string concatenation requires quadratic time. We originally analyzed that algorithm on page 172 of Chapter 4. In effect, that approach is akin to a dynamic array with an arithmetic progression of size one, repeatedly copying all characters from one array to a new array with size one greater than before.

7.3 Positional Lists

When working with array-based sequences, integer indices provide an excellent means for describing the location of an element, or the location at which an insertion or deletion should take place. However, numeric indices are not a good choice for describing positions within a linked list because, knowing only an element's index, the only way to reach it is to traverse the list incrementally from its beginning or end, counting elements along the way.

Furthermore, indices are not a good abstraction for describing a more local view of a position in a sequence, because the index of an entry changes over time due to insertions or deletions that happen earlier in the sequence. For example, it may not be convenient to describe the location of a person waiting in line based on the index, as that requires knowledge of precisely how far away that person is from the front of the line. We prefer an abstraction, as characterized in Figure 7.7, in which there is some other means for describing a position.

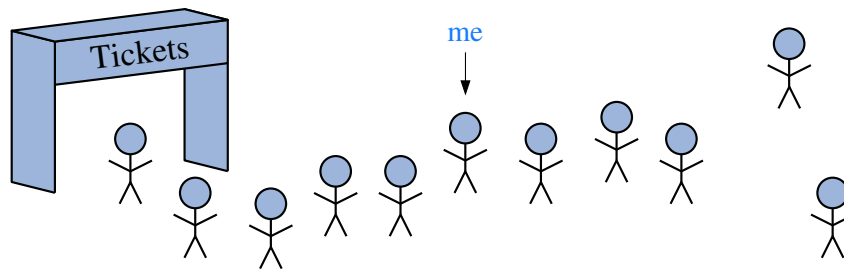


Figure 7.7: We wish to be able to identify the position of an element in a sequence without the use of an integer index. The label “me” represents some abstraction that identifies the position.

Our goal is to design an abstract data type that provides a user a way to refer to elements anywhere in a sequence, and to perform arbitrary insertions and deletions. This would allow us to efficiently describe actions such as a person deciding to leave the line before reaching the front, or allowing a friend to “cut” into line right behind him or her.

As another example, a text document can be viewed as a long sequence of characters. A word processor uses the abstraction of a **cursor** to describe a position within the document without explicit use of an integer index, allowing operations such as “delete the character at the cursor” or “insert a new character just after the cursor.” Furthermore, we may be able to refer to an inherent position within a document, such as the beginning of a particular chapter, without relying on a character index (or even a chapter number) that may change as the document evolves.

For these reasons, we temporarily forego the index-based methods of Java's formal List interface, and instead develop our own abstract data type that we denote as a **positional list**. Although a positional list is an abstraction, and need not rely on a linked list for its implementation, we certainly have a linked list in mind as we design the ADT, ensuring that it takes best advantage of particular capabilities of a linked list, such as $O(1)$ -time insertions and deletions at arbitrary positions (something that is not possible with an array-based sequence).

We face an immediate challenge in designing the ADT; to achieve constant time insertions and deletions at arbitrary locations, we effectively need a reference to the node at which an element is stored. It is therefore very tempting to develop an ADT in which a node reference serves as the mechanism for describing a position. In fact, our DoublyLinkedList class of Section 3.4.1 has methods addBetween and remove that accept node references as parameters; however, we intentionally declared those methods as private.

Unfortunately, the public use of nodes in the ADT would violate the object-oriented design principles of abstraction and encapsulation, which were introduced in Chapter 2. There are several reasons to prefer that we encapsulate the nodes of a linked list, for both our sake and for the benefit of users of our abstraction:

- It will be simpler for users of our data structure if they are not bothered with unnecessary details of our implementation, such as low-level manipulation of nodes, or our reliance on the use of sentinel nodes. Notice that to use the addBetween method of our DoublyLinkedList class to add a node at the beginning of a sequence, the header sentinel must be sent as a parameter.
- We can provide a more robust data structure if we do not permit users to directly access or manipulate the nodes. We can then ensure that users do not invalidate the consistency of a list by mismanaging the linking of nodes. A more subtle problem arises if a user were allowed to call the addBetween or remove method of our DoublyLinkedList class, sending a node that does not belong to the given list as a parameter. (Go back and look at that code and see why it causes a problem!)
- By better encapsulating the internal details of our implementation, we have greater flexibility to redesign the data structure and improve its performance. In fact, with a well-designed abstraction, we can provide a notion of a nonnumeric position, even if using an array-based sequence. (See Exercise C-7.43.)

Therefore, in defining the positional list ADT, we also introduce the concept of a **position**, which formalizes the intuitive notion of the “location” of an element relative to others in the list. (When we do use a linked list for the implementation, we will later see how we can privately use node references as natural manifestations of positions.)

7.3.1 Positions

To provide a general abstraction for the location of an element within a structure, we define a simple **position** abstract data type. A position supports the following single method:

`getElement()`: Returns the element stored at this position.

A position acts as a marker or token within a broader positional list. A position p , which is associated with some element e in a list L , does not change, even if the index of e changes in L due to insertions or deletions elsewhere in the list. Nor does position p change if we replace the element e stored at p with another element. The only way in which a position becomes invalid is if that position (and its element) are explicitly removed from the list.

Having a formal definition of a position type allows positions to serve as parameters to some methods and return values from other methods of the positional list ADT, which we next describe.

7.3.2 The Positional List Abstract Data Type

We now view a **positional list** as a collection of positions, each of which stores an element. The accessor methods provided by the positional list ADT include the following, for a list L :

`first()`: Returns the position of the first element of L (or null if empty).

`last()`: Returns the position of the last element of L (or null if empty).

`before( $p$ )`: Returns the position of L immediately before position p (or null if p is the first position).

`after( $p$ )`: Returns the position of L immediately after position p (or null if p is the last position).

`isEmpty()`: Returns true if list L does not contain any elements.

`size()`: Returns the number of elements in list L .

An error occurs if a position p , sent as a parameter to a method, is not a valid position for the list.

Note well that the `first()` and `last()` methods of the positional list ADT return the associated *positions*, not the *elements*. (This is in contrast to the corresponding `first` and `last` methods of the deque ADT.) The first element of a positional list can be determined by subsequently invoking the `getElement` method on that position, as `first().getElement`. The advantage of receiving a position as a return value is that we can subsequently use that position to traverse the list.

As a demonstration of a typical traversal of a positional list, Code Fragment 7.6 traverses a list, named `guests`, that stores string elements, and prints each element while traversing from the beginning of the list to the end.

```
1 Position<String> cursor = guests.first();
2 while (cursor != null) {
3     System.out.println(cursor.getElement());
4     cursor = guests.after(cursor);           // advance to the next position (if any)
5 }
```

Code Fragment 7.6: A traversal of a positional list.

This code relies on the convention that the null reference is returned when the `after` method is called upon the last position. (That return value is clearly distinguishable from any legitimate position.) The positional list ADT similarly indicates that the null value is returned when the `before` method is invoked at the front of the list, or when `first` or `last` methods are called upon an empty list. Therefore, the above code fragment works correctly even if the `guests` list is empty.

Updated Methods of a Positional List

The positional list ADT also includes the following *update* methods:

addFirst(*e*): Inserts a new element *e* at the front of the list, returning the position of the new element.

addLast(*e*): Inserts a new element *e* at the back of the list, returning the position of the new element.

addBefore(*p*, *e*): Inserts a new element *e* in the list, just before position *p*, returning the position of the new element.

addAfter(*p*, *e*): Inserts a new element *e* in the list, just after position *p*, returning the position of the new element.

set(*p*, *e*): Replaces the element at position *p* with element *e*, returning the element formerly at position *p*.

remove(*p*): Removes and returns the element at position *p* in the list, invalidating the position.

There may at first seem to be redundancy in the above repertoire of operations for the positional list ADT, since we can perform operation `addFirst(e)` with `addBefore(first(), e)`, and operation `addLast(e)` with `addAfter(last(), e)`. But these substitutions can only be done for a nonempty list.

Example 7.4: The following table shows a series of operations on an initially empty positional list storing integers. To identify position instances, we use variables such as p and q . For ease of exposition, when displaying the list contents, we use subscript notation to denote the position storing an element.

Method	Return Value	List Contents
addLast(8)	p	(8_p)
first()	p	(8_p)
addAfter(p , 5)	q	$(8_p, 5_q)$
before(q)	p	$(8_p, 5_q)$
addBefore(q , 3)	r	$(8_p, 3_r, 5_q)$
r .getElement()	3	$(8_p, 3_r, 5_q)$
after(p)	r	$(8_p, 3_r, 5_q)$
before(p)	null	$(8_p, 3_r, 5_q)$
addFirst(9)	s	$(9_s, 8_p, 3_r, 5_q)$
remove(last())	5	$(9_s, 8_p, 3_r)$
set(p , 7)	8	$(9_s, 7_p, 3_r)$
remove(q)	“error”	$(9_s, 7_p, 3_r)$

Java Interface Definitions

We are now ready to formalize the position ADT and positional list ADT. A Java Position interface, representing the position ADT, is given in Code Fragment 7.7. Following that, Code Fragment 7.8 presents a Java definition for our PositionalList interface. If the getElement() method is called on a Position instance that has previously been removed from its list, an IllegalStateException is thrown. If an invalid Position instance is sent as a parameter to a method of a PositionalList, an IllegalArgumentException is thrown. (Both of those exception types are defined in the standard Java hierarchy.)

```

1 public interface Position<E> {
2     /**
3      * Returns the element stored at this position.
4      *
5      * @return the stored element
6      * @throws IllegalStateException if position no longer valid
7      */
8     E getElement() throws IllegalStateException;
9 }

```

Code Fragment 7.7: The Position interface.

```
1  /** An interface for positional lists. */
2  public interface PositionalList<E> {
3
4      /** Returns the number of elements in the list. */
5      int size();
6
7      /** Tests whether the list is empty. */
8      boolean isEmpty();
9
10     /** Returns the first Position in the list (or null, if empty). */
11     Position<E> first();
12
13     /** Returns the last Position in the list (or null, if empty). */
14     Position<E> last();
15
16     /** Returns the Position immediately before Position p (or null, if p is first). */
17     Position<E> before(Position<E> p) throws IllegalArgumentException;
18
19     /** Returns the Position immediately after Position p (or null, if p is last). */
20     Position<E> after(Position<E> p) throws IllegalArgumentException;
21
22     /** Inserts element e at the front of the list and returns its new Position. */
23     Position<E> addFirst(E e);
24
25     /** Inserts element e at the back of the list and returns its new Position. */
26     Position<E> addLast(E e);
27
28     /** Inserts element e immediately before Position p and returns its new Position. */
29     Position<E> addBefore(Position<E> p, E e)
30         throws IllegalArgumentException;
31
32     /** Inserts element e immediately after Position p and returns its new Position. */
33     Position<E> addAfter(Position<E> p, E e)
34         throws IllegalArgumentException;
35
36     /** Replaces the element stored at Position p and returns the replaced element. */
37     E set(Position<E> p, E e) throws IllegalArgumentException;
38
39     /** Removes the element stored at Position p and returns it (invalidating p). */
40     E remove(Position<E> p) throws IllegalArgumentException;
41 }
```

Code Fragment 7.8: The PositionalList interface.

7.3.3 Doubly Linked List Implementation

Not surprisingly, our preferred implementation of the `PositionalList` interface relies on a doubly linked list. Although we implemented a `DoublyLinkedList` class in Chapter 3, that class does not adhere to the `PositionalList` interface.

In this section, we develop a concrete implementation of the `PositionalList` interface using a doubly linked list. The low-level details of our new linked-list representation, such as the use of header and trailer sentinels, will be identical to our earlier version; we refer the reader to Section 3.4 for a discussion of the doubly linked list operations. What differs in this section is our management of the positional abstraction.

The obvious way to identify locations within a linked list are node references. Therefore, we declare the nested `Node` class of our linked list so as to implement the `Position` interface, supporting the required `getElement` method. So the nodes *are* the positions. Yet, the `Node` class is declared as private, to maintain proper encapsulation. All of the public methods of the positional list rely on the `Position` type, so although we know we are sending and receiving nodes, these are only known to be positions from the outside; as a result, users of our class cannot call any method other than `getElement()`.

In Code Fragments 7.9–7.12, we define a `LinkedPositionalList` class, which implements the positional list ADT. We provide the following guide to that code:

- Code Fragment 7.9 contains the definition of the nested `Node<E>` class, which implements the `Position<E>` interface. Following that are the declaration of the instance variables of the outer `LinkedPositionalList` class and its constructor.
- Code Fragment 7.10 begins with two important utility methods that help us robustly cast between the `Position` and `Node` types. The `validate(p)` method is called anytime the user sends a `Position` instance as a parameter. It throws an exception if it determines that the position is invalid, and otherwise returns that instance, implicitly cast as a `Node`, so that methods of the `Node` class can subsequently be called. The private `position(node)` method is used when about to return a `Position` to the user. Its primary purpose is to make sure that we do not expose either sentinel node to a caller, returning a **null** reference in such a case. We rely on both of these private utility methods in the public accessor methods that follow.
- Code Fragment 7.11 provides most of the public update methods, relying on a private `addBetween` method to unify the implementations of the various insertion operations.
- Code Fragment 7.12 provides the public `remove` method. Note that it sets all fields of the removed node back to null—a condition we can later detect to recognize a defunct position.

```

1  /** Implementation of a positional list stored as a doubly linked list. */
2  public class LinkedPositionalList<E> implements PositionalList<E> {
3      //----- nested Node class -----
4      private static class Node<E> implements Position<E> {
5          private E element;           // reference to the element stored at this node
6          private Node<E> prev;         // reference to the previous node in the list
7          private Node<E> next;         // reference to the subsequent node in the list
8          public Node(E e, Node<E> p, Node<E> n) {
9              element = e;
10             prev = p;
11             next = n;
12         }
13         public E getElement() throws IllegalStateException {
14             if (next == null)           // convention for defunct node
15                 throw new IllegalStateException("Position no longer valid");
16             return element;
17         }
18         public Node<E> getPrev() {
19             return prev;
20         }
21         public Node<E> getNext() {
22             return next;
23         }
24         public void setElement(E e) {
25             element = e;
26         }
27         public void setPrev(Node<E> p) {
28             prev = p;
29         }
30         public void setNext(Node<E> n) {
31             next = n;
32         }
33     } //----- end of nested Node class -----
34
35     // instance variables of the LinkedPositionalList
36     private Node<E> header;           // header sentinel
37     private Node<E> trailer;          // trailer sentinel
38     private int size = 0;              // number of elements in the list
39
40     /** Constructs a new empty list. */
41     public LinkedPositionalList() {
42         header = new Node<>(null, null, null); // create header
43         trailer = new Node<>(null, header, null); // trailer is preceded by header
44         header.setNext(trailer);           // header is followed by trailer
45     }

```

Code Fragment 7.9: An implementation of the `LinkedPositionalList` class.
(Continues in Code Fragments 7.10–7.12.)

```

46 // private utilities
47 /** Validates the position and returns it as a node. */
48 private Node<E> validate(Position<E> p) throws IllegalArgumentException {
49     if (!(p instanceof Node)) throw new IllegalArgumentException("Invalid p");
50     Node<E> node = (Node<E>) p; // safe cast
51     if (node.getNext() == null) // convention for defunct node
52         throw new IllegalArgumentException("p is no longer in the list");
53     return node;
54 }
55
56 /** Returns the given node as a Position (or null, if it is a sentinel). */
57 private Position<E> position(Node<E> node) {
58     if (node == header || node == trailer)
59         return null; // do not expose user to the sentinels
60     return node;
61 }
62
63 // public accessor methods
64 /** Returns the number of elements in the linked list. */
65 public int size() { return size; }
66
67 /** Tests whether the linked list is empty. */
68 public boolean isEmpty() { return size == 0; }
69
70 /** Returns the first Position in the linked list (or null, if empty). */
71 public Position<E> first() {
72     return position(header.getNext());
73 }
74
75 /** Returns the last Position in the linked list (or null, if empty). */
76 public Position<E> last() {
77     return position(trailer.getPrev());
78 }
79
80 /** Returns the Position immediately before Position p (or null, if p is first). */
81 public Position<E> before(Position<E> p) throws IllegalArgumentException {
82     Node<E> node = validate(p);
83     return position(node.getPrev());
84 }
85
86 /** Returns the Position immediately after Position p (or null, if p is last). */
87 public Position<E> after(Position<E> p) throws IllegalArgumentException {
88     Node<E> node = validate(p);
89     return position(node.getNext());
90 }

```

Code Fragment 7.10: An implementation of the `LinkedPositionalList` class.
(Continued from Code Fragment 7.9; continues in Code Fragments 7.11 and 7.12.)

```

91 // private utilities
92 /** Adds element e to the linked list between the given nodes. */
93 private Position<E> addBetween(E e, Node<E> pred, Node<E> succ) {
94     Node<E> newest = new Node<>(e, pred, succ); // create and link a new node
95     pred.setNext(newest);
96     succ.setPrev(newest);
97     size++;
98     return newest;
99 }
100
101 // public update methods
102 /** Inserts element e at the front of the linked list and returns its new Position. */
103 public Position<E> addFirst(E e) {
104     return addBetween(e, header, header.getNext()); // just after the header
105 }
106
107 /** Inserts element e at the back of the linked list and returns its new Position. */
108 public Position<E> addLast(E e) {
109     return addBetween(e, trailer.getPrev(), trailer); // just before the trailer
110 }
111
112 /** Inserts element e immediately before Position p, and returns its new Position. */
113 public Position<E> addBefore(Position<E> p, E e)
114     throws IllegalArgumentException {
115     Node<E> node = validate(p);
116     return addBetween(e, node.getPrev(), node);
117 }
118
119 /** Inserts element e immediately after Position p, and returns its new Position. */
120 public Position<E> addAfter(Position<E> p, E e)
121     throws IllegalArgumentException {
122     Node<E> node = validate(p);
123     return addBetween(e, node, node.getNext());
124 }
125
126 /** Replaces the element stored at Position p and returns the replaced element. */
127 public E set(Position<E> p, E e) throws IllegalArgumentException {
128     Node<E> node = validate(p);
129     E answer = node.getElement();
130     node.setElement(e);
131     return answer;
132 }

```

Code Fragment 7.11: An implementation of the `LinkedPositionalList` class.
 (Continued from Code Fragments 7.9 and 7.10; continues in Code Fragment 7.12.)


```

133  /** Removes the element stored at Position p and returns it (invalidating p). */
134  public E remove(Position<E> p) throws IllegalArgumentException {
135      Node<E> node = validate(p);
136      Node<E> predecessor = node.getPrev();
137      Node<E> successor = node.getNext();
138      predecessor.setNext(successor);
139      successor.setPrev(predecessor);
140      size--;
141      E answer = node.getElement();
142      node.setElement(null);           // help with garbage collection
143      node.setNext(null);             // and convention for defunct node
144      node.setPrev(null);
145      return answer;
146  }
147  }

```

Code Fragment 7.12: An implementation of the `LinkedPositionalList` class.
(Continued from Code Fragments 7.9–7.11.)

The Performance of a Linked Positional List

The positional list ADT is ideally suited for implementation with a doubly linked list, as all operations run in worst-case constant time, as shown in Table 7.2. This is in stark contrast to the `ArrayList` structure (analyzed in Table 7.1), which requires linear time for insertions or deletions at arbitrary positions, due to the need for a loop to shift other elements.

Of course, our positional list does not support the index-based methods of the official `List` interface of Section 7.1. It is possible to add support for those methods by traversing the list while counting nodes (see Exercise C-7.38), but that requires time proportional to the sublist that is traversed.

Method	Running Time
<code>size()</code>	$O(1)$
<code>isEmpty()</code>	$O(1)$
<code>first()</code> , <code>last()</code>	$O(1)$
<code>before(p)</code> , <code>after(p)</code>	$O(1)$
<code>addFirst(e)</code> , <code>addLast(e)</code>	$O(1)$
<code>addBefore(p, e)</code> , <code>addAfter(p, e)</code>	$O(1)$
<code>set(p, e)</code>	$O(1)$
<code>remove(p)</code>	$O(1)$

Table 7.2: Performance of a positional list with n elements realized by a doubly linked list. The space usage is $O(n)$.

Implementing a Positional List with an Array

We can implement a positional list L using an array A for storage, but some care is necessary in designing objects that will serve as positions. At first glance, it would seem that a position p need only store the index i at which its associated element is stored within the array. We can then implement method `getElement( $p$ )` simply by returning $A[i]$. The problem with this approach is that the index of an element e changes when other insertions or deletions occur before it. If we have already returned a position p associated with element e that stores an outdated index i to a user, the wrong array cell would be accessed when the position was used. (Remember that positions in a positional list should always be defined relative to their neighboring positions, not their indices.)

Hence, if we are going to implement a positional list with an array, we need a different approach. We recommend the following representation. Instead of storing the elements of L directly in array A , we store a new kind of position object in each cell of A . A position p stores the element e as well as the current index i of that element within the list. Such a data structure is illustrated in Figure 7.8.

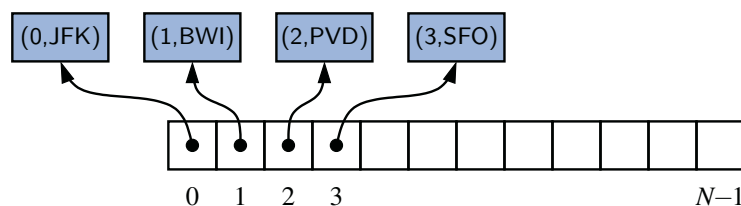


Figure 7.8: An array-based representation of a positional list.

With this representation, we can determine the index currently associated with a position, and we can determine the position currently associated with a specific index. We can therefore implement an accessor, such as `before( $p$ )`, by finding the index of the given position and using the array to find the neighboring position.

When an element is inserted or deleted somewhere in the list, we can loop through the array to update the index variable stored in all later positions in the list that are shifted during the update.

Efficiency Trade-Offs with an Array-Based Sequence

In this array implementation of a sequence, the `addFirst`, `addBefore`, `addAfter`, and `remove` methods take $O(n)$ time, because we have to shift position objects to make room for the new position or to fill in the hole created by the removal of the old position (just as in the insert and remove methods based on index). All the other position-based methods take $O(1)$ time.

7.4 Iterators

An *iterator* is a software design pattern that abstracts the process of scanning through a sequence of elements, one element at a time. The underlying elements might be stored in a container class, streaming through a network, or generated by a series of computations.

In order to unify the treatment and syntax for iterating objects in a way that is independent from a specific organization, Java defines the `java.util.Iterator` interface with the following two methods:

`hasNext()`: Returns true if there is at least one additional element in the sequence, and false otherwise.

`next()`: Returns the next element in the sequence.

The interface uses Java's generic framework, with the `next()` method returning a parameterized element type. For example, the `Scanner` class (described in Section 1.6) formally implements the `Iterator<String>` interface, with its `next()` method returning a `String` instance.

If the `next()` method of an iterator is called when no further elements are available, a `NoSuchElementException` is thrown. Of course, the `hasNext()` method can be used to detect that condition before calling `next()`.

The combination of these two methods allows a general loop construct for processing elements of the iterator. For example, if we let variable, `iter`, denote an instance of the `Iterator<String>` type, then we can write the following:

```
while (iter.hasNext()) {  
    String value = iter.next();  
    System.out.println(value);  
}
```

The `java.util.Iterator` interface contains a third method, which is *optionally* supported by some iterators:

`remove()`: Removes from the collection the element returned by the most recent call to `next()`. Throws an `IllegalStateException` if `next` has not yet been called, or if `remove` was already called since the most recent call to `next`.

This method can be used to filter a collection of elements, for example to discard all negative numbers from a data set.

For the sake of simplicity, we will not implement the `remove` method for most data structures in this book, but we will give two tangible examples later in this section. If removal is not supported, an `UnsupportedOperationException` is conventionally thrown.

7.4.1 The Iterable Interface and Java's For-Each Loop

A single iterator instance supports only one pass through a collection; calls to next can be made until all elements have been reported, but there is no way to “reset” the iterator back to the beginning of the sequence.

However, a data structure that wishes to allow repeated iterations can support a method that returns a *new* iterator, each time it is called. To provide greater standardization, Java defines another parameterized interface, named `Iterable`, that includes the following single method:

`iterator()`: Returns an iterator of the elements in the collection.

An instance of a typical collection class in Java, such as an `ArrayList`, is *iterable* (but not itself an *iterator*); it produces an iterator for its collection as the return value of the `iterator()` method. Each call to `iterator()` returns a new iterator instance, thereby allowing multiple (even simultaneous) traversals of a collection.

Java's `Iterable` class also plays a fundamental role in support of the “for-each” loop syntax (described in Section 1.5.2). The loop syntax,

```
for (ElementType variable : collection) {  
    loopBody                                // may refer to "variable"  
}
```

is supported for any instance, *collection*, of an iterable class. *ElementType* must be the type of object returned by its iterator, and *variable* will take on element values within the *loopBody*. Essentially, this syntax is shorthand for the following:

```
Iterator<ElementType> iter = collection.iterator();  
while (iter.hasNext()) {  
    ElementType variable = iter.next();  
    loopBody                                // may refer to "variable"  
}
```

We note that the iterator's `remove` method cannot be invoked when using the for-each loop syntax. Instead, we must explicitly use an iterator. As an example, the following loop can be used to remove all negative numbers from an `ArrayList` of floating-point values.

```
ArrayList<Double> data; // populate with random numbers (not shown)  
Iterator<Double> walk = data.iterator();  
while (walk.hasNext())  
    if (walk.next() < 0.0)  
        walk.remove();
```

7.4.2 Implementing Iterators

There are two general styles for implementing iterators that differ in terms of what work is done when the iterator instance is first created, and what work is done each time the iterator is advanced with a call to `next()`.

A **snapshot iterator** maintains its own private copy of the sequence of elements, which is constructed at the time the iterator object is created. It effectively records a “snapshot” of the sequence of elements at the time the iterator is created, and is therefore unaffected by any subsequent changes to the primary collection that may occur. Implementing snapshot iterators tends to be very easy, as it requires a simple traversal of the primary structure. The downside of this style of iterator is that it requires $O(n)$ time and $O(n)$ auxiliary space, upon construction, to copy and store a collection of n elements.

A **lazy iterator** is one that does not make an upfront copy, instead performing a piecewise traversal of the primary structure only when the `next()` method is called to request another element. The advantage of this style of iterator is that it can typically be implemented so the iterator requires only $O(1)$ space and $O(1)$ construction time. One downside (or feature) of a lazy iterator is that its behavior is affected if the primary structure is modified (by means other than by the iterator’s own `remove` method) before the iteration completes. Many of the iterators in Java’s libraries implement a “fail-fast” behavior that immediately invalidates such an iterator if its underlying collection is modified unexpectedly.

We will demonstrate how to implement iterators for both the `ArrayList` and `LinkedPositionalList` classes as examples. We implement lazy iterators for both, including support for the `remove` operation (but without any fail-fast guarantee).

Iterations with the `ArrayList` class

We begin by discussing iteration for the `ArrayList<E>` class. We will have it implement the `Iterable<E>` interface. (In fact, that requirement is already part of Java’s `List` interface.) Therefore, we must add an `iterator()` method to that class definition, which returns an instance of an object that implements the `Iterator<E>` interface. For this purpose, we define a new class, `ArrayIterator`, as a nonstatic nested class of `ArrayList` (i.e., an **inner class**, as described in Section 2.6). The advantage of having the iterator as an inner class is that it can access private fields (such as the array `A`) that are members of the containing list.

Our implementation is given in Code Fragment 7.13. The `iterator()` method of `ArrayList` returns a new instance of the inner `ArrayIterator` class. Each iterator maintains a field `j` that represents the index of the next element to be returned. It is initialized to 0, and when `j` reaches the size of the list, there are no more elements to return. In order to support element removal through the iterator, we also maintain a boolean variable that denotes whether a call to `remove` is currently permissible.

```

1  //----- nested ArrayIterator class -----
2  /**
3   * A (nonstatic) inner class. Note well that each instance contains an implicit
4   * reference to the containing list, allowing it to access the list's members.
5   */
6  private class ArrayIterator implements Iterator<E> {
7      private int j = 0;           // index of the next element to report
8      private boolean removable = false; // can remove be called at this time?
9
10     /**
11      * Tests whether the iterator has a next object.
12      * @return true if there are further objects, false otherwise
13      */
14     public boolean hasNext() { return j < size; } // size is field of outer instance
15
16     /**
17      * Returns the next object in the iterator.
18      *
19      * @return next object
20      * @throws NoSuchElementException if there are no further elements
21      */
22     public E next() throws NoSuchElementException {
23         if (j == size) throw new NoSuchElementException("No next element");
24         removable = true; // this element can subsequently be removed
25         return data[j++]; // post-increment j, so it is ready for future call to next
26     }
27
28     /**
29      * Removes the element returned by most recent call to next.
30      * @throws IllegalStateException if next has not yet been called
31      * @throws IllegalStateException if remove was already called since recent next
32      */
33     public void remove() throws IllegalStateException {
34         if (!removable) throw new IllegalStateException("nothing to remove");
35         ArrayList.this.remove(j-1); // that was the last one returned
36         j--; // next element has shifted one cell to the left
37         removable = false; // do not allow remove again until next is called
38     }
39 } //----- end of nested ArrayIterator class -----
40
41 /** Returns an iterator of the elements stored in the list. */
42 public Iterator<E> iterator() {
43     return new ArrayIterator(); // create a new instance of the inner class
44 }

```

Code Fragment 7.13: Code providing support for ArrayList iterators. (This should be nested within the ArrayList class definition of Code Fragments 7.2 and 7.3.)

Iterations with the `LinkedPositionalList` class

In support the concept of iteration with the `LinkedPositionalList` class, a first question is whether to support iteration of the *elements* of the list or the *positions* of the list. If we allow a user to iterate through all positions of the list, those positions could be used to access the underlying elements, so support for position iteration is more general. However, it is more standard for a container class to support iteration of the core elements, by default, so that the for-each loop syntax could be used to write code such as the following,

```
for (String guest : waitlist)
```

assuming that variable `waitlist` has type `LinkedPositionalList<String>`.

For maximum convenience, we will support *both* forms of iteration. We will have the standard `iterator()` method return an iterator of the elements of the list, so that our list class formally implements the `Iterable` interface for the declared element type.

For those wishing to iterate through the positions of a list, we will provide a new method, `positions()`. At first glance, it would seem a natural choice for such a method to return an `Iterator`. However, we prefer for the return type of that method to be an instance that is `Iterable` (and hence, has its own `iterator()` method that returns an iterator of positions). Our reason for the extra layer of complexity is that we wish for users of our class to be able to use a for-each loop with a simple syntax such as the following:

```
for (Position<String> p : waitlist.positions())
```

For this syntax to be legal, the return type of `positions()` must be `Iterable`.

Code Fragment 7.14 presents our new support for the iteration of positions and elements of a `LinkedPositionalList`. We define three new inner classes. The first of these is `PositionIterator`, providing the core functionality of our list iterations. Whereas the array list iterator maintained the index of the next element to be returned as a field, this class maintains the position of the next element to be returned (as well as the position of the most recently returned element, to support removal).

To support our goal of the `positions()` method returning an `Iterable` object, we define a trivial `PositionIterable` inner class, which simply constructs and returns a new `PositionIterator` object each time its `iterator()` method is called. The `positions()` method of the top-level class returns a new `PositionIterable` instance. Our framework relies heavily on these being inner classes, not static nested classes.

Finally, we wish to have the top-level `iterator()` method return an iterator of elements (not positions). Rather than reinvent the wheel, we trivially adapt the `PositionIterator` class to define a new `ElementIterator` class, which lazily manages a position iterator instance, while returning the element stored at each position when `next()` is called.

```

1  //----- nested PositionIterator class -----
2  private class PositionIterator implements Iterator<Position<E>> {
3      private Position<E> cursor = first(); // position of the next element to report
4      private Position<E> recent = null;    // position of last reported element
5      /** Tests whether the iterator has a next object. */
6      public boolean hasNext() { return (cursor != null); }
7      /** Returns the next position in the iterator. */
8      public Position<E> next() throws NoSuchElementException {
9          if (cursor == null) throw new NoSuchElementException("nothing left");
10         recent = cursor; // element at this position might later be removed
11         cursor = after(cursor);
12         return recent;
13     }
14     /** Removes the element returned by most recent call to next. */
15     public void remove() throws IllegalStateException {
16         if (recent == null) throw new IllegalStateException("nothing to remove");
17         LinkedPositionalList.this.remove(recent); // remove from outer list
18         recent = null; // do not allow remove again until next is called
19     }
20 } //----- end of nested PositionIterator class -----
21
22 //----- nested PositionIterable class -----
23 private class PositionIterable implements Iterable<Position<E>> {
24     public Iterator<Position<E>> iterator() { return new PositionIterator(); }
25 } //----- end of nested PositionIterable class -----
26
27 /** Returns an iterable representation of the list's positions. */
28 public Iterable<Position<E>> positions() {
29     return new PositionIterable(); // create a new instance of the inner class
30 }
31
32 //----- nested ElementIterator class -----
33 /* This class adapts the iteration produced by positions() to return elements. */
34 private class ElementIterator implements Iterator<E> {
35     Iterator<Position<E>> posIterator = new PositionIterator();
36     public boolean hasNext() { return posIterator.hasNext(); }
37     public E next() { return posIterator.next().getElement(); } // return element!
38     public void remove() { posIterator.remove(); }
39 }
40
41 /** Returns an iterator of the elements stored in the list. */
42 public Iterator<E> iterator() { return new ElementIterator(); }

```

Code Fragment 7.14: Support for providing iterations of positions and elements of a `LinkedPositionalList`. (This should be nested within the `LinkedPositionalList` class definition of Code Fragments 7.9–7.12.)

7.5 The Java Collections Framework

Java provides many data structure interfaces and classes, which together form the **Java Collections Framework**. This framework, which is part of the `java.util` package, includes versions of several of the data structures discussed in this book, some of which we have already discussed and others of which we will discuss later in this book. The root interface in the Java collections framework is named `Collection`. This is a general interface for any data structure, such as a list, that represents a collection of elements. The `Collection` interface includes many methods, including some we have already seen (e.g., `size()`, `isEmpty()`, `iterator()`). It is a superinterface for other interfaces in the Java Collections Framework that can hold elements, including the `java.util` interfaces `Deque`, `List`, and `Queue`, and other subinterfaces discussed later in this book, including `Set` (Section 10.5.1) and `Map` (Section 10.1).

The Java Collections Framework also includes concrete classes implementing various interfaces with a combination of properties and underlying representations. We summarize but a few of those classes in Table 7.3. For each, we denote which of the `Queue`, `Deque`, or `List` interfaces are implemented (possibly several). We also discuss several behavioral properties. Some classes enforce, or allow, a fixed capacity limit. Robust classes provide support for **concurrency**, allowing multiple processes to share use of a data structure in a thread-safe manner. If the structure is designated as **blocking**, a call to retrieve an element from an empty collection waits until some other process inserts an element. Similarly, a call to insert into a full blocking structure must wait until room becomes available.

Class	Interfaces			Properties			Storage	
	Queue	Deque	List	Capacity Limit	Thread-Safe	Blocking	Array	Linked List
<code>ArrayBlockingQueue</code>	✓			✓	✓	✓	✓	
<code>LinkedBlockingQueue</code>	✓			✓	✓	✓		✓
<code>ConcurrentLinkedQueue</code>	✓				✓		✓	
<code>ArrayDeque</code>	✓	✓					✓	
<code>LinkedBlockingDeque</code>	✓	✓		✓	✓	✓		✓
<code>ConcurrentLinkedDeque</code>	✓	✓			✓			✓
<code>ArrayList</code>			✓				✓	
<code>LinkedList</code>	✓	✓	✓					✓

Table 7.3: Several classes in the Java Collections Framework.

7.5.1 List Iterators in Java

The `java.util.LinkedList` class does not expose a position concept to users in its API, as we do in our positional list ADT. Instead, the preferred way to access and update a `LinkedList` object in Java, without using indices, is to use a `ListIterator` that is returned by the list's `listIterator()` method. Such an iterator provides forward and backward traversal methods as well as local update methods. It views its current position as being before the first element, between two elements, or after the last element. That is, it uses a list *cursor*, much like a screen cursor is viewed as being located between two characters on a screen. Specifically, the `java.util.ListIterator` interface includes the following methods:

- `add(e)`: Adds the element *e* at the current position of the iterator.
- `hasNext()`: Returns true if there is an element after the current position of the iterator.
- `hasPrevious()`: Returns true if there is an element before the current position of the iterator.
- `previous()`: Returns the element *e* before the current position and sets the current position to be before *e*.
- `next()`: Returns the element *e* after the current position and sets the current position to be after *e*.
- `nextIndex()`: Returns the index of the next element.
- `previousIndex()`: Returns the index of the previous element.
- `remove()`: Removes the element returned by the most recent next or previous operation.
- `set(e)`: Replaces the element returned by the most recent call to the next or previous operation with *e*.

It is risky to use multiple iterators over the same list while modifying its contents. If insertions, deletions, or replacements are required at multiple “places” in a list, it is safer to use positions to specify these locations. But the `java.util.LinkedList` class does not expose its position objects to the user. So, to avoid the risks of modifying a list that has created multiple iterators, the iterators have a “fail-fast” feature that invalidates such an iterator if its underlying collection is modified unexpectedly. For example, if a `java.util.LinkedList` object *L* has returned five different iterators and one of them modifies *L*, a `ConcurrentModificationException` is thrown if any of the other four is subsequently used. That is, Java allows many list iterators to be traversing a linked list *L* at the same time, but if one of them modifies *L* (using an `add`, `set`, or `remove` method), then all the other iterators for *L* become invalid. Likewise, if *L* is modified by one of its own update methods, then all existing iterators for *L* immediately become invalid.

7.5.2 Comparison to Our Positional List ADT

Java provides functionality similar to our array list and positional lists ADT in the `java.util.List` interface, which is implemented with an array in `java.util.ArrayList` and with a linked list in `java.util.LinkedList`.

Moreover, Java uses iterators to achieve a functionality similar to what our positional list ADT derives from positions. Table 7.4 shows corresponding methods between our (array and positional) list ADTs and the `java.util` interfaces `List` and `ListIterator` interfaces, with notes about their implementations in the `java.util` classes `ArrayList` and `LinkedList`.

Positional List ADT Method	java.util.List Method	ListIterator Method	Notes
<code>size()</code>	<code>size()</code>		$O(1)$ time
<code>isEmpty()</code>	<code>isEmpty()</code>		$O(1)$ time
	<code>get(<i>i</i>)</code>		<i>A</i> is $O(1)$, <i>L</i> is $O(\min\{i, n - i\})$
<code>first()</code>	<code>listIterator()</code>		first element is next
<code>last()</code>	<code>listIterator(size())</code>		last element is previous
<code>before(<i>p</i>)</code>		<code>previous()</code>	$O(1)$ time
<code>after(<i>p</i>)</code>		<code>next()</code>	$O(1)$ time
<code>set(<i>p</i>, <i>e</i>)</code>		<code>set(<i>e</i>)</code>	$O(1)$ time
	<code>set(<i>i</i>, <i>e</i>)</code>		<i>A</i> is $O(1)$, <i>L</i> is $O(\min\{i, n - i\})$
	<code>add(<i>i</i>, <i>e</i>)</code>		$O(n)$ time
<code>addFirst(<i>e</i>)</code>	<code>add(0, <i>e</i>)</code>		<i>A</i> is $O(n)$, <i>L</i> is $O(1)$
<code>addFirst(<i>e</i>)</code>	<code>addFirst(<i>e</i>)</code>		only exists in <i>L</i> , $O(1)$
<code>addLast(<i>e</i>)</code>	<code>add(<i>e</i>)</code>		$O(1)$ time
<code>addLast(<i>e</i>)</code>	<code>addLast(<i>e</i>)</code>		only exists in <i>L</i> , $O(1)$
<code>addAfter(<i>p</i>, <i>e</i>)</code>		<code>add(<i>e</i>)</code>	insertion is at cursor; <i>A</i> is $O(n)$, <i>L</i> is $O(1)$
<code>addBefore(<i>p</i>, <i>e</i>)</code>		<code>add(<i>e</i>)</code>	insertion is at cursor; <i>A</i> is $O(n)$, <i>L</i> is $O(1)$
<code>remove(<i>p</i>)</code>		<code>remove()</code>	deletion is at cursor; <i>A</i> is $O(n)$, <i>L</i> is $O(1)$
	<code>remove(<i>i</i>)</code>		<i>A</i> is $O(1)$, <i>L</i> is $O(\min\{i, n - i\})$

Table 7.4: Correspondences between methods in our positional list ADT and the `java.util` interfaces `List` and `ListIterator`. We use *A* and *L* as abbreviations for `java.util.ArrayList` and `java.util.LinkedList` (or their running times).

7.5.3 List-Based Algorithms in the Java Collections Framework

In addition to the classes that are provided in the Java Collections Framework, there are a number of simple algorithms that it provides as well. These algorithms are implemented as static methods in the `java.util.Collections` class (not to be confused with the `java.util.Collection` interface) and they include the following methods:

`copy( $L_{dest}$ ,  $L_{src}$ )`: Copies all elements of the L_{src} list into corresponding indices of the L_{dest} list.

`disjoint( $C$ ,  $D$ )`: Returns a boolean value indicating whether the collections C and D are disjoint.

`fill( $L$ ,  $e$ )`: Replaces each element of the list L with element e .

`frequency( $C$ ,  $e$ )`: Returns the number of elements in the collection C that are equal to e .

`max( $C$ )`: Returns the maximum element in the collection C , based on the natural ordering of its elements.

`min( $C$ )`: Returns the minimum element in the collection C , based on the natural ordering of its elements.

`replaceAll( $L$ ,  $e$ ,  $f$ )`: Replaces each element in L that is equal to e with element f .

`reverse( $L$ )`: Reverses the ordering of elements in the list L .

`rotate( $L$ ,  $d$ )`: Rotates the elements in the list L by the distance d (which can be negative), in a circular fashion.

`shuffle( $L$ )`: Pseudorandomly permutes the ordering of the elements in the list L .

`sort( $L$ )`: Sorts the list L , using the natural ordering of its elements.

`swap( $L$ ,  $i$ ,  $j$ )`: Swap the elements at indices i and j of list L .

Converting Lists into Arrays

Lists are a beautiful concept and they can be applied in a number of different contexts, but there are some instances where it would be useful if we could treat a list like an array. Fortunately, the `java.util.Collection` interface includes the following helpful methods for generating an array that has the same elements as the given collection:

`toArray()`: Returns an array of elements of type `Object` containing all the elements in this collection.

`toArray(A)`: Returns an array of elements of the same element type as `A` containing all the elements in this collection.

If the collection is a list, then the returned array will have its elements stored in the same order as that of the original list. Thus, if we have a useful array-based method that we want to use on a list or other type of collection, then we can do so by simply using that collection's `toArray()` method to produce an array representation of that collection.

Converting Arrays into Lists

In a similar vein, it is often useful to be able to convert an array into an equivalent list. Fortunately, the `java.util.Arrays` class includes the following method:

`asList(A)`: Returns a list representation of the array `A`, with the same element type as the elements of `A`.

The list returned by this method uses the array `A` as its internal representation for the list. So this list is guaranteed to be an array-based list and any changes made to it will automatically be reflected in `A`. Because of these types of side effects, use of the `asList` method should always be done with caution, so as to avoid unintended consequences. But, used with care, this method can often save us a lot of work. For instance, the following code fragment could be used to randomly shuffle an array of `Integer` objects, `arr`:

```
Integer[ ] arr = {1, 2, 3, 4, 5, 6, 7, 8};           // allowed by autoboxing
List<Integer> listArr = Arrays.asList(arr);
Collections.shuffle(listArr);                       // this has side effect of shuffling arr
```

It is worth noting that the array `A` sent to the `asList` method should be a reference type (hence, our use of `Integer` rather than `int` in the above example). This is because the `List` interface is generic, and requires that the element type be an object.

7.6 Sorting a Positional List

In Section 3.1.2, we introduced the *insertion-sort* algorithm in the context of an array-based sequence. In this section, we develop an implementation that operates on a `PositionalList`, relying on the same high-level algorithm in which each element is placed relative to a growing collection of previously sorted elements.

We maintain a variable named `marker` that represents the rightmost position of the currently sorted portion of a list. During each pass, we consider the position just past the `marker` as the `pivot` and consider where the pivot's element belongs relative to the sorted portion; we use another variable, named `walk`, to move leftward from the `marker`, as long as there remains a preceding element with value larger than the pivot's. A typical configuration of these variables is diagrammed in Figure 7.9. A Java implementation of this strategy is given in Code 7.15.

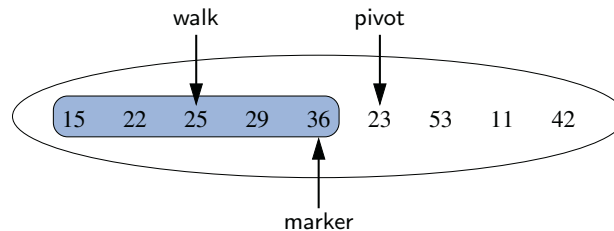


Figure 7.9: Overview of one step of our insertion-sort algorithm. The shaded elements, those up to and including `marker`, have already been sorted. In this step, the pivot's element should be relocated immediately before the `walk` position.

```

1  /** Insertion-sort of a positional list of integers into nondecreasing order */
2  public static void insertionSort(PositionalList<Integer> list) {
3      Position<Integer> marker = list.first();    // last position known to be sorted
4      while (marker != list.last()) {
5          Position<Integer> pivot = list.after(marker);
6          int value = pivot.getElement();          // number to be placed
7          if (value > marker.getElement())         // pivot is already sorted
8              marker = pivot;
9          else {                                   // must relocate pivot
10             Position<Integer> walk = marker;     // find leftmost item greater than value
11             while (walk != list.first() && list.before(walk).getElement() > value)
12                 walk = list.before(walk);
13             list.remove(pivot);                   // remove pivot entry and
14             list.addBefore(walk, value);          // reinsert value in front of walk
15         }
16     }
17 }

```

Code Fragment 7.15: Java code for performing insertion-sort on a positional list.

7.7 Case Study: Maintaining Access Frequencies

The positional list ADT is useful in a number of settings. For example, a program that simulates a game of cards could model each person's hand as a positional list (Exercise P-7.60). Since most people keep cards of the same suit together, inserting and removing cards from a person's hand could be implemented using the methods of the positional list ADT, with the positions being determined by a natural order of the suits. Likewise, a simple text editor embeds the notion of positional insertion and deletion, since such editors typically perform all updates relative to a *cursor*, which represents the current position in the list of characters of text being edited.

In this section, we will consider maintaining a collection of elements while keeping track of the number of times each element is accessed. Keeping such access counts allows us to know which elements are among the most popular. Examples of such scenarios include a Web browser that keeps track of a user's most accessed pages, or a music collection that maintains a list of the most frequently played songs for a user. We will model this with a new *favorites list ADT* that supports the size and isEmpty methods as well as the following:

`access(e)`: Accesses the element *e*, adding it to the favorites list if it is not already present, and increments its access count.

`remove(e)`: Removes element *e* from the favorites list, if present.

`getFavorites(k)`: Returns an iterable collection of the *k* most accessed elements.

7.7.1 Using a Sorted List

Our first approach for managing a list of favorites is to store elements in a linked list, keeping them in nonincreasing order of access counts. We access or remove an element by searching the list from the most frequently accessed to the least frequently accessed. Reporting the *k* most accessed elements is easy, as they are the first *k* entries of the list.

To maintain the invariant that elements are stored in nonincreasing order of access counts, we must consider how a single access operation may affect the order. The accessed element's count increases by one, and so it may become larger than one or more of its preceding neighbors in the list, thereby violating the invariant.

Fortunately, we can reestablish the sorted invariant using a technique similar to a single pass of the insertion-sort algorithm, introduced in the previous section. We can perform a backward traversal of the list, starting at the position of the element whose access count has increased, until we locate a valid position after which the element can be relocated.

Using the Composition Pattern

We wish to implement a favorites list by making use of a `PositionalList` for storage. If elements of the positional list were simply elements of the favorites list, we would be challenged to maintain access counts and to keep the proper count with the associated element as the contents of the list are reordered. We use a general object-oriented design pattern, the *composition pattern*, in which we define a single object that is composed of two or more other objects. (See, for example, Section 2.5.2.)

Specifically, we define a nonpublic nested class, `Item`, that stores the element and its access count as a single instance. We then maintain our favorites list as a `PositionalList` of *item* instances, so that the access count for a user's element is embedded alongside it in our representation. (An `Item` is never exposed to a user of a `FavoritesList`.)

```

1  /** Maintains a list of elements ordered according to access frequency. */
2  public class FavoritesList<E> {
3      // ----- nested Item class -----
4      protected static class Item<E> {
5          private E value;
6          private int count = 0;
7          /** Constructs new item with initial count of zero. */
8          public Item(E val) { value = val; }
9          public int getCount() { return count; }
10         public E getValue() { return value; }
11         public void increment() { count++; }
12     } //----- end of nested Item class -----
13
14     PositionalList<Item<E>> list = new LinkedPositionalList<>(); // list of Items
15     public FavoritesList() { } // constructs initially empty favorites list
16
17     // nonpublic utilities
18     /** Provides shorthand notation to retrieve user's element stored at Position p. */
19     protected E value(Position<Item<E>> p) { return p.getElement().getValue(); }
20
21     /** Provides shorthand notation to retrieve count of item stored at Position p. */
22     protected int count(Position<Item<E>> p) { return p.getElement().getCount(); }
23
24     /** Returns Position having element equal to e (or null if not found). */
25     protected Position<Item<E>> findPosition(E e) {
26         Position<Item<E>> walk = list.first();
27         while (walk != null && !e.equals(value(walk)))
28             walk = list.after(walk);
29         return walk;
30     }

```

Code Fragment 7.16: Class `FavoritesList`. (Continues in Code Fragment 7.17.)


```

31  /** Moves item at Position p earlier in the list based on access count. */
32  protected void moveUp(Position<Item<E>> p) {
33      int cnt = count(p);                                // revised count of accessed item
34      Position<Item<E>> walk = p;
35      while (walk != list.first() && count(list.before(walk)) < cnt)
36          walk = list.before(walk);                    // found smaller count ahead of item
37      if (walk != p)
38          list.addBefore(walk, list.remove(p));        // remove/reinsert item
39  }
40
41  // public methods
42  /** Returns the number of items in the favorites list. */
43  public int size() { return list.size(); }
44
45  /** Returns true if the favorites list is empty. */
46  public boolean isEmpty() { return list.isEmpty(); }
47
48  /** Accesses element e (possibly new), increasing its access count. */
49  public void access(E e) {
50      Position<Item<E>> p = findPosition(e);            // try to locate existing element
51      if (p == null)
52          p = list.addLast(new Item<E>(e));            // if new, place at end
53      p.getElement().increment();                      // always increment count
54      moveUp(p);                                       // consider moving forward
55  }
56
57  /** Removes element equal to e from the list of favorites (if found). */
58  public void remove(E e) {
59      Position<Item<E>> p = findPosition(e);            // try to locate existing element
60      if (p != null)
61          list.remove(p);
62  }
63
64  /** Returns an iterable collection of the k most frequently accessed elements. */
65  public Iterable<E> getFavorites(int k) throws IllegalArgumentException {
66      if (k < 0 || k > size())
67          throw new IllegalArgumentException("Invalid k");
68      PositionalList<E> result = new LinkedPositionalList<>();
69      Iterator<Item<E>> iter = list.iterator();
70      for (int j=0; j < k; j++)
71          result.addLast(iter.next().getValue());
72      return result;
73  }
74  }

```

Code Fragment 7.17: Class FavoritesList. (Continued from Code Fragment 7.16.)

7.7.2 Using a List with the Move-to-Front Heuristic

The previous implementation of a favorites list performs the $\text{access}(e)$ method in time proportional to the index of e in the favorites list. That is, if e is the k^{th} most popular element in the favorites list, then accessing it takes $O(k)$ time. In many real-life access sequences (e.g., Web pages visited by a user), once an element is accessed it is more likely to be accessed again in the near future. Such scenarios are said to possess *locality of reference*.

A *heuristic*, or rule of thumb, that attempts to take advantage of the locality of reference that is present in an access sequence is the *move-to-front heuristic*. To apply this heuristic, each time we access an element we move it all the way to the front of the list. Our hope, of course, is that this element will be accessed again in the near future. Consider, for example, a scenario in which we have n elements and the following series of n^2 accesses:

- element 1 is accessed n times.
- element 2 is accessed n times.
- ...
- element n is accessed n times.

If we store the elements sorted by their access counts, inserting each element the first time it is accessed, then

- each access to element 1 runs in $O(1)$ time.
- each access to element 2 runs in $O(2)$ time.
- ...
- each access to element n runs in $O(n)$ time.

Thus, the total time for performing the series of accesses is proportional to

$$n + 2n + 3n + \cdots + n \cdot n = n(1 + 2 + 3 + \cdots + n) = n \cdot \frac{n(n+1)}{2},$$

which is $O(n^3)$.

On the other hand, if we use the move-to-front heuristic, inserting each element the first time it is accessed, then

- each subsequent access to element 1 takes $O(1)$ time.
- each subsequent access to element 2 takes $O(1)$ time.
- ...
- each subsequent access to element n runs in $O(1)$ time.

So the running time for performing all the accesses in this case is $O(n^2)$. Thus, the move-to-front implementation has faster access times for this scenario. Still, the move-to-front approach is just a heuristic, for there are access sequences where using the move-to-front approach is slower than simply keeping the favorites list ordered by access counts.

The Trade-Offs with the Move-to-Front Heuristic

If we no longer maintain the elements of the favorites list ordered by their access counts, when we are asked to find the k most accessed elements, we need to search for them. We will implement the `getFavorites( $k$ )` method as follows:

1. We copy all entries of our favorites list into another list, named `temp`.
2. We scan the `temp` list k times. In each scan, we find the entry with the largest access count, remove this entry from `temp`, and add it to the results.

This implementation of method `getFavorites( $k$ )` takes $O(kn)$ time. Thus, when k is a constant, method `getFavorites( $k$ )` runs in $O(n)$ time. This occurs, for example, when we want to get the “top ten” list. However, if k is proportional to n , then the method `getFavorites( $k$ )` runs in $O(n^2)$ time. This occurs, for example, when we want a “top 25%” list.

In Chapter 9 we will introduce a data structure that will allow us to implement `getFavorites` in $O(n + k \log n)$ time (see Exercise P-9.51), and more advanced techniques could be used to perform `getFavorites` in $O(n + k \log k)$ time.

We could easily achieve $O(n \log n)$ time if we use a standard sorting algorithm to reorder the temporary list before reporting the top k (see Chapter 12); this approach would be preferred to the original in the case that k is $\Omega(\log n)$. (Recall the big-Omega notation introduced in Section 4.3.1 to give an asymptotic lower bound on the running time of an algorithm.) There is a specialized sorting algorithm (see Section 12.3.2) that can take advantage of the fact that access counts are integers in order to achieve $O(n)$ time for `getFavorites`, for any value of k .

Implementing the Move-to-Front Heuristic in Java

We give an implementation of a favorites list using the move-to-front heuristic in Code Fragment 7.18. The new `FavoritesListMTF` class inherits most of its functionality from the original `FavoritesList` as a base class.

By our original design, the access method of the original class relies on a protected utility named `moveUp` to enact the potential shifting of an element forward in the list, after its access count had been incremented. Therefore, we implement the move-to-front heuristic by simply overriding the `moveUp` method so that each accessed element is moved directly to the front of the list (if not already there). This action is easily implemented by means of the positional list ADT.

The more complex portion of our `FavoritesListMTF` class is the new definition for the `getFavorites` method. We rely on the first of the approaches outlined above, inserting copies of the items into a temporary list and then repeatedly finding, reporting, and removing an element that has the largest access count of those remaining.

```

1  /** Maintains a list of elements ordered with move-to-front heuristic. */
2  public class FavoritesListMTF<E> extends FavoritesList<E> {
3
4      /** Moves accessed item at Position p to the front of the list. */
5      protected void moveUp(Position<Item<E>> p) {
6          if (p != list.first())
7              list.addFirst(list.remove(p));           // remove/reinsert item
8      }
9
10     /** Returns an iterable collection of the k most frequently accessed elements. */
11     public Iterable<E> getFavorites(int k) throws IllegalArgumentException {
12         if (k < 0 || k > size())
13             throw new IllegalArgumentException("Invalid k");
14
15         // we begin by making a copy of the original list
16         PositionalList<Item<E>> temp = new LinkedPositionalList<>();
17         for (Item<E> item : list)
18             temp.addLast(item);
19
20         // we repeated find, report, and remove element with largest count
21         PositionalList<E> result = new LinkedPositionalList<>();
22         for (int j=0; j < k; j++) {
23             Position<Item<E>> highPos = temp.first();
24             Position<Item<E>> walk = temp.after(highPos);
25             while (walk != null) {
26                 if (count(walk) > count(highPos))
27                     highPos = walk;
28                 walk = temp.after(walk);
29             }
30             // we have now found element with highest count
31             result.addLast(value(highPos));
32             temp.remove(highPos);
33         }
34         return result;
35     }
36 }

```

Code Fragment 7.18: Class FavoritesListMTF implementing the move-to-front heuristic. This class extends FavoritesList (Code Fragments 7.16 and 7.17) and overrides methods moveUp and getFavorites.

7.8 Exercises

Reinforcement

- R-7.1 Draw a representation, akin to Example 7.1, of an initially empty list L after performing the following sequence of operations: `add(0, 4)`, `add(0, 3)`, `add(0, 2)`, `add(2, 1)`, `add(1, 5)`, `add(1, 6)`, `add(3, 7)`, `add(0, 8)`.
- R-7.2 Give an implementation of the stack ADT using an array list for storage.
- R-7.3 Give an implementation of the deque ADT using an array list for storage.
- R-7.4 Give a justification of the running times shown in Table 7.1 for the methods of an array list implemented with a (nonexpanding) array.
- R-7.5 The `java.util.ArrayList` includes a method, `trimToSize()`, that replaces the underlying array with one whose capacity precisely equals the number of elements currently in the list. Implement such a method for our dynamic version of the `ArrayList` class from Section 7.2.
- R-7.6 Redo the justification of Proposition 7.2 assuming that the cost of growing the array from size k to size $2k$ is $3k$ cyber-dollars. How much should each push operation be charged to make the amortization work?
- R-7.7 Consider an implementation of the array list ADT using a dynamic array, but instead of copying the elements into an array of double the size (that is, from N to $2N$) when its capacity is reached, we copy the elements into an array with $\lceil N/4 \rceil$ additional cells, going from capacity N to $N + \lceil N/4 \rceil$. Show that performing a sequence of n push operations (that is, insertions at the end) still runs in $O(n)$ time in this case.
- R-7.8 Suppose we are maintaining a collection C of elements such that, each time we add a new element to the collection, we copy the contents of C into a new array list of just the right size. What is the running time of adding n elements to an initially empty collection C in this case?
- R-7.9 The `add` method for a dynamic array, as described in Code Fragment 7.5, has the following inefficiency. In the case when a resize occurs, the resize operation takes time to copy all the elements from the old array to a new array, and then the subsequent loop in the body of `add` shifts some of them to make room for a new element. Give an improved implementation of the `add` method, so that, in the case of a resize, the elements are copied into their final place in the new array (that is, no shifting is done).
- R-7.10 Reimplement the `ArrayStack` class, from Section 6.1.2, using dynamic arrays to support unlimited capacity.
- R-7.11 Describe an implementation of the positional list methods `addLast` and `addBefore` realized by using only methods in the set $\{\text{isEmpty}, \text{first}, \text{last}, \text{before}, \text{after}, \text{addAfter}, \text{addFirst}\}$.

- R-7.12 Suppose we want to extend the `PositionalList` abstract data type with a method, `indexOf( $p$ )`, that returns the current index of the element stored at position p . Show how to implement this method using only other methods of the `PositionalList` interface (not details of our `LinkedPositionalList` implementation).
- R-7.13 Suppose we want to extend the `PositionalList` abstract data type with a method, `findPosition( $e$ )`, that returns the first position containing an element equal to e (or null if no such position exists). Show how to implement this method using only existing methods of the `PositionalList` interface (not details of our `LinkedPositionalList` implementation).
- R-7.14 The `LinkedPositionalList` implementation of Code Fragments 7.9–7.12 does not do any error checking to test if a given position p is actually a member of the relevant list. Give a detailed explanation of the effect of a call `L.addAfter( $p$ ,  $e$ )` on a list L , yet with a position p that belongs to some other list M .
- R-7.15 To better model a FIFO queue in which entries may be deleted before reaching the front, design a `LinkedPositionalQueue` class that supports the complete queue ADT, yet with `enqueue` returning a position instance and support for a new method, `remove( $p$ )`, that removes the element associated with position p from the queue. You may use the adapter design pattern (Section 6.1.3), using a `LinkedPositionalList` as your storage.
- R-7.16 Describe how to implement a method, `alternateIterator()`, for a positional list that returns an iterator that reports only those elements having even index in the list.
- R-7.17 Redesign the `Progression` class, from Section 2.2.3, so that it formally implements the `Iterator<long>` interface.
- R-7.18 The `java.util.Collection` interface includes a method, `contains( $o$ )`, that returns true if the collection contains any object that equals Object o . Implement such a method in the `ArrayList` class of Section 7.2.
- R-7.19 The `java.util.Collection` interface includes a method, `clear()`, that removes all elements from a collection. Implement such a method in the `ArrayList` class of Section 7.2.
- R-7.20 Demonstrate how to use the `java.util.Collections.reverse` method to reverse an array of objects.
- R-7.21 Given the set of element $\{a, b, c, d, e, f\}$ stored in a list, show the final state of the list, assuming we use the move-to-front heuristic and access the elements according to the following sequence: $(a, b, c, d, e, f, a, c, f, b, d, e)$.
- R-7.22 Suppose that we have made kn total accesses to the elements in a list L of n elements, for some integer $k \geq 1$. What are the minimum and maximum number of elements that have been accessed fewer than k times?
- R-7.23 Let L be a list of n items maintained according to the move-to-front heuristic. Describe a series of $O(n)$ accesses that will reverse L .
- R-7.24 Implement a `resetCounts()` method for the `FavoritesList` class that resets all elements' access counts to zero (while leaving the order of the list unchanged).

Creativity

- C-7.25 Give an array-based list implementation, with fixed capacity, treating the array circularly so that it achieves $O(1)$ time for insertions and removals at index 0, as well as insertions and removals at the end of the array list. Your implementation should also provide for a constant-time get method.
- C-7.26 Complete the previous exercise, except using a dynamic array to provide unbounded capacity.
- C-7.27 Modify our ArrayList implementation to support the Cloneable interface, as described in Section 3.6.
- C-7.28 In Section 7.5.3, we demonstrated how the Collections.shuffle method can be adapted to shuffle a reference-type array. Give a direct implementation of a shuffle method for an array of **int** values. You may use the method, nextInt(n) of the Random class, which returns a random number between 0 and $n - 1$, inclusive. Your method should guarantee that every possible ordering is equally likely. What is the running time of your method?
- C-7.29 Revise the array list implementation given in Section 7.2.1 so that when the actual number of elements, n , in the array goes below $N/4$, where N is the array capacity, the array shrinks to half its size.
- C-7.30 Prove that when using a dynamic array that grows and shrinks as in the previous exercise, the following series of $2n$ operations takes $O(n)$ time: n insertions at the end of an initially empty list, followed by n deletions, each from the end of the list.
- C-7.31 Give a formal proof that any sequence of n push or pop operations (that is, insertions or deletions at the end) on an initially empty dynamic array takes $O(n)$ time, if using the strategy described in Exercise C-7.29.
- C-7.32 Consider a variant of Exercise C-7.29, in which an array of capacity N is resized to capacity precisely that of the number of elements, any time the number of elements in the array goes strictly below $N/4$. Give a formal proof that any sequence of n push or pop operations on an initially empty dynamic array takes $O(n)$ time.
- C-7.33 Consider a variant of Exercise C-7.29, in which an array of capacity N , is resized to capacity precisely that of the number of elements, any time the number of elements in the array goes strictly below $N/2$. Show that there exists a sequence of n push and pop operations that requires $\Omega(n^2)$ time to execute.
- C-7.34 Describe how to implement the queue ADT using two stacks as instance variables, such that all queue operations execute in amortized $O(1)$ time. Give a formal proof of the amortized bound.
- C-7.35 Reimplement the ArrayQueue class, from Section 6.2.2, using dynamic arrays to support unlimited capacity. Be especially careful about the treatment of a circular array when resizing.

- C-7.36 Suppose we want to extend the `PositionalList` interface to include a method, `positionAtIndex( $i$ )`, that returns the position of the element having index i (or throws an `IndexOutOfBoundsException`, if warranted). Show how to implement this method, using only existing methods of the `PositionalList` interface, by traversing the appropriate number of steps from the front of the list.
- C-7.37 Repeat the previous problem, but use knowledge of the size of the list to traverse from the end of the list that is closest to the desired index.
- C-7.38 Explain how any implementation of the `PositionalList` ADT can be made to support all methods of the `List` ADT, described in Section 7.1, assuming an implementation is given for the `positionAtIndex( $i$ )` method, proposed in Exercise C-7.36.
- C-7.39 Suppose we want to extend the `PositionalList` abstract data type with a method, `moveToFront( $p$ )`, that moves the element at position p to the front of a list (if not already there), while keeping the relative order of the remaining elements unchanged. Show how to implement this method using only existing methods of the `PositionalList` interface (not details of our `LinkedPositionalList` implementation).
- C-7.40 Redo the previous problem, but providing an implementation within the class `LinkedPositionalList` that does not create or destroy any nodes.
- C-7.41 Modify our `LinkedPositionalList` implementation to support the `Cloneable` interface, as described in Section 3.6.
- C-7.42 Describe a nonrecursive method for reversing a positional list represented with a doubly linked list using a single pass through the list.
- C-7.43 Page 281 describes an *array-based* representation for implementing the positional list ADT. Give a pseudocode description of the `addBefore` method for that representation.
- C-7.44 Describe a method for performing a *card shuffle* of a list of $2n$ elements, by converting it into two lists. A card shuffle is a permutation where a list L is cut into two lists, L_1 and L_2 , where L_1 is the first half of L and L_2 is the second half of L , and then these two lists are merged into one by taking the first element in L_1 , then the first element in L_2 , followed by the second element in L_1 , the second element in L_2 , and so on.
- C-7.45 How might the `LinkedPositionalList` class be redesigned to detect the error described in Exercise R-7.14.
- C-7.46 Modify the `LinkedPositionalList` class to support a method `swap( $p$ ,  $q$ )` that causes the underlying nodes referenced by positions p and q to be exchanged for each other. Relink the existing nodes; do not create any new nodes.
- C-7.47 An array is *sparse* if most of its entries are **null**. A list L can be used to implement such an array, A , efficiently. In particular, for each nonnull cell $A[i]$, we can store a pair (i, e) in L , where e is the element stored at $A[i]$. This approach allows us to represent A using $O(m)$ storage, where m is the number of nonnull entries in A . Describe and analyze efficient ways of performing the methods of the array list ADT on such a representation.

- C-7.48 Design a circular positional list ADT that abstracts a circularly linked list in the same way that the positional list ADT abstracts a doubly linked list.
- C-7.49 Provide an implementation of the `listiterator()` method, in the context of the class `LinkedPositionalList`, that returns an object that supports the `java.util.ListIterator` interface described in Section 7.5.1.
- C-7.50 Describe a scheme for creating list iterators that *fail fast*, that is, they all become invalid as soon as the underlying list changes.
- C-7.51 There is a simple algorithm, called *bubble-sort*, for sorting a list L of n comparable elements. This algorithm scans the list $n - 1$ times, where, in each scan, the algorithm compares the current element with the next one and swaps them if they are out of order. Give a pseudocode description of bubble-sort that is as efficient as possible assuming L is implemented with a doubly linked list. What is the running time of this algorithm?
- C-7.52 Redo Exercise C-7.51 assuming L is implemented with an array list.
- C-7.53 Describe an efficient method for maintaining a favorites list L , with the move-to-front heuristic, such that elements that have not been accessed in the most recent n accesses are automatically purged from the list.
- C-7.54 Suppose we have an n -element list L maintained according to the move-to-front heuristic. Describe a sequence of n^2 accesses that is guaranteed to take $\Omega(n^3)$ time to perform on L .
- C-7.55 A useful operation in databases is the *natural join*. If we view a database as a list of *ordered* pairs of objects, then the natural join of databases A and B is the list of all ordered triples (x, y, z) such that the pair (x, y) is in A and the pair (y, z) is in B . Describe and analyze an efficient algorithm for computing the natural join of a list A of n pairs and a list B of m pairs.
- C-7.56 When Bob wants to send Alice a message M on the Internet, he breaks M into n *data packets*, numbers the packets consecutively, and injects them into the network. When the packets arrive at Alice's computer, they may be out of order, so Alice must assemble the sequence of n packets in order before she can be sure she has the entire message. Describe an efficient scheme for Alice to do this. What is the running time of this algorithm?
- C-7.57 Implement the `FavoritesList` class using an array list.

Projects

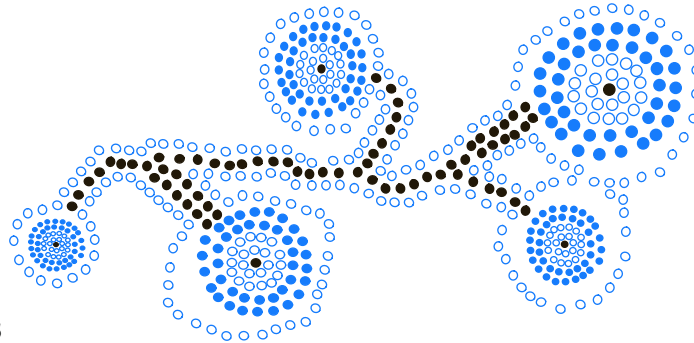
- P-7.58 Develop an experiment, using techniques similar to those in Section 4.1, to test the efficiency of n successive calls to the `add` method of an `ArrayList`, for various n , under each of the following three scenarios:
- Each add takes place at index 0.
 - Each add takes place at index `size()/2`.
 - Each add takes place at index `size()`.

Analyze your empirical results.

- P-7.59 Reimplement the `LinkedPositionalList` class so that an invalid position is reported in a scenario such as the one described in Exercise R-7.14.
- P-7.60 Implement a `CardHand` class that supports a person arranging a group of cards in his or her hand. The simulator should represent the sequence of cards using a single positional list ADT so that cards of the same suit are kept together. Implement this strategy by means of four “fingers” into the hand, one for each of the suits of hearts, clubs, spades, and diamonds, so that adding a new card to the person’s hand or playing a correct card from the hand can be done in constant time. The class should support the following methods:
- `addCard(r, s)`: Add a new card with rank *r* and suit *s* to the hand.
 - `play(s)`: Remove and return a card of suit *s* from the player’s hand; if there is no card of suit *s*, then remove and return an arbitrary card from the hand.
 - `iterator()`: Return an iterator for all cards currently in the hand.
 - `suitIterator(s)`: Return an iterator for all cards of suit *s* that are currently in the hand.
- P-7.61 Write a simple text editor, which stores and displays a string of characters using the positional list ADT, together with a cursor object that highlights a position in the string. The editor must support the following operations:
- `left`: Move cursor left one character (do nothing if at beginning).
 - `right`: Move cursor right one character (do nothing if at end).
 - `insert c`: Insert the character *c* just after the cursor.
 - `delete`: Delete the character just after the cursor (if not at end).

Chapter Notes

The treatment of data structures as collections (and other principles of object-oriented design) can be found in object-oriented design books by Booch [16], Budd [19], and Liskov and Guttag [67]. Lists and iterators are pervasive concepts in the Java Collections Framework. Our positional list ADT is derived from the “position” abstraction introduced by Aho, Hopcroft, and Ullman [6], and the list ADT of Wood [96]. Implementations of lists via arrays and linked lists are discussed by Knuth [60].



Contents

9.1	The Priority Queue Abstract Data Type	360
9.1.1	Priorities	360
9.1.2	The Priority Queue ADT	361
9.2	Implementing a Priority Queue	362
9.2.1	The Entry Composite	362
9.2.2	Comparing Keys with Total Orders	363
9.2.3	The AbstractPriorityQueue Base Class	364
9.2.4	Implementing a Priority Queue with an Unsorted List . . .	366
9.2.5	Implementing a Priority Queue with a Sorted List	368
9.3	Heaps	370
9.3.1	The Heap Data Structure	370
9.3.2	Implementing a Priority Queue with a Heap	372
9.3.3	Analysis of a Heap-Based Priority Queue	379
9.3.4	Bottom-Up Heap Construction ★	380
9.3.5	Using the java.util.PriorityQueue Class	384
9.4	Sorting with a Priority Queue	385
9.4.1	Selection-Sort and Insertion-Sort	386
9.4.2	Heap-Sort	388
9.5	Adaptable Priority Queues	390
9.5.1	Location-Aware Entries	391
9.5.2	Implementing an Adaptable Priority Queue	392
9.6	Exercises	395

9.1 The Priority Queue Abstract Data Type

9.1.1 Priorities

In Chapter 6, we introduced the queue ADT as a collection of objects that are added and removed according to the *first-in, first-out (FIFO)* principle. A company's customer call center embodies such a model in which waiting customers are told "calls will be answered in the order that they were received." In that setting, a new call is added to the back of the queue, and each time a customer service representative becomes available, he or she is connected with the call that is removed from the front of the call queue.

In practice, there are many applications in which a queue-like structure is used to manage objects that must be processed in some way, but for which the first-in, first-out policy does not suffice. Consider, for example, an air-traffic control center that has to decide which flight to clear for landing from among many approaching the airport. This choice may be influenced by factors such as each plane's distance from the runway, time spent waiting in a holding pattern, or amount of remaining fuel. It is unlikely that the landing decisions are based purely on a FIFO policy.

There are other situations in which a "first come, first serve" policy might seem reasonable, yet for which other priorities come into play. To use another airline analogy, suppose a certain flight is fully booked an hour prior to departure. Because of the possibility of cancellations, the airline maintains a queue of standby passengers hoping to get a seat. Although the priority of a standby passenger is influenced by the check-in time of that passenger, other considerations include the fare paid and frequent-flyer status. So it may be that an available seat is given to a passenger who has arrived *later* than another, if such a passenger is assigned a better priority by the airline agent.

In this chapter, we introduce a new abstract data type known as a *priority queue*. This is a collection of prioritized elements that allows arbitrary element insertion, and allows the removal of the element that has first priority. When an element is added to a priority queue, the user designates its priority by providing an associated *key*. The element with the *minimal* key will be the next to be removed from the queue (thus, an element with key 1 will be given priority over an element with key 2). Although it is quite common for priorities to be expressed numerically, any Java object may be used as a key, as long as there exists means to compare any two instances *a* and *b*, in a way that defines a natural order of the keys. With such generality, applications may develop their own notion of priority for each element. For example, different financial analysts may assign different ratings (i.e., priorities) to a particular asset, such as a share of stock.

9.1.2 The Priority Queue ADT

We model an element and its priority as a key-value composite known as an *entry*. (However, we defer until Section 9.2.1 the technical definition of the Entry type.)

We define the priority queue ADT to support the following methods:

`insert(k, v)`: Creates an entry with key *k* and value *v* in the priority queue.

`min()`: Returns (but does not remove) a priority queue entry (*k*,*v*) having minimal key; returns null if the priority queue is empty.

`removeMin()`: Removes and returns an entry (*k*,*v*) having minimal key from the priority queue; returns null if the priority queue is empty.

`size()`: Returns the number of entries in the priority queue.

`isEmpty()`: Returns a boolean indicating whether the priority queue is empty.

A priority queue may have multiple entries with equivalent keys, in which case methods `min` and `removeMin` may report an arbitrary choice among those entry having minimal key. Values may be any type of object.

In our initial model for a priority queue, we assume that an element's key remains fixed once it has been added to a priority queue. In Section 9.5, we consider an extension that allows a user to update an element's key within the priority queue.

Example 9.1: *The following table shows a series of operations and their effects on an initially empty priority queue. The “Priority Queue Contents” column is somewhat deceiving since it shows the entries sorted by key. Such an internal representation is not required of a priority queue.*

Method	Return Value	Priority Queue Contents
<code>insert(5,A)</code>		{ (5,A) }
<code>insert(9,C)</code>		{ (5,A), (9,C) }
<code>insert(3,B)</code>		{ (3,B), (5,A), (9,C) }
<code>min()</code>	(3,B)	{ (3,B), (5,A), (9,C) }
<code>removeMin()</code>	(3,B)	{ (5,A), (9,C) }
<code>insert(7,D)</code>		{ (5,A), (7,D), (9,C) }
<code>removeMin()</code>	(5,A)	{ (7,D), (9,C) }
<code>removeMin()</code>	(7,D)	{ (9,C) }
<code>removeMin()</code>	(9,C)	{ }
<code>removeMin()</code>	null	{ }
<code>isEmpty()</code>	true	{ }

9.2 Implementing a Priority Queue

In this section, we discuss several technical issues involving the implementation of the priority queue ADT in Java, and we define an abstract base class that provides functionality that is shared by all priority queue implementations in this chapter. We then provide two concrete priority queue implementations using a positional list *L* (see Section 7.3) for storage. They differ in whether or not entries are maintained in sorted order according to their keys.

9.2.1 The Entry Composite

One challenge in implementing a priority queue is that we must keep track of both an element and its key, even as entries are relocated within a data structure. This is reminiscent of a case study from Section 7.7 in which we maintain a list of elements with access frequencies. In that setting, we introduced the *composition design pattern*, defining an *Item* class that paired each element with its associated count in our primary data structure. For priority queues, we use composition to pair a key *k* and a value *v* as a single object. To formalize this, we define the public interface, *Entry*, shown in Code Fragment 9.1.

```
1  /** Interface for a key-value pair. */
2  public interface Entry<K,V> {
3      K getKey();                // returns the key stored in this entry
4      V getValue();              // returns the value stored in this entry
5  }
```

Code Fragment 9.1: Java interface for an entry storing a key-value pair.

We then use the *Entry* type in the formal interface for the priority queue, shown in Code Fragment 9.2. This allows us to return both a key and value as a single object from methods such as *min* and *removeMin*. We also define the *insert* method to return an entry; in a more advanced *adaptable priority queue* (see Section 9.5), that entry can be subsequently updated or removed.

```
1  /** Interface for the priority queue ADT. */
2  public interface PriorityQueue<K,V> {
3      int size();
4      boolean isEmpty();
5      Entry<K,V> insert(K key, V value) throws IllegalArgumentException;
6      Entry<K,V> min();
7      Entry<K,V> removeMin();
8  }
```

Code Fragment 9.2: Java interface for the priority queue ADT.

9.2.2 Comparing Keys with Total Orders

In defining the priority queue ADT, we can allow any type of object to serve as a key, but we must be able to compare keys to each other in a meaningful way. More so, the results of the comparisons must not be contradictory. For a comparison rule, which we denote by $\leq$, to be self-consistent, it must define a **total order** relation, which is to say that it satisfies the following properties for any keys k_1 , k_2 , and k_3 :

- **Comparability property:** $k_1 \leq k_2$ or $k_2 \leq k_1$.
- **Antisymmetric property:** if $k_1 \leq k_2$ and $k_2 \leq k_1$, then $k_1 = k_2$.
- **Transitive property:** if $k_1 \leq k_2$ and $k_2 \leq k_3$, then $k_1 \leq k_3$.

The comparability property states that comparison rule is defined for every pair of keys. Note that this property implies the following one:

- **Reflexive property:** $k \leq k$.

A comparison rule, $\leq$, that defines a total order relation will never lead to a contradiction. Such a rule defines a linear ordering among a set of keys; hence, if a (finite) set of elements has a total order defined for it, then the notion of a **minimal** key, $k_{\min}$, is well defined, as a key in which $k_{\min} \leq k$, for any other key k in our set.

The Comparable Interface

Java provides two means for defining comparisons between object types. The first of these is that a class may define what is known as the **natural ordering** of its instances by formally implementing the `java.lang.Comparable` interface, which includes a single method, `compareTo`. The syntax `a.compareTo(b)` must return an integer i with the following meaning:

- $i < 0$ designates that $a < b$.
- $i = 0$ designates that $a = b$.
- $i > 0$ designates that $a > b$.

For example, the `compareTo` method of the `String` class defines the natural ordering of strings to be **lexicographic**, which is a case-sensitive extension of the alphabetic ordering to Unicode.

The Comparator Interface

In some applications, we may want to compare objects according to some notion other than their natural ordering. For example, we might be interested in which of two strings is the shortest, or in defining our own complex rules for judging which of two stocks is more promising. To support generality, Java defines the `java.util.Comparator` interface. A **comparator** is an object that is external to the class of the keys it compares. It provides a method with the signature `compare(a, b)` that returns an integer with similar meaning to the `compareTo` method described above.

As a concrete example, Code Fragment 9.3 defines a comparator that evaluates strings based on their length (rather than their natural lexicographic order).

```

1 public class StringLengthComparator implements Comparator<String> {
2     /** Compares two strings according to their lengths. */
3     public int compare(String a, String b) {
4         if (a.length() < b.length()) return -1;
5         else if (a.length() == b.length()) return 0;
6         else return 1;
7     }
8 }

```

Code Fragment 9.3: A comparator that evaluates strings based on their lengths.

Comparators and the Priority Queue ADT

For a general and reusable form of a priority queue, we allow a user to choose any key type and to send an appropriate comparator instance as a parameter to the priority queue constructor. The priority queue will use that comparator anytime it needs to compare two keys to each other.

For convenience, we also allow a default priority queue to instead rely on the natural ordering for the given keys (assuming those keys come from a comparable class). In that case, we build our own instance of a `DefaultComparator` class, shown in Code Fragment 9.4.

```

1 public class DefaultComparator<E> implements Comparator<E> {
2     public int compare(E a, E b) throws ClassCastException {
3         return ((Comparable<E>) a).compareTo(b);
4     }
5 }

```

Code Fragment 9.4: A `DefaultComparator` class that implements a comparator based upon the natural ordering of its element type.

9.2.3 The `AbstractPriorityQueue` Base Class

To manage technical issues common to all our priority queue implementations, we define an abstract base class named `AbstractPriorityQueue` in Code Fragment 9.5. (See Section 2.3.3 for a discussion of abstract base classes.) This includes a nested `PQEntry` class that implements the public `Entry` interface.

Our abstract class also declares and initializes an instance variable, `comp`, that stores the comparator being used for the priority queue. We then provide a protected method, `compare`, that invokes the comparator on the keys of two given entries.

```

1  /** An abstract base class to assist implementations of the PriorityQueue interface.*/
2  public abstract class AbstractPriorityQueue<K,V>
3                                     implements PriorityQueue<K,V> {
4      //----- nested PQEntry class -----
5      protected static class PQEntry<K,V> implements Entry<K,V> {
6          private K k; // key
7          private V v; // value
8          public PQEntry(K key, V value) {
9              k = key;
10             v = value;
11         }
12         // methods of the Entry interface
13         public K getKey() { return k; }
14         public V getValue() { return v; }
15         // utilities not exposed as part of the Entry interface
16         protected void setKey(K key) { k = key; }
17         protected void setValue(V value) { v = value; }
18     } //----- end of nested PQEntry class -----
19
20     // instance variable for an AbstractPriorityQueue
21     /** The comparator defining the ordering of keys in the priority queue. */
22     private Comparator<K> comp;
23     /** Creates an empty priority queue using the given comparator to order keys. */
24     protected AbstractPriorityQueue(Comparator<K> c) { comp = c; }
25     /** Creates an empty priority queue based on the natural ordering of its keys. */
26     protected AbstractPriorityQueue() { this(new DefaultComparator<K>()); }
27     /** Method for comparing two entries according to key */
28     protected int compare(Entry<K,V> a, Entry<K,V> b) {
29         return comp.compare(a.getKey(), b.getKey());
30     }
31     /** Determines whether a key is valid. */
32     protected boolean checkKey(K key) throws IllegalArgumentException {
33         try {
34             return (comp.compare(key,key) == 0); // see if key can be compared to itself
35         } catch (ClassCastException e) {
36             throw new IllegalArgumentException("Incompatible key");
37         }
38     }
39     /** Tests whether the priority queue is empty. */
40     public boolean isEmpty() { return size() == 0; }
41 }

```

Code Fragment 9.5: The `AbstractPriorityQueue` class. This provides a nested `PQEntry` class that composes a key and a value into a single object, and support for managing a comparator. For convenience, we also provide an implementation of `isEmpty` based on a presumed `size` method.

9.2.4 Implementing a Priority Queue with an Unsorted List

In our first concrete implementation of a priority queue, we store entries within an *unsorted* linked list. Code Fragment 9.6 presents our `UnsortedPriorityQueue` class as a subclass of the `AbstractPriorityQueue` class (from Code Fragment 9.5). For internal storage, key-value pairs are represented as composites, using instances of the inherited `PQEntry` class. These entries are stored within a `PositionalList` that is an instance variable. We assume that the positional list is implemented with a doubly linked list, as in Section 7.3, so that all operations of that ADT execute in $O(1)$ time.

We begin with an empty list when a new priority queue is constructed. At all times, the size of the list equals the number of key-value pairs currently stored in the priority queue. For this reason, our priority queue size method simply returns the length of the internal list. By the design of our `AbstractPriorityQueue` class, we inherit a concrete implementation of the `isEmpty` method that relies on a call to our size method.

Each time a key-value pair is added to the priority queue, via the `insert` method, we create a new `PQEntry` composite for the given key and value, and add that entry to the end of the list. Such an implementation takes $O(1)$ time.

The remaining challenge is that when `min` or `removeMin` is called, we must locate the entry with minimal key. Because the entries are not sorted, we must inspect all entries to find one with a minimal key. For convenience, we define a private `findMin` utility that returns the *position* of an entry with minimal key. Knowledge of the position allows the `removeMin` method to invoke the `remove` method on the positional list. The `min` method simply uses the position to retrieve the entry when preparing a key-value tuple to return. Due to the loop for finding the minimal key, both `min` and `removeMin` methods run in $O(n)$ time, where n is the number of entries in the priority queue.

A summary of the running times for the `UnsortedPriorityQueue` class is given in Table 9.1.

Method	Running Time
size	$O(1)$
isEmpty	$O(1)$
insert	$O(1)$
min	$O(n)$
removeMin	$O(n)$

Table 9.1: Worst-case running times of the methods of a priority queue of size n , realized by means of an unsorted, doubly linked list. The space requirement is $O(n)$.

```

1  /** An implementation of a priority queue with an unsorted list. */
2  public class UnsortedPriorityQueue<K,V> extends AbstractPriorityQueue<K,V> {
3      /** primary collection of priority queue entries */
4      private PositionalList<Entry<K,V>> list = new LinkedPositionalList<>();
5
6      /** Creates an empty priority queue based on the natural ordering of its keys. */
7      public UnsortedPriorityQueue() { super(); }
8      /** Creates an empty priority queue using the given comparator to order keys. */
9      public UnsortedPriorityQueue(Comparator<K> comp) { super(comp); }
10
11     /** Returns the Position of an entry having minimal key. */
12     private Position<Entry<K,V>> findMin() { // only called when nonempty
13         Position<Entry<K,V>> small = list.first();
14         for (Position<Entry<K,V>> walk : list.positions())
15             if (compare(walk.getElement(), small.getElement()) < 0)
16                 small = walk; // found an even smaller key
17         return small;
18     }
19
20     /** Inserts a key-value pair and returns the entry created. */
21     public Entry<K,V> insert(K key, V value) throws IllegalArgumentException {
22         checkKey(key); // auxiliary key-checking method (could throw exception)
23         Entry<K,V> newest = new PQEntry<>(key, value);
24         list.addLast(newest);
25         return newest;
26     }
27
28     /** Returns (but does not remove) an entry with minimal key. */
29     public Entry<K,V> min() {
30         if (list.isEmpty()) return null;
31         return findMin().getElement();
32     }
33
34     /** Removes and returns an entry with minimal key. */
35     public Entry<K,V> removeMin() {
36         if (list.isEmpty()) return null;
37         return list.remove(findMin());
38     }
39
40     /** Returns the number of items in the priority queue. */
41     public int size() { return list.size(); }
42 }

```

Code Fragment 9.6: An implementation of a priority queue using an unsorted list. The parent class `AbstractPriorityQueue` is given in Code Fragment 9.5, and the `LinkedPositionalList` class is from Section 7.3.

9.2.5 Implementing a Priority Queue with a Sorted List

Our next implementation of a priority queue also uses a positional list, yet maintains entries sorted by nondecreasing keys. This ensures that the first element of the list is an entry with the smallest key.

Our SortedPriorityQueue class is given in Code Fragment 9.7. The implementation of min and removeMin are rather straightforward given knowledge that the first element of a list has a minimal key. We rely on the first method of the positional list to find the position of the first entry, and the remove method to remove the entry from the list. Assuming that the list is implemented with a doubly linked list, operations min and removeMin take $O(1)$ time.

This benefit comes at a cost, however, for method insert now requires that we scan the list to find the appropriate position to insert the new entry. Our implementation starts at the end of the list, walking backward until the new key is smaller than that of an existing entry; in the worst case, it progresses until reaching the front of the list. Therefore, the insert method takes $O(n)$ worst-case time, where n is the number of entries in the priority queue at the time the method is executed. In summary, when using a sorted list to implement a priority queue, insertion runs in linear time, whereas finding and removing the minimum can be done in constant time.

Comparing the Two List-Based Implementations

Table 9.2 compares the running times of the methods of a priority queue realized by means of a sorted and unsorted list, respectively. We see an interesting trade-off when we use a list to implement the priority queue ADT. An unsorted list supports fast insertions but slow queries and deletions, whereas a sorted list allows fast queries and deletions, but slow insertions.

Method	Unsorted List	Sorted List
size	$O(1)$	$O(1)$
isEmpty	$O(1)$	$O(1)$
insert	$O(1)$	$O(n)$
min	$O(n)$	$O(1)$
removeMin	$O(n)$	$O(1)$

Table 9.2: Worst-case running times of the methods of a priority queue of size n , realized by means of an unsorted or sorted list, respectively. We assume that the list is implemented by a doubly linked list. The space requirement is $O(n)$.

```

1  /** An implementation of a priority queue with a sorted list. */
2  public class SortedPriorityQueue<K,V> extends AbstractPriorityQueue<K,V> {
3      /** primary collection of priority queue entries */
4      private PositionalList<Entry<K,V>> list = new LinkedPositionalList<>();
5
6      /** Creates an empty priority queue based on the natural ordering of its keys. */
7      public SortedPriorityQueue() { super(); }
8      /** Creates an empty priority queue using the given comparator to order keys. */
9      public SortedPriorityQueue(Comparator<K> comp) { super(comp); }
10
11     /** Inserts a key-value pair and returns the entry created. */
12     public Entry<K,V> insert(K key, V value) throws IllegalArgumentException {
13         checkKey(key);    // auxiliary key-checking method (could throw exception)
14         Entry<K,V> newest = new PQEntry<>(key, value);
15         Position<Entry<K,V>> walk = list.last();
16         // walk backward, looking for smaller key
17         while (walk != null && compare(newest, walk.getElement()) < 0)
18             walk = list.before(walk);
19         if (walk == null)
20             list.addFirst(newest);                // new key is smallest
21         else
22             list.addAfter(walk, newest);           // newest goes after walk
23         return newest;
24     }
25
26     /** Returns (but does not remove) an entry with minimal key. */
27     public Entry<K,V> min() {
28         if (list.isEmpty()) return null;
29         return list.first().getElement();
30     }
31
32     /** Removes and returns an entry with minimal key. */
33     public Entry<K,V> removeMin() {
34         if (list.isEmpty()) return null;
35         return list.remove(list.first());
36     }
37
38     /** Returns the number of items in the priority queue. */
39     public int size() { return list.size(); }
40 }

```

Code Fragment 9.7: An implementation of a priority queue using a sorted list. The parent class `AbstractPriorityQueue` is given in Code Fragment 9.5, and the `LinkedPositionalList` class is from Section 7.3.

9.3 Heaps

The two strategies for implementing a priority queue ADT in the previous section demonstrate an interesting trade-off. When using an *unsorted* list to store entries, we can perform insertions in $O(1)$ time, but finding or removing an element with minimal key requires an $O(n)$ -time loop through the entire collection. In contrast, if using a *sorted* list, we can trivially find or remove the minimal element in $O(1)$ time, but adding a new element to the queue may require $O(n)$ time to restore the sorted order.

In this section, we provide a more efficient realization of a priority queue using a data structure called a **binary heap**. This data structure allows us to perform both insertions and removals in logarithmic time, which is a significant improvement over the list-based implementations discussed in Section 9.2. The fundamental way the heap achieves this improvement is to use the structure of a binary tree to find a compromise between elements being entirely unsorted and perfectly sorted.

9.3.1 The Heap Data Structure

A heap (see Figure 9.1) is a binary tree T that stores entries at its positions, and that satisfies two additional properties: a relational property defined in terms of the way keys are stored in T and a structural property defined in terms of the shape of T itself. The relational property is the following:

Heap-Order Property: In a heap T , for every position p other than the root, the key stored at p is greater than or equal to the key stored at p 's parent.

As a consequence of the heap-order property, the keys encountered on a path from the root to a leaf of T are in nondecreasing order. Also, a minimal key is always stored at the root of T . This makes it easy to locate such an entry when `min` or `removeMin` is called, as it is informally said to be “at the top of the heap” (hence, the name “heap” for the data structure). By the way, the heap data structure defined here has nothing to do with the memory heap (Section 15.1.2) used in the runtime environment supporting a programming language like Java.

For the sake of efficiency, as will become clear later, we want the heap T to have as small a height as possible. We enforce this requirement by insisting that the heap T satisfy an additional structural property; it must be what we term **complete**.

Complete Binary Tree Property: A heap T with height h is a **complete** binary tree if levels $0, 1, 2, \dots, h - 1$ of T have the maximal number of nodes possible (namely, level i has 2^i nodes, for $0 \leq i \leq h - 1$) and the remaining nodes at level h reside in the leftmost possible positions at that level.

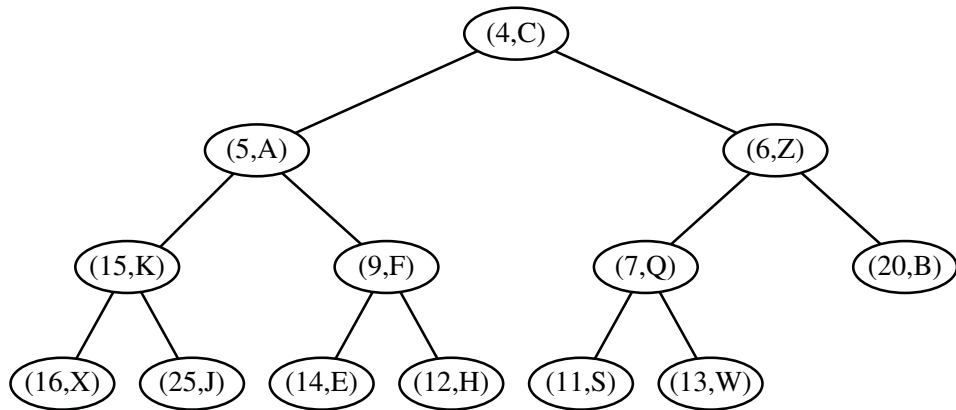


Figure 9.1: Example of a heap storing 13 entries with integer keys. The last position is the one storing entry (13, *W*).

The tree in Figure 9.1 is complete because levels 0, 1, and 2 are full, and the six nodes in level 3 are in the six leftmost possible positions at that level. In formalizing what we mean by the leftmost possible positions, we refer to the discussion of **level numbering** from Section 8.3.2, in the context of an array-based representation of a binary tree. (In fact, in Section 9.3.2 we will discuss the use of an array to represent a heap.) A complete binary tree with n elements is one that has positions with level numbering 0 through $n - 1$. For example, in an array-based representation of the above tree, its 13 entries would be stored consecutively from $A[0]$ to $A[12]$.

The Height of a Heap

Let h denote the height of T . Insisting that T be complete also has an important consequence, as shown in Proposition 9.2.

Proposition 9.2: A heap T storing n entries has height $h = \lfloor \log n \rfloor$.

Justification: From the fact that T is complete, we know that the number of nodes in levels 0 through $h - 1$ of T is precisely $1 + 2 + 4 + \cdots + 2^{h-1} = 2^h - 1$, and that the number of nodes in level h is at least 1 and at most 2^h . Therefore

$$n \geq 2^h - 1 + 1 = 2^h \quad \text{and} \quad n \leq 2^h - 1 + 2^h = 2^{h+1} - 1.$$

By taking the logarithm of both sides of inequality $n \geq 2^h$, we see that height $h \leq \log n$. By rearranging terms and taking the logarithm of both sides of inequality $n \leq 2^{h+1} - 1$, we see that $h \geq \log(n + 1) - 1$. Since h is an integer, these two inequalities imply that $h = \lfloor \log n \rfloor$. ■

9.3.2 Implementing a Priority Queue with a Heap

Proposition 9.2 has an important consequence, for it implies that if we can perform update operations on a heap in time proportional to its height, then those operations will run in logarithmic time. Let us therefore turn to the problem of how to efficiently perform various priority queue methods using a heap.

We will use the composition pattern from Section 9.2.1 to store key-value pairs as entries in the heap. The size and isEmpty methods can be implemented based on examination of the tree, and the min operation is equally trivial because the heap property assures that the element at the root of the tree has a minimal key. The interesting algorithms are those for implementing the insert and removeMin methods.

Adding an Entry to the Heap

Let us consider how to perform $\text{insert}(k, v)$ on a priority queue implemented with a heap T . We store the pair (k, v) as an entry at a new node of the tree. To maintain the **complete binary tree property**, that new node should be placed at a position p just beyond the rightmost node at the bottom level of the tree, or as the leftmost position of a new level, if the bottom level is already full (or if the heap is empty).

Up-Heap Bubbling After an Insertion

After this action, the tree T is complete, but it may violate the **heap-order property**. Hence, unless position p is the root of T (that is, the priority queue was empty before the insertion), we compare the key at position p to that of p 's parent, which we denote as q . If key $k_p \geq k_q$, the heap-order property is satisfied and the algorithm terminates. If instead $k_p < k_q$, then we need to restore the heap-order property, which can be locally achieved by swapping the entries stored at positions p and q . (See Figure 9.2c and d.) This swap causes the new entry to move up one level. Again, the heap-order property may be violated, so we repeat the process, going up in T until no violation of the heap-order property occurs. (See Figure 9.2e and h.)

The upward movement of the newly inserted entry by means of swaps is conventionally called **up-heap bubbling**. A swap either resolves the violation of the heap-order property or propagates it one level up in the heap. In the worst case, up-heap bubbling causes the new entry to move all the way up to the root of heap T . Thus, in the worst case, the number of swaps performed in the execution of method insert is equal to the height of T . By Proposition 9.2, that bound is $\lfloor \log n \rfloor$.

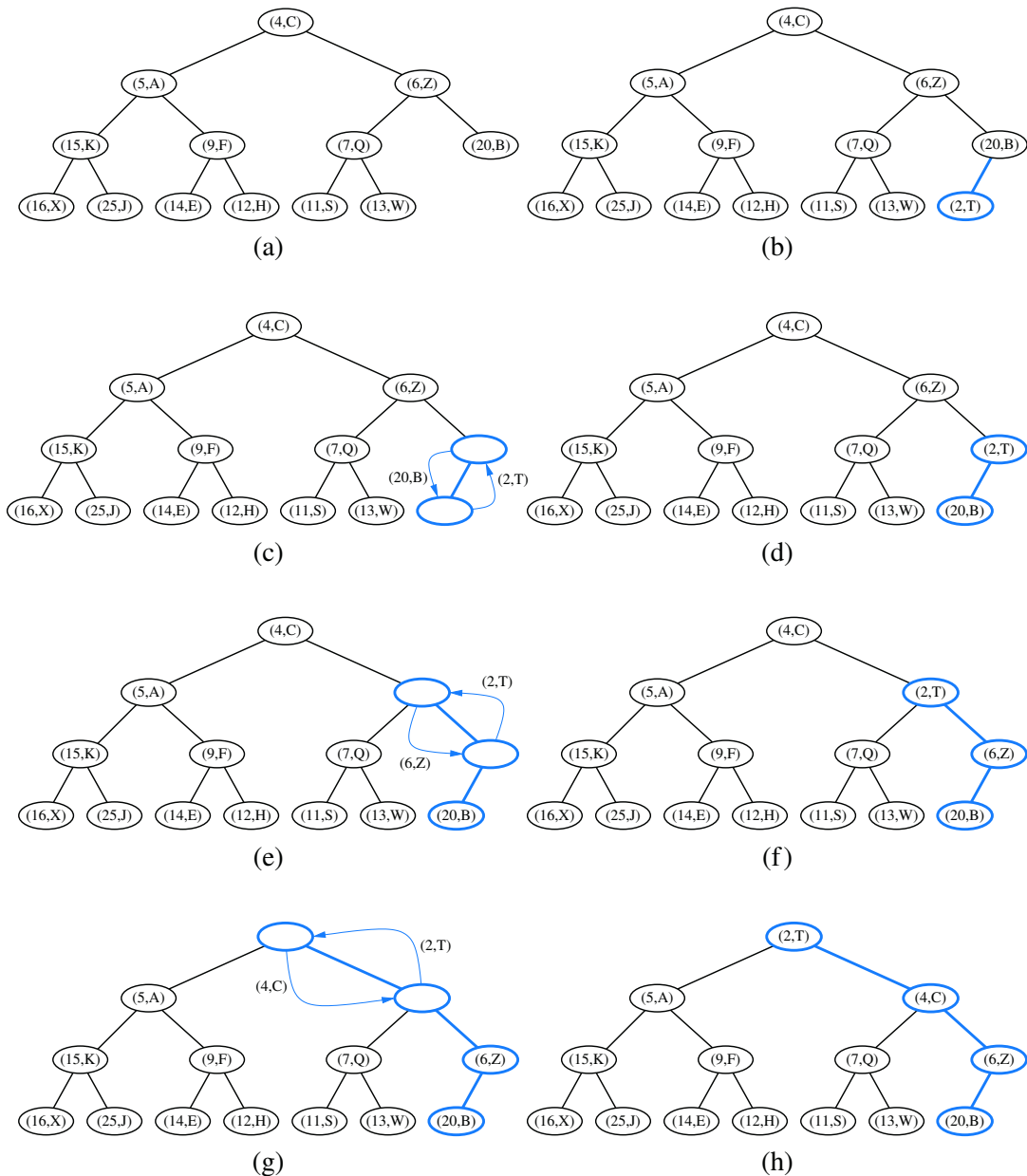


Figure 9.2: Insertion of a new entry with key 2 into the heap of Figure 9.1: (a) initial heap; (b) after adding a new node; (c and d) swap to locally restore the partial order property; (e and f) another swap; (g and h) final swap.

Removing the Entry with Minimal Key

Let us now turn to method `removeMin` of the priority queue ADT. We know that an entry with the smallest key is stored at the root r of T (even if there is more than one entry with smallest key). However, in general we cannot simply delete node r , because this would leave two disconnected subtrees.

Instead, we ensure that the shape of the heap respects the *complete binary tree property* by deleting the leaf at the *last* position p of T , defined as the rightmost position at the bottommost level of the tree. To preserve the entry from the last position p , we copy it to the root r (in place of the entry with minimal key that is being removed by the operation). Figure 9.3a and b illustrates an example of these steps, with minimal entry $(4, C)$ being removed from the root and replaced by entry $(13, W)$ from the last position. The node at the last position is removed from the tree.

Down-Heap Bubbling After a Removal

We are not yet done, however, for even though T is now complete, it likely violates the heap-order property. If T has only one node (the root), then the heap-order property is trivially satisfied and the algorithm terminates. Otherwise, we distinguish two cases, where p initially denotes the root of T :

- If p has no right child, let c be the left child of p .
- Otherwise (p has both children), let c be a child of p with minimal key.

If key $k_p \leq k_c$, the heap-order property is satisfied and the algorithm terminates. If instead $k_p > k_c$, then we need to restore the heap-order property. This can be locally achieved by swapping the entries stored at p and c . (See Figure 9.3c and d.) It is worth noting that when p has two children, we intentionally consider the *smaller* key of the two children. Not only is the key of c smaller than that of p , it is at least as small as the key at c 's sibling. This ensures that the heap-order property is locally restored when that smaller key is promoted above the key that had been at p and that at c 's sibling.

Having restored the heap-order property for node p relative to its children, there may be a violation of this property at c ; hence, we may have to continue swapping down T until no violation of the heap-order property occurs. (See Figure 9.3e–h.) This downward swapping process is called *down-heap bubbling*. A swap either resolves the violation of the heap-order property or propagates it one level down in the heap. In the worst case, an entry moves all the way down to the bottom level. (See Figure 9.3.) Thus, the number of swaps performed in the execution of method `removeMin` is, in the worst case, equal to the height of heap T , that is, it is $\lfloor \log n \rfloor$ by Proposition 9.2.

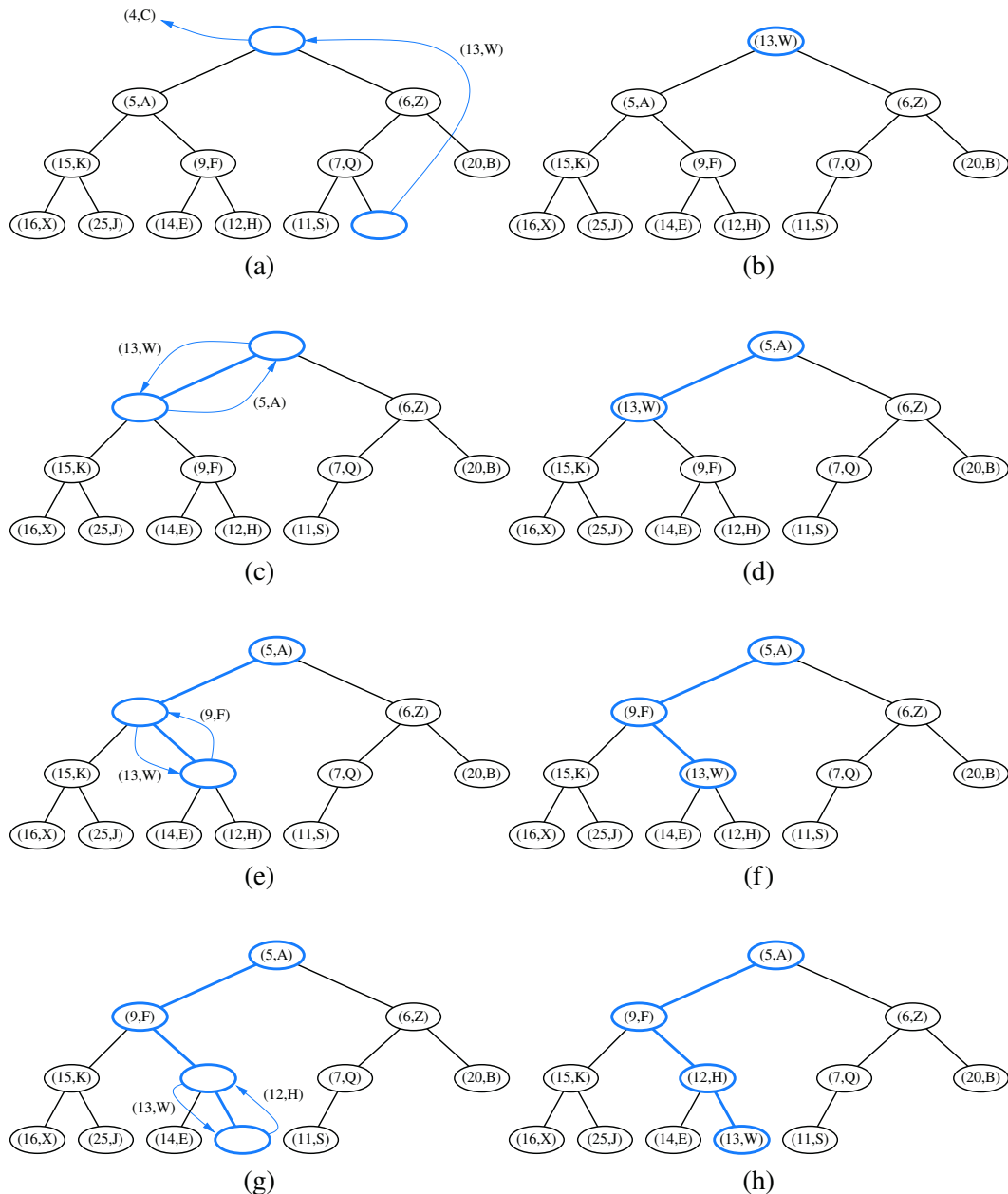


Figure 9.3: Removal of the entry with the smallest key from a heap: (a and b) deletion of the last node, whose entry gets stored into the root; (c and d) swap to locally restore the heap-order property; (e and f) another swap; (g and h) final swap.

Array-Based Representation of a Complete Binary Tree

The array-based representation of a binary tree (Section 8.3.2) is especially suitable for a complete binary tree. We recall that in this implementation, the elements of the tree are stored in an array-based list A such that the element at position p is stored in A with index equal to the level number $f(p)$ of p , defined as follows:

- If p is the root, then $f(p) = 0$.
- If p is the left child of position q , then $f(p) = 2f(q) + 1$.
- If p is the right child of position q , then $f(p) = 2f(q) + 2$.

For a tree with of size n , the elements have contiguous indices in the range $[0, n - 1]$ and the last position of is always at index $n - 1$. (See Figure 9.4.)

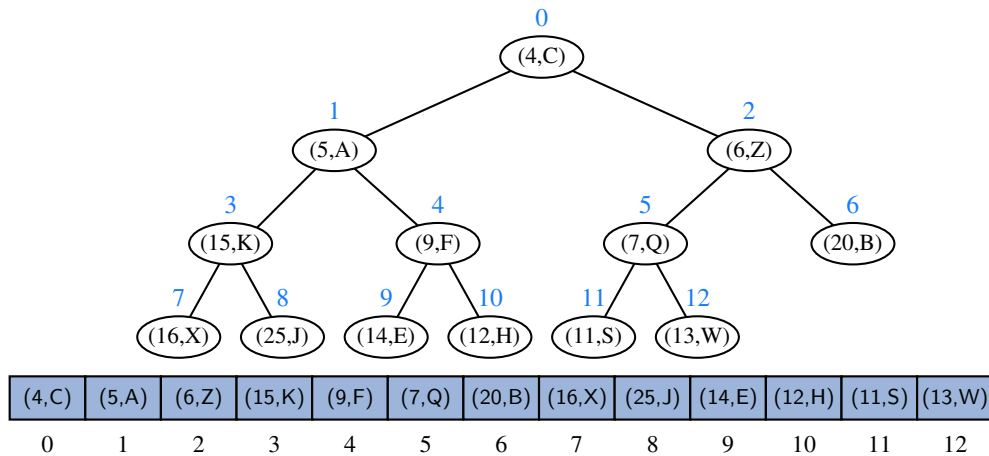


Figure 9.4: Array-based representation of a heap.

The array-based heap representation avoids some complexities of a linked tree structure. Specifically, methods `insert` and `removeMin` depend on locating the last position of a heap. With the array-based representation of a heap of size n , the last position is simply at index $n - 1$. Locating the last position in a heap implemented with a linked tree structure requires more effort. (See Exercise C-9.33.)

If the size of a priority queue is not known in advance, use of an array-based representation does introduce the need to dynamically resize the array on occasion, as is done with a Java `ArrayList`. The space usage of such an array-based representation of a complete binary tree with n nodes is $O(n)$, and the time bounds of methods for adding or removing elements become *amortized*. (See Section 7.2.2.)

Java Heap Implementation

In Code Fragments 9.8 and 9.9, we provide a Java implementation of a heap-based priority queue. Although we think of our heap as a binary tree, we do not formally

use the binary tree ADT. We prefer to use the more efficient array-based representation of a tree, maintaining a Java `ArrayList` of entry composites. To allow us to formalize our algorithms using tree-like terminology of *parent*, *left*, and *right*, the class includes protected utility methods that compute the level numbering of a parent or child of another position (lines 10–14 of Code Fragment 9.8). However, the “positions” in this representation are simply integer indices into the array-list.

Our class also has protected utilities `swap`, `upheap`, and `downheap` for the low-level movement of entries within the array-list. A new entry is added the end of the array-list, and then repositioned as needed with `upheap`. To remove the entry with minimal key (which resides at index 0), we move the last entry of the array-list from index $n - 1$ to index 0, and then invoke `downheap` to reposition it.

```

1  /** An implementation of a priority queue using an array-based heap. */
2  public class HeapPriorityQueue<K,V> extends AbstractPriorityQueue<K,V> {
3      /** primary collection of priority queue entries */
4      protected ArrayList<Entry<K,V>> heap = new ArrayList<>();
5      /** Creates an empty priority queue based on the natural ordering of its keys. */
6      public HeapPriorityQueue() { super(); }
7      /** Creates an empty priority queue using the given comparator to order keys. */
8      public HeapPriorityQueue(Comparator<K> comp) { super(comp); }
9      // protected utilities
10     protected int parent(int j) { return (j-1) / 2; }           // truncating division
11     protected int left(int j) { return 2*j + 1; }
12     protected int right(int j) { return 2*j + 2; }
13     protected boolean hasLeft(int j) { return left(j) < heap.size(); }
14     protected boolean hasRight(int j) { return right(j) < heap.size(); }
15     /** Exchanges the entries at indices i and j of the array list. */
16     protected void swap(int i, int j) {
17         Entry<K,V> temp = heap.get(i);
18         heap.set(i, heap.get(j));
19         heap.set(j, temp);
20     }
21     /** Moves the entry at index j higher, if necessary, to restore the heap property. */
22     protected void upheap(int j) {
23         while (j > 0) { // continue until reaching root (or break statement)
24             int p = parent(j);
25             if (compare(heap.get(j), heap.get(p)) >= 0) break; // heap property verified
26             swap(j, p);
27             j = p; // continue from the parent's location
28         }
29     }

```

Code Fragment 9.8: Priority queue that uses an array-based heap and extends `AbstractPriorityQueue` (Code Fragment 9.5). (Continues in Code Fragment 9.9.)

```

30  /** Moves the entry at index j lower, if necessary, to restore the heap property. */
31  protected void downheap(int j) {
32      while (hasLeft(j)) { // continue to bottom (or break statement)
33          int leftIndex = left(j);
34          int smallChildIndex = leftIndex; // although right may be smaller
35          if (hasRight(j)) {
36              int rightIndex = right(j);
37              if (compare(heap.get(leftIndex), heap.get(rightIndex)) > 0)
38                  smallChildIndex = rightIndex; // right child is smaller
39          }
40          if (compare(heap.get(smallChildIndex), heap.get(j)) >= 0)
41              break; // heap property has been restored
42          swap(j, smallChildIndex);
43          j = smallChildIndex; // continue at position of the child
44      }
45  }
46
47  // public methods
48  /** Returns the number of items in the priority queue. */
49  public int size() { return heap.size(); }
50  /** Returns (but does not remove) an entry with minimal key (if any). */
51  public Entry<K,V> min() {
52      if (heap.isEmpty()) return null;
53      return heap.get(0);
54  }
55  /** Inserts a key-value pair and returns the entry created. */
56  public Entry<K,V> insert(K key, V value) throws IllegalArgumentException {
57      checkKey(key); // auxiliary key-checking method (could throw exception)
58      Entry<K,V> newest = new PQEntry<>(key, value);
59      heap.add(newest); // add to the end of the list
60      upheap(heap.size() - 1); // upheap newly added entry
61      return newest;
62  }
63  /** Removes and returns an entry with minimal key (if any). */
64  public Entry<K,V> removeMin() {
65      if (heap.isEmpty()) return null;
66      Entry<K,V> answer = heap.get(0);
67      swap(0, heap.size() - 1); // put minimum item at the end
68      heap.remove(heap.size() - 1); // and remove it from the list;
69      downheap(0); // then fix new root
70      return answer;
71  }
72  }

```

Code Fragment 9.9: Priority queue implemented with an array-based heap (continued from Code Fragment 9.8).

9.3.3 Analysis of a Heap-Based Priority Queue

Table 9.3 shows the running time of the priority queue ADT methods for the heap implementation of a priority queue, assuming that two keys can be compared in $O(1)$ time and that the heap T is implemented with an array-based or linked-based tree representation.

In short, each of the priority queue ADT methods can be performed in $O(1)$ or in $O(\log n)$ time, where n is the number of entries at the time the method is executed. The analysis of the running time of the methods is based on the following:

- The heap T has n nodes, each storing a reference to a key-value entry.
- The height of heap T is $O(\log n)$, since T is complete (Proposition 9.2).
- The min operation runs in $O(1)$ because the root of the tree contains such an element.
- Locating the last position of a heap, as required for insert and removeMin, can be performed in $O(1)$ time for an array-based representation, or $O(\log n)$ time for a linked-tree representation. (See Exercise C-9.33.)
- In the worst case, up-heap and down-heap bubbling perform a number of swaps equal to the height of T .

Method	Running Time
size, isEmpty	$O(1)$
min	$O(1)$
insert	$O(\log n)^*$
removeMin	$O(\log n)^*$

*amortized, if using dynamic array

Table 9.3: Performance of a priority queue realized by means of a heap. We let n denote the number of entries in the priority queue at the time an operation is executed. The space requirement is $O(n)$. The running time of operations min and removeMin are amortized for an array-based representation, due to occasional re-sizing of a dynamic array; those bounds are worst case with a linked tree structure.

We conclude that the heap data structure is a very efficient realization of the priority queue ADT, independent of whether the heap is implemented with a linked structure or an array. The heap-based implementation achieves fast running times for both insertion and removal, unlike the implementations that were based on using an unsorted or sorted list.

9.3.4 Bottom-Up Heap Construction ★

If we start with an initially empty heap, n successive calls to the insert operation will run in $O(n \log n)$ time in the worst case. However, if all n key-value pairs to be stored in the heap are given in advance, such as during the first phase of the heap-sort algorithm (introduced in Section 9.4.2), there is an alternative **bottom-up** construction method that runs in $O(n)$ time.

In this section, we describe the bottom-up heap construction, and provide an implementation that can be used by the constructor of a heap-based priority queue.

For simplicity of exposition, we describe this bottom-up heap construction assuming the number of keys, n , is an integer such that $n = 2^{h+1} - 1$. That is, the heap is a complete binary tree with every level being full, so the heap has height $h = \log(n + 1) - 1$. Viewed nonrecursively, bottom-up heap construction consists of the following $h + 1 = \log(n + 1)$ steps:

1. In the first step (see Figure 9.5b), we construct $(n + 1)/2$ elementary heaps storing one entry each.
2. In the second step (see Figure 9.5c–d), we form $(n + 1)/4$ heaps, each storing three entries, by joining pairs of elementary heaps and adding a new entry. The new entry is placed at the root and may have to be swapped with the entry stored at a child to preserve the heap-order property.
3. In the third step (see Figure 9.5e–f), we form $(n + 1)/8$ heaps, each storing 7 entries, by joining pairs of 3-entry heaps (constructed in the previous step) and adding a new entry. The new entry is placed initially at the root, but may have to move down with a down-heap bubbling to preserve the heap-order property.
- ⋮
- i.* In the generic i^{th} step, $2 \leq i \leq h$, we form $(n + 1)/2^i$ heaps, each storing $2^i - 1$ entries, by joining pairs of heaps storing $(2^{i-1} - 1)$ entries (constructed in the previous step) and adding a new entry. The new entry is placed initially at the root, but may have to move down with a down-heap bubbling to preserve the heap-order property.
- ⋮
- $h + 1.$ In the last step (see Figure 9.5g–h), we form the final heap, storing all the n entries, by joining two heaps storing $(n - 1)/2$ entries (constructed in the previous step) and adding a new entry. The new entry is placed initially at the root, but may have to move down with a down-heap bubbling to preserve the heap-order property.

We illustrate bottom-up heap construction in Figure 9.5 for $h = 3$.

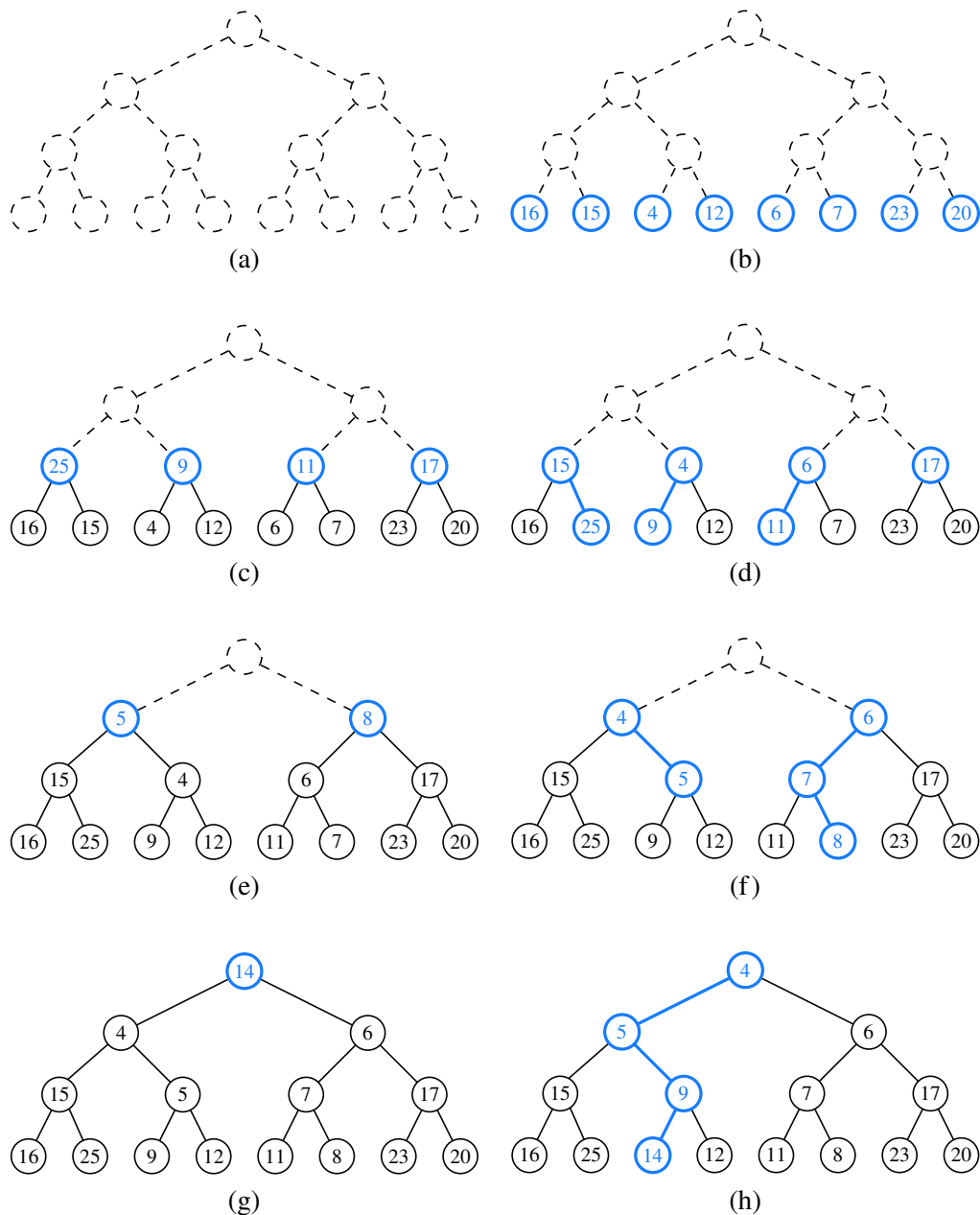


Figure 9.5: Bottom-up construction of a heap with 15 entries: (a and b) we begin by constructing 1-entry heaps on the bottom level; (c and d) we combine these heaps into 3-entry heaps; (e and f) we build 7-entry heaps; (g and h) we create the final heap. The paths of the down-heap bubblings are highlighted in (d, f, and h). For simplicity, we only show the key within each node instead of the entire entry.

Java Implementation of a Bottom-Up Heap Construction

Implementing a bottom-up heap construction is quite easy, given the existence of a “down-heap” utility method. The “merging” of two equally sized heaps that are subtrees of a common position p , as described in the opening of this section, can be accomplished simply by down-heap-ing p ’s entry. For example, that is what happened to the key 14 in going from Figure 9.5(f) to (g).

With our array-based representation of a heap, if we initially store all n entries in arbitrary order within the array, we can implement the bottom-up heap construction process with a single loop that makes a call to downheap from each position of the tree, as long as those calls are ordered starting with the *deepest* level and ending with the root of the tree. In fact, that loop can start with the deepest internal position, since there is no effect when down-heap is called at an external position.

In Code Fragment 9.10, we augment the original HeapPriorityQueue class from Section 9.3.2 to provide support for the bottom-up construction of an initial collection. We introduce a nonpublic utility method, heapify, that calls downheap on each nonleaf position, beginning with the deepest and concluding with a call at the root of the tree.

We introduce an additional constructor for the class that accepts an initial sequence of keys and values, parameterized as two coordinate arrays that are presumed to have the same length. We create new entries, pairing the first key with the first value, the second key with the second value, and so on. We then call the heapify utility to establish the heap ordering. For brevity, we omit a similar constructor that accepts a nondefault comparator for the priority queue.

```

/** Creates a priority queue initialized with the given key-value pairs. */
public HeapPriorityQueue(K[] keys, V[] values) {
    super();
    for (int j=0; j < Math.min(keys.length, values.length); j++)
        heap.add(new PQEntry<>(keys[j], values[j]));
    heapify();
}

/** Performs a bottom-up construction of the heap in linear time. */
protected void heapify() {
    int startIndex = parent(size()-1);           // start at PARENT of last entry
    for (int j=startIndex; j >= 0; j--)           // loop until processing the root
        downheap(j);
}

```

Code Fragment 9.10: Revision to the HeapPriorityQueue class of Code Fragments 9.8 and 9.9, supporting linear-time construction given an initial collection of key-value pairs.

Asymptotic Analysis of Bottom-Up Heap Construction

Bottom-up heap construction is asymptotically faster than incrementally inserting n entries into an initially empty heap. Intuitively, we are performing a single down-heap operation at each position in the tree, rather than a single up-heap operation from each. Since more nodes are closer to the bottom of a tree than the top, the sum of the downward paths is linear, as shown in the following proposition.

Proposition 9.3: *Bottom-up construction of a heap with n entries takes $O(n)$ time, assuming two keys can be compared in $O(1)$ time.*

Justification: The primary cost of the construction is due to the down-heap steps performed at each nonleaf position. Let π_v denote the path of T from nonleaf node v to its “inorder successor” leaf, that is, the path that starts at v , goes to the right child of v , and then goes down leftward until it reaches a leaf. Although, π_v is not necessarily the path followed by the down-heap bubbling step from v , its number of edges $\|\pi_v\|$ is proportional to the height of the subtree rooted at v , and thus a bound on the complexity of the down-heap operation at v . The total running time of the bottom-up heap construction algorithm is therefore bounded by the sum $\sum_v \|\pi_v\|$. For intuition, Figure 9.6 illustrates the justification “visually,” marking each edge with the label of the nonleaf node v whose path π_v contains that edge.

We claim that the paths π_v for all nonleaf v are edge-disjoint, and thus the sum of the path lengths is bounded by the number of total edges in the tree, hence $O(n)$. To show this, we consider what we term “right-leaning” and “left-leaning” edges (i.e., those going from a parent to a right, respectively left, child). A particular right-leaning edge e can only be part of the path π_v for node v that is the parent in the relationship represented by e . Left-leaning edges can be partitioned by considering the leaf that is reached if continuing down leftward until reaching a leaf. Each nonleaf node only uses left-leaning edges in the group leading to that nonleaf node’s inorder successor. Since each nonleaf node must have a different inorder successor, no two such paths can contain the same left-leaning edge. We conclude that the bottom-up construction of heap T takes $O(n)$ time. ■

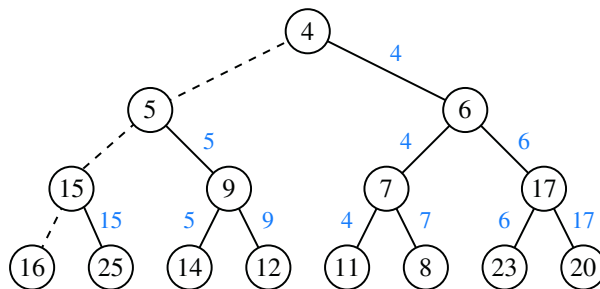


Figure 9.6: Visual justification of the linear running time of bottom-up heap construction. Each edge e is labeled with a node v for which π_v contains e (if any).

9.3.5 Using the `java.util.PriorityQueue` Class

There is no priority queue interface built into Java, but Java does include a class, `java.util.PriorityQueue`, which implements the `java.util.Queue` interface. Instead of adding and removing elements according to the standard FIFO policy used by most queues, the `java.util.PriorityQueue` class processes its entries according to a priority. The “front” of the queue will always be a minimal element, with priorities based either on the natural ordering of the elements, or in accordance with a comparator object sent as a parameter when constructing the priority queue.

The most notable difference between the `java.util.PriorityQueue` class and our own priority queue ADT is the model for managing keys and values. Whereas our public interface distinguishes between keys and values, the `java.util.PriorityQueue` class relies on a single element type. That element is effectively treated as a key.

If a user wishes to insert distinct keys and values, the burden is on the user to define and insert appropriate composite objects, and to ensure that those objects can be compared based on their keys. (The Java Collections Framework does include its own entry interface, `java.util.Map.Entry`, and a concrete implementation in the `java.util.AbstractMap.SimpleEntry` class; we discuss the map ADT in the next chapter.)

Table 9.4 shows the correspondance between methods of our priority queue ADT and those of the `java.util.PriorityQueue` class. The `java.util.PriorityQueue` class is implemented with a heap, so it guarantees $O(\log n)$ -time performance for methods `add` and `remove`, and constant-time performance for accessors `peek`, `size`, and `isEmpty`. In addition, it provides a parameterized method, `remove(e)`, that removes a specific element e from the priority queue. However, that method runs in $O(n)$ time, performing a sequential search to locate the element within the heap. (In Section 9.5, we extend our heap-based priority queue implementation to support a more efficient means for removing an arbitrary entry, or for updating the priority of an existing entry.)

Our Priority Queue ADT	<code>java.util.PriorityQueue</code> Class
<code>insert(k, v)</code>	<code>add(new SimpleEntry(k, v))</code>
<code>min()</code>	<code>peek()</code>
<code>removeMin()</code>	<code>remove()</code>
<code>size()</code>	<code>size()</code>
<code>isEmpty()</code>	<code>isEmpty()</code>

Table 9.4: Methods of our priority queue ADT and the corresponding methods when using the `java.util.PriorityQueue` class.

9.4 Sorting with a Priority Queue

One application of priority queues is sorting, where we are given a sequence of elements that can be compared according to a total order relation, and we want to rearrange them in increasing order (or at least in nondecreasing order if there are ties). The algorithm for sorting a sequence S with a priority queue P is quite simple and consists of the following two phases:

1. In the first phase, we insert the elements of S as keys into an initially empty priority queue P by means of a series of n insert operations, one for each element.
2. In the second phase, we extract the elements from P in nondecreasing order by means of a series of n removeMin operations, putting them back into S in that order.

A Java implementation of this algorithm is given in Code Fragment 9.11, assuming that the sequence is stored as a positional list. (Code for a different type of collection, such as an array or an array list, would be similar.)

The algorithm works correctly for any priority queue P , no matter how P is implemented. However, the running time of the algorithm is determined by the running times of operations insert and removeMin, which do depend on how P is implemented. Indeed, pqSort should be considered more a sorting “scheme” than a sorting “algorithm,” because it does not specify how the priority queue P is implemented. The pqSort scheme is the paradigm of several popular sorting algorithms, including selection-sort, insertion-sort, and heap-sort, which we will discuss in this section.

```

1  /** Sorts sequence S, using initially empty priority queue P to produce the order. */
2  public static <E> void pqSort(PositionalList<E> S, PriorityQueue<E,?> P) {
3      int n = S.size();
4      for (int j=0; j < n; j++) {
5          E element = S.remove(S.first());
6          P.insert(element, null);          // element is key; null value
7      }
8      for (int j=0; j < n; j++) {
9          E element = P.removeMin().getKey();
10         S.addLast(element);              // the smallest key in P is next placed in S
11     }
12 }
```

Code Fragment 9.11: An implementation of a pqSort method that sorts elements of a positional list using an initially empty priority queue to produce the ordering.

9.4.1 Selection-Sort and Insertion-Sort

We next demonstrate how the pqSort scheme results in two classic sorting algorithms when using an unsorted or sorted list for a priority queue.

Selection-Sort

In Phase 1 of the pqSort scheme, we insert all elements into a priority queue P ; in Phase 2 we repeatedly remove the minimal element from P using the `removeMin` method. If we implement P with an unsorted list, then Phase 1 of pqSort takes $O(n)$ time, for we can insert each element in $O(1)$ time. In Phase 2, the running time of each `removeMin` operation is proportional to the size of P . Thus, the bottleneck computation is the repeated “selection” of the minimum element in Phase 2. For this reason, this algorithm is better known as *selection-sort*. (See Figure 9.7.)

	<i>Sequence S</i>	<i>Priority Queue P</i>
Input	(7, 4, 8, 2, 5, 3, 9)	()
Phase 1 (a)	(4, 8, 2, 5, 3, 9)	(7)
(b)	(8, 2, 5, 3, 9)	(7, 4)
⋮	⋮	⋮
(g)	()	(7, 4, 8, 2, 5, 3, 9)
Phase 2 (a)	(2)	(7, 4, 8, 5, 3, 9)
(b)	(2, 3)	(7, 4, 8, 5, 9)
(c)	(2, 3, 4)	(7, 8, 5, 9)
(d)	(2, 3, 4, 5)	(7, 8, 9)
(e)	(2, 3, 4, 5, 7)	(8, 9)
(f)	(2, 3, 4, 5, 7, 8)	(9)
(g)	(2, 3, 4, 5, 7, 8, 9)	()

Figure 9.7: Execution of selection-sort on sequence $S = (7, 4, 8, 2, 5, 3, 9)$.

As noted above, the bottleneck is in Phase 2 where we repeatedly remove an entry with smallest key from the priority queue P . The size of P starts at n and incrementally decreases with each `removeMin` until it becomes 0. Thus, the first `removeMin` operation takes time $O(n)$, the second one takes time $O(n-1)$, and so on, until the last (n^{th}) operation takes time $O(1)$. Therefore, the total time needed for the second phase is

$$O(n + (n-1) + \cdots + 2 + 1) = O\left(\sum_{i=1}^n i\right).$$

By Proposition 4.3, we have $\sum_{i=1}^n i = n(n+1)/2$. Thus, Phase 2 takes time $O(n^2)$, as does the entire selection-sort algorithm.

Insertion-Sort

If we implement the priority queue P using a sorted list, then the running time of Phase 2 improves to $O(n)$, for each operation `removeMin` on P now takes $O(1)$ time. Unfortunately, Phase 1 now becomes the bottleneck for the running time, since, in the worst case, each insert operation takes time proportional to the size of P . This sorting algorithm is therefore better known as **insertion-sort** (see Figure 9.8), for the bottleneck in this sorting algorithm involves the repeated “insertion” of a new element at the appropriate position in a sorted list.

	<i>Sequence S</i>	<i>Priority Queue P</i>
Input	(7, 4, 8, 2, 5, 3, 9)	()
Phase 1 (a)	(4, 8, 2, 5, 3, 9)	(7)
(b)	(8, 2, 5, 3, 9)	(4, 7)
(c)	(2, 5, 3, 9)	(4, 7, 8)
(d)	(5, 3, 9)	(2, 4, 7, 8)
(e)	(3, 9)	(2, 4, 5, 7, 8)
(f)	(9)	(2, 3, 4, 5, 7, 8)
(g)	()	(2, 3, 4, 5, 7, 8, 9)
Phase 2 (a)	(2)	(3, 4, 5, 7, 8, 9)
(b)	(2, 3)	(4, 5, 7, 8, 9)
⋮	⋮	⋮
(g)	(2, 3, 4, 5, 7, 8, 9)	()

Figure 9.8: Execution of insertion-sort on sequence $S = (7, 4, 8, 2, 5, 3, 9)$. In Phase 1, we repeatedly remove the first element of S and insert it into P . In Phase 2, we repeatedly perform the `removeMin` operation on P and add the returned element to the end of S .

Analyzing the running time of Phase 1 of insertion-sort, we note that it is

$$O(1 + 2 + \dots + (n-1) + n) = O\left(\sum_{i=1}^n i\right).$$

Again, by recalling Proposition 4.3, Phase 1 runs in $O(n^2)$ time, and hence, so does the entire insertion-sort algorithm.

Alternatively, we could change our definition of insertion-sort so that we insert elements starting from the end of the priority-queue list in Phase 1, in which case performing insertion-sort on a sequence that is already sorted would run in $O(n)$ time. Indeed, the running time of insertion-sort in this case is $O(n + I)$, where I is the number of **inversions** in the sequence, that is, the number of pairs of elements that start out in the input sequence in the wrong relative order.

9.4.2 Heap-Sort

As we have previously observed, realizing a priority queue with a heap has the advantage that all the methods in the priority queue ADT run in logarithmic time or better. Hence, this realization is suitable for applications where fast running times are sought for all the priority queue methods. Therefore, let us again consider the pqSort scheme, this time using a heap-based implementation of the priority queue.

During Phase 1, the i^{th} insert operation takes $O(\log i)$ time, since the heap has i entries after the operation is performed. Therefore, this phase takes $O(n \log n)$ time. (It could be improved to $O(n)$ with the bottom-up heap construction described in Section 9.3.4.)

During the second phase of method pqSort, the j^{th} removeMin operation runs in $O(\log(n - j + 1))$, since the heap has $n - j + 1$ entries at the time the operation is performed. Summing over all j , this phase takes $O(n \log n)$ time, so the entire priority-queue sorting algorithm runs in $O(n \log n)$ time when we use a heap to implement the priority queue. This sorting algorithm is better known as **heap-sort**, and its performance is summarized in the following proposition.

Proposition 9.4: *The heap-sort algorithm sorts a sequence S of n elements in $O(n \log n)$ time, assuming two elements of S can be compared in $O(1)$ time.*

Let us stress that the $O(n \log n)$ running time of heap-sort is considerably better than the $O(n^2)$ running time of selection-sort and insertion-sort.

Implementing Heap-Sort In-Place

If the sequence S to be sorted is implemented by means of an array-based sequence, such as an ArrayList in Java, we can speed up heap-sort and reduce its space requirement by a constant factor by using a portion of the array itself to store the heap, thus avoiding the use of an auxiliary heap data structure. This is accomplished by modifying the algorithm as follows:

1. We redefine the heap operations to be a *maximum-oriented* heap, with each position key being at least as *large* as its children. This can be done by re-coding the algorithm, or by providing a new comparator that reverses the outcome of each comparison. At any time during the execution of the algorithm, we use the left portion of S , up to a certain index $i - 1$, to store the entries of the heap, and the right portion of S , from index i to $n - 1$, to store the elements of the sequence. Thus, the first i elements of S (at indices $0, \dots, i - 1$) provide the array-list representation of the heap.
2. In the first phase of the algorithm, we start with an empty heap and move the boundary between the heap and the sequence from left to right, one step at a time. In step i , for $i = 1, \dots, n$, we expand the heap by adding the element at index $i - 1$.

3. In the second phase of the algorithm, we start with an empty sequence and move the boundary between the heap and the sequence from right to left, one step at a time. At step i , for $i = 1, \dots, n$, we remove a maximal element from the heap and store it at index $n - i$.

In general, we say that a sorting algorithm is *in-place* if it uses only a small amount of memory in addition to the sequence storing the objects to be sorted. The variation of heap-sort above qualifies as in-place; instead of transferring elements out of the sequence and then back in, we simply rearrange them. We illustrate the second phase of in-place heap-sort in Figure 9.9.

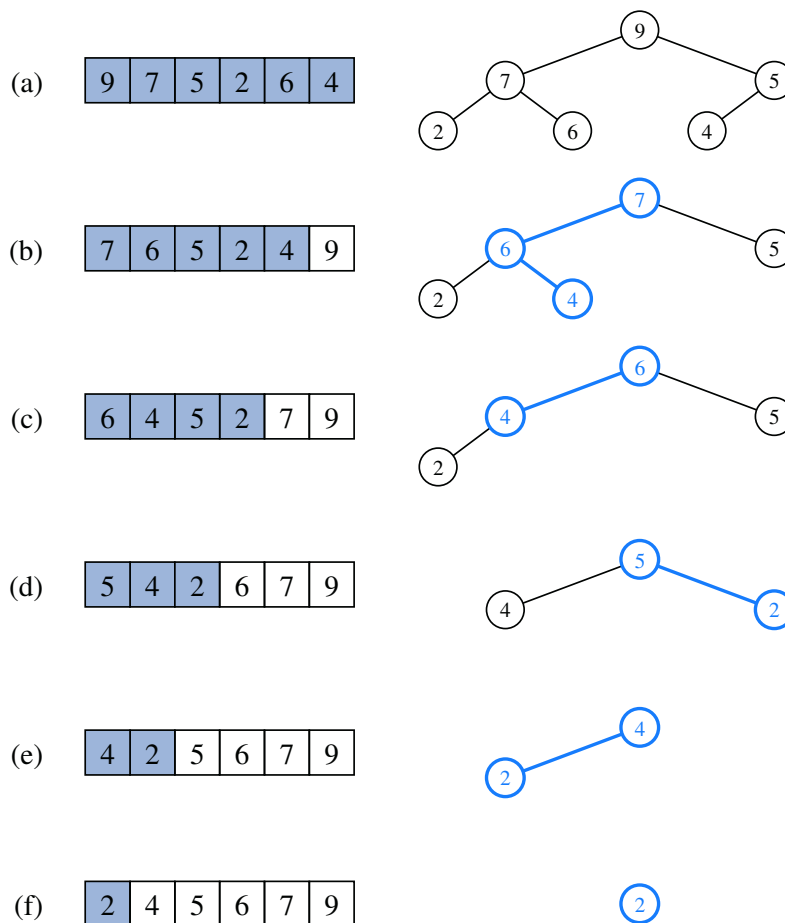


Figure 9.9: Phase 2 of an in-place heap-sort. The heap portion of each sequence representation is highlighted. The binary tree that each sequence (implicitly) represents is diagrammed with the most recent path of down-heap bubbling highlighted.

9.5 Adaptable Priority Queues

The methods of the priority queue ADT given in Section 9.1.2 are sufficient for most basic applications of priority queues, such as sorting. However, there are situations in which additional methods would be useful, as shown by the scenarios below involving the standby airline passenger application.

- A standby passenger with a pessimistic attitude may become tired of waiting and decide to leave ahead of the boarding time, requesting to be removed from the waiting list. Thus, we would like to remove from the priority queue the entry associated with this passenger. Operation `removeMin` does not suffice since the passenger leaving does not necessarily have first priority. Instead, we want a new operation, `remove`, that removes an arbitrary entry.
- Another standby passenger finds her gold frequent-flyer card and shows it to the agent. Thus, her priority has to be modified accordingly. To achieve this change of priority, we would like to have a new operation `replaceKey` allowing us to replace the key of an existing entry with a new key.
- Finally, a third standby passenger notices her name is misspelled on the ticket and asks it to be corrected. To perform this change, we need to update the passenger's record. Hence, we would like to have a new operation `replaceValue`, allowing us to replace the value of an existing entry with a new value.

The Adaptable Priority Queue ADT

The above scenarios motivate the definition of a new *adaptable priority queue* ADT that extends the priority queue ADT with additional functionality. We will see another application of adaptable priority queues when implementing certain graph algorithms in Sections 14.6.2 and 14.7.1.

In order to implement methods `remove`, `replaceKey`, and `replaceValue` efficiently, we need a mechanism for finding a user's element within a priority queue, ideally in a way that avoids performing a linear search through the entire collection. In the original definition of the priority queue ADT, a call to `insert( $k, v$ )` formally returns an instance of type `Entry` to the user. In order to be able to update or remove an entry in our new adaptable priority queue ADT, the user must retain that `Entry` object as a token that can be sent back as a parameter to identify the relevant entry. Formally, the adaptable priority queue ADT includes the following methods (in addition to those of the standard priority queue):

`remove( $e$ )`: Removes entry e from the priority queue.

`replaceKey( $e, k$ )`: Replaces the key of existing entry e with k .

`replaceValue( $e, v$ )`: Replaces the value of existing entry e with v .

An error occurs with each of these methods if parameter e is invalid (for example, because it had previously been removed from the priority queue).

9.5.1 Location-Aware Entries

To allow an entry instance to encode a location within a priority queue, we extend the `PQEntry` class (originally defined with the `AbstractPriorityQueue` base class), adding a third field that designates the current index of an entry within the array-based representation of the heap, as shown in Figure 9.10. (This approach is similar to our recommendation, on page 281, for implementing the positional list abstraction with an array.)

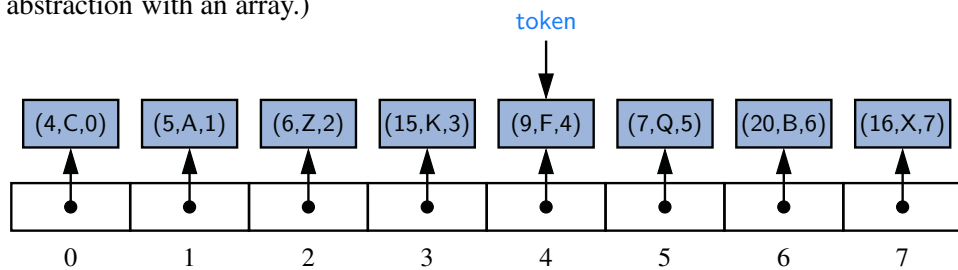


Figure 9.10: Representing a heap using an array of location-aware entries. The third field of each entry instance corresponds to the index of that entry within the array. Identifier token is presumed to be an entry reference in the user’s scope.

When we perform priority queue operations on our heap, causing entries to be relocated within our structure, we must make sure to update the third field of each affected entry to reflect its new index within the array. As an example, Figure 9.11 shows the state of the above heap after a call to `removeMin()`. The heap operation causes the minimal entry, (4,C), to be removed, and the last entry, (16,X), to be temporarily moved from the last position to the root, followed by a down-heap bubble phase. During the down-heap, element (16,X) is swapped with its left child, (5,A), at index 1 of the list, then swapped with its right child, (9,F), at index 4 of the list. In the final configuration, the last field for all affected entries has been modified to reflect their new location.

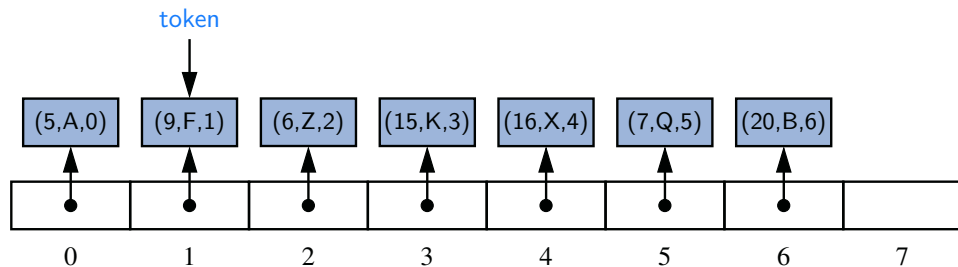


Figure 9.11: The result of a call to `removeMin()` on the heap originally portrayed in Figure 9.10. Identifier token continues to reference the same entry as in the original configuration, but the placement of that entry in the array has changed, as has the third field of the entry.

9.5.2 Implementing an Adaptable Priority Queue

Code Fragments 9.12 and 9.13 present a Java implementation of an adaptable priority queue, as a subclass of the `HeapPriorityQueue` class from Section 9.3.2. We begin by defining a nested `AdaptablePQEntry` class (lines 5–15) that extends the inherited `PQEntry` class, augmenting it with an additional index field. The inherited `insert` method is overridden, so that we create and initialize an instance of the `AdaptablePQEntry` class (not the original `PQEntry` class).

An important aspect of our design is that the original `HeapPriorityQueue` class relies exclusively on a protected `swap` method for all low-level data movement during up-heap or down-heap operations. The `AdaptablePriorityQueue` class overrides that utility in order to update the stored indices of our location-aware entries when they are relocated (as discussed on the previous page).

When an entry is sent as a parameter to `remove`, `replaceKey`, or `replaceValue`, we rely on the new index field of that entry to designate where the element resides in the heap (a fact that is easily validated). When a key of an existing entry is replaced, that new key may violate the heap-order property by being either too big or too small. We provide a new bubble utility that determines whether an up-heap or down-heap bubbling step is warranted. When removing an arbitrary entry, we replace it with the last entry in the heap (to maintain the complete binary tree property) and perform the bubbling step, since the displaced element may have a key that is too large or too small for its new location.

Performance of Adaptable Priority Queue Implementations

The performance of an adaptable priority queue by means of our location-aware heap structure is summarized in Table 9.5. The new class provides the same asymptotic efficiency and space usage as the nonadaptive version, and provides logarithmic performance for the new locator-based `remove` and `replaceKey` methods, and constant-time performance for the new `replaceValue` method.

Method	Running Time
size, isEmpty, min	$O(1)$
insert	$O(\log n)$
remove	$O(\log n)$
removeMin	$O(\log n)$
replaceKey	$O(\log n)$
replaceValue	$O(1)$

Table 9.5: Running times of the methods of an adaptable priority queue with size n , realized by means of our array-based heap representation. The space requirement is $O(n)$.

```

1  /** An implementation of an adaptable priority queue using an array-based heap. */
2  public class HeapAdaptablePriorityQueue<K,V> extends HeapPriorityQueue<K,V>
3              implements AdaptablePriorityQueue<K,V> {
4
5      //----- nested AdaptablePQEntry class -----
6      /** Extension of the PQEntry to include location information. */
7      protected static class AdaptablePQEntry<K,V> extends PQEntry<K,V> {
8          private int index; // entry's current index within the heap
9          public AdaptablePQEntry(K key, V value, int j) {
10             super(key, value); // this sets the key and value
11             index = j; // this sets the new field
12         }
13         public int getIndex() { return index; }
14         public void setIndex(int j) { index = j; }
15     } //----- end of nested AdaptablePQEntry class -----
16
17     /** Creates an empty adaptable priority queue using natural ordering of keys. */
18     public HeapAdaptablePriorityQueue() { super(); }
19     /** Creates an empty adaptable priority queue using the given comparator. */
20     public HeapAdaptablePriorityQueue(Comparator<K> comp) { super(comp); }
21
22     // protected utilities
23     /** Validates an entry to ensure it is location-aware. */
24     protected AdaptablePQEntry<K,V> validate(Entry<K,V> entry)
25         throws IllegalArgumentException {
26         if (!(entry instanceof AdaptablePQEntry))
27             throw new IllegalArgumentException("Invalid entry");
28         AdaptablePQEntry<K,V> locator = (AdaptablePQEntry<K,V>) entry; // safe
29         int j = locator.getIndex();
30         if (j >= heap.size() || heap.get(j) != locator)
31             throw new IllegalArgumentException("Invalid entry");
32         return locator;
33     }
34
35     /** Exchanges the entries at indices i and j of the array list. */
36     protected void swap(int i, int j) {
37         super.swap(i,j); // perform the swap
38         ((AdaptablePQEntry<K,V>) heap.get(i)).setIndex(i); // reset entry's index
39         ((AdaptablePQEntry<K,V>) heap.get(j)).setIndex(j); // reset entry's index
40     }

```

Code Fragment 9.12: An implementation of an adaptable priority queue. (Continues in Code Fragment 9.13.) This extends the `HeapPriorityQueue` class of Code Fragments 9.8 and 9.9.

```

41  /** Restores the heap property by moving the entry at index j upward/downward.*/
42  protected void bubble(int j) {
43      if (j > 0 && compare(heap.get(j), heap.get(parent(j))) < 0)
44          upheap(j);
45      else
46          downheap(j);                      // although it might not need to move
47  }
48
49  /** Inserts a key-value pair and returns the entry created. */
50  public Entry<K,V> insert(K key, V value) throws IllegalArgumentException {
51      checkKey(key);                        // might throw an exception
52      Entry<K,V> newest = new AdaptablePQEntry<>(key, value, heap.size());
53      heap.add(newest);                     // add to the end of the list
54      upheap(heap.size() - 1);             // upheap newly added entry
55      return newest;
56  }
57
58  /** Removes the given entry from the priority queue. */
59  public void remove(Entry<K,V> entry) throws IllegalArgumentException {
60      AdaptablePQEntry<K,V> locator = validate(entry);
61      int j = locator.getIndex();
62      if (j == heap.size() - 1)              // entry is at last position
63          heap.remove(heap.size() - 1);     // so just remove it
64      else {
65          swap(j, heap.size() - 1);         // swap entry to last position
66          heap.remove(heap.size() - 1);     // then remove it
67          bubble(j);                       // and fix entry displaced by the swap
68      }
69  }
70
71  /** Replaces the key of an entry. */
72  public void replaceKey(Entry<K,V> entry, K key)
73      throws IllegalArgumentException {
74      AdaptablePQEntry<K,V> locator = validate(entry);
75      checkKey(key);                       // might throw an exception
76      locator.setKey(key);                  // method inherited from PQEntry
77      bubble(locator.getIndex());          // with new key, may need to move entry
78  }
79
80  /** Replaces the value of an entry. */
81  public void replaceValue(Entry<K,V> entry, V value)
82      throws IllegalArgumentException {
83      AdaptablePQEntry<K,V> locator = validate(entry);
84      locator.setValue(value);              // method inherited from PQEntry
85  }

```

Code Fragment 9.13: An implementation of an adaptable priority queue (continued from Code Fragment 9.12).

9.6 Exercises

Reinforcement

- R-9.1 How long would it take to remove the $\lceil \log n \rceil$ smallest elements from a heap that contains n entries, using the `removeMin` operation?
- R-9.2 Suppose you set the key for each position p of a binary tree T equal to its preorder rank. Under what circumstances is T a heap?
- R-9.3 What does each `removeMin` call return within the following sequence of priority queue ADT operations: `insert(5, A)`, `insert(4, B)`, `insert(7, F)`, `insert(1, D)`, `removeMin()`, `insert(3, J)`, `insert(6, L)`, `removeMin()`, `removeMin()`, `insert(8, G)`, `removeMin()`, `insert(2, H)`, `removeMin()`, `removeMin()`?
- R-9.4 An airport is developing a computer simulation of air-traffic control that handles events such as landings and takeoffs. Each event has a *time stamp* that denotes the time when the event will occur. The simulation program needs to efficiently perform the following two fundamental operations:
- Insert an event with a given time stamp (that is, add a future event).
 - Extract the event with smallest time stamp (that is, determine the next event to process).
- Which data structure should be used for the above operations? Why?
- R-9.5 The `min` method for the `UnsortedPriorityQueue` class executes in $O(n)$ time, as analyzed in Table 9.2. Give a simple modification to the class so that `min` runs in $O(1)$ time. Explain any necessary modifications to other methods of the class.
- R-9.6 Can you adapt your solution to the previous problem to make `removeMin` run in $O(1)$ time for the `UnsortedPriorityQueue` class? Explain your answer.
- R-9.7 Illustrate the execution of the selection-sort algorithm on the following input sequence: (22, 15, 36, 44, 10, 3, 9, 13, 29, 25).
- R-9.8 Illustrate the execution of the insertion-sort algorithm on the input sequence of the previous problem.
- R-9.9 Give an example of a worst-case sequence with n elements for insertion-sort, and show that insertion-sort runs in $\Omega(n^2)$ time on such a sequence.
- R-9.10 At which positions of a heap might the third smallest key be stored?
- R-9.11 At which positions of a heap might the largest key be stored?
- R-9.12 Consider a situation in which a user has numeric keys and wishes to have a priority queue that is *maximum-oriented*. How could a standard (min-oriented) priority queue be used for such a purpose?
- R-9.13 Illustrate the execution of the in-place heap-sort algorithm on the following input sequence: (2, 5, 16, 4, 10, 23, 39, 18, 26, 15).

- R-9.14 Let T be a complete binary tree such that position p stores an element with key $f(p)$, where $f(p)$ is the level number of p (see Section 8.3.2). Is tree T a heap? Why or why not?
- R-9.15 Explain why the description of down-heap bubbling does not consider the case in which position p has a right child but not a left child.
- R-9.16 Is there a heap H storing seven entries with distinct keys such that a preorder traversal of H yields the entries of H in increasing or decreasing order by key? How about an inorder traversal? How about a postorder traversal? If so, give an example; if not, say why.
- R-9.17 Let H be a heap storing 15 entries using the array-based representation of a complete binary tree. What is the sequence of indices of the array that are visited in a preorder traversal of H ? What about an inorder traversal of H ? What about a postorder traversal of H ?
- R-9.18 Show that the sum $\sum_{i=1}^n \log i$, appearing in the analysis of heap-sort, is $\Omega(n \log n)$.
- R-9.19 Bill claims that a preorder traversal of a heap will list its keys in nondecreasing order. Draw an example of a heap that proves him wrong.
- R-9.20 Hillary claims that a postorder traversal of a heap will list its keys in nonincreasing order. Draw an example of a heap that proves her wrong.
- R-9.21 Illustrate all the steps of the adaptable priority queue call `remove( $e$ )` for entry e storing (16, X) in the heap of Figure 9.1.
- R-9.22 Illustrate all the steps of the adaptable priority queue call `replaceKey( $e$ , 18)` for entry e storing (5, A) in the heap of Figure 9.1.
- R-9.23 Draw an example of a heap whose keys are all the odd numbers from 1 to 59 (with no repeats), such that the insertion of an entry with key 32 would cause up-heap bubbling to proceed all the way up to a child of the root (replacing that child's key with 32).
- R-9.24 Describe a sequence of n insertions in a heap that requires $\Omega(n \log n)$ time to process.

Creativity

- C-9.25 Show how to implement the stack ADT using only a priority queue and one additional integer instance variable.
- C-9.26 Show how to implement the FIFO queue ADT using only a priority queue and one additional integer instance variable.
- C-9.27 Professor Idle suggests the following solution to the previous problem. Whenever an entry is inserted into the queue, it is assigned a key that is equal to the current size of the queue. Does such a strategy result in FIFO semantics? Prove that it is so or provide a counterexample.

- C-9.28** Reimplement the SortedPriorityQueue using a Java array. Make sure to maintain removeMin's $O(1)$ performance.
- C-9.29** Give an alternative implementation of the HeapPriorityQueue's upheap method that uses recursion (and no loop).
- C-9.30** Give an implementation of the HeapPriorityQueue's downheap method that uses recursion (and no loop).
- C-9.31** Assume that we are using a linked representation of a complete binary tree T , and an extra reference to the last node of that tree. Show how to update the reference to the last node after operations insert or remove in $O(\log n)$ time, where n is the current number of nodes of T . Be sure to handle all possible cases, as illustrated in Figure 9.12.
- C-9.32** When using a linked-tree representation for a heap, an alternative method for finding the last node during an insertion in a heap T is to store, in the last node and each leaf node of T , a reference to the leaf node immediately to its right (wrapping to the first node in the next lower level for the rightmost leaf node). Show how to maintain such references in $O(1)$ time per operation of the priority queue ADT assuming that T is implemented with a linked structure.
- C-9.33** We can represent a path from the root to a given node of a binary tree by means of a binary string, where 0 means “go to the left child” and 1 means “go to the right child.” For example, the path from the root to the node storing $(8, W)$ in the heap of Figure 9.12a is represented by “101.” Design an $O(\log n)$ -time algorithm for finding the last node of a complete binary tree with n nodes, based on the above representation. Show how this algorithm can be used in the implementation of a complete binary tree by means of a linked structure that does not keep an explicit reference to the last node instance.
- C-9.34** Given a heap H and a key k , give an algorithm to compute all the entries in H having a key less than or equal to k . For example, given the heap of Figure 9.12a and query $k = 7$, the algorithm should report the entries with keys 2, 4, 5, 6, and 7 (but not necessarily in this order). Your algorithm should run in time proportional to the number of entries returned, and should *not* modify the heap.

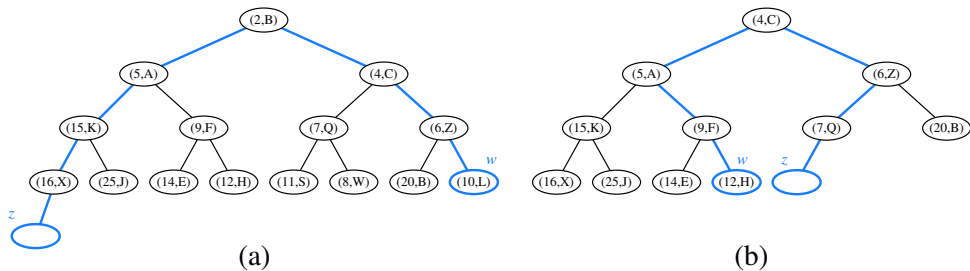


Figure 9.12: Two cases of updating the last node in a complete binary tree after operation insert or remove. Node w is the last node before operation insert or after operation remove. Node z is the last node after operation insert or before operation remove.

- C-9.35 Provide a justification of the time bounds in Table 9.5.
- C-9.36 Give an alternative analysis of bottom-up heap construction by showing the following summation is $O(1)$, for any positive integer h :

$$\sum_{i=1}^h (i/2^i).$$

- C-9.37 Suppose two binary trees, T_1 and T_2 , hold entries satisfying the heap-order property (but not necessarily the complete binary tree property). Describe a method for combining T_1 and T_2 into a binary tree T , whose nodes hold the union of the entries in T_1 and T_2 and also satisfy the heap-order property. Your algorithm should run in time $O(h_1 + h_2)$ where h_1 and h_2 are the respective heights of T_1 and T_2 .
- C-9.38 Tamarindo Airlines wants to give a first-class upgrade coupon to their top $\log n$ frequent flyers, based on the number of miles accumulated, where n is the total number of the airlines' frequent flyers. The algorithm they currently use, which runs in $O(n \log n)$ time, sorts the flyers by the number of miles flown and then scans the sorted list to pick the top $\log n$ flyers. Describe an algorithm that identifies the top $\log n$ flyers in $O(n)$ time.
- C-9.39 Explain how the k largest elements from an unordered collection of size n can be found in time $O(n + k \log n)$ using a maximum-oriented heap.
- C-9.40 Explain how the k largest elements from an unordered collection of size n can be found in time $O(n \log k)$ using $O(k)$ auxiliary space.
- C-9.41 Write a comparator for nonnegative integers that determines order based on the number of 1's in each integer's binary expansion, so that $i < j$ if the number of 1's in the binary representation of i is less than the number of 1's in the binary representation of j .
- C-9.42 Implement the `binarySearch` algorithm (see Section 5.1.3) using a `Comparator` for an array with elements of generic type `E`.
- C-9.43 Given a class, `MinPriorityQueue`, that implements the minimum-oriented priority queue ADT, provide an implementation of a `MaxPriorityQueue` class that adapts to provide a maximum-oriented abstraction with methods `insert`, `max`, and `removeMax`. Your implementation should not make any assumption about the internal workings of the original `MinPriorityQueue` class, nor the type of keys that might be used.
- C-9.44 Describe an in-place version of the selection-sort algorithm for an array that uses only $O(1)$ space for instance variables in addition to the array.
- C-9.45 Assuming the input to the sorting problem is given in an array A , describe how to implement the insertion-sort algorithm using only the array A and at most six additional (base-type) variables.
- C-9.46 Give an alternate description of the in-place heap-sort algorithm using the standard minimum-oriented priority queue (instead of a maximum-oriented one).

- C-9.47 A group of children want to play a game, called *Unmonopoly*, where in each turn the player with the most money must give half of his/her money to the player with the least amount of money. What data structure(s) should be used to play this game efficiently? Why?
- C-9.48 An online computer system for trading stocks needs to process orders of the form “buy 100 shares at \$ x each” or “sell 100 shares at \$ y each.” A buy order for \$ x can only be processed if there is an existing sell order with price \$ y such that $y \leq x$. Likewise, a sell order for \$ y can only be processed if there is an existing buy order with price \$ x such that $y \leq x$. If a buy or sell order is entered but cannot be processed, it must wait for a future order that allows it to be processed. Describe a scheme that allows buy and sell orders to be entered in $O(\log n)$ time, independent of whether or not they can be immediately processed.
- C-9.49 Extend a solution to the previous problem so that users are allowed to update the prices for their buy or sell orders that have yet to be processed.

Projects

- P-9.50 Implement the in-place heap-sort algorithm. Experimentally compare its running time with that of the standard heap-sort that is not in-place.
- P-9.51 Use the approach of either Exercise C-9.39 or C-9.40 to reimplement the method `getFavorites` of the `FavoritesListMTF` class from Section 7.7.2. Make sure that results are generated from largest to smallest.
- P-9.52 Develop a Java implementation of an adaptable priority queue that is based on an unsorted list and supports location-aware entries.
- P-9.53 Write an applet or stand-alone graphical program that animates a heap. Your program should support all the priority queue operations and should visualize the swaps in the up-heap and down-heap bubbings. (Extra: Visualize bottom-up heap construction as well.)
- P-9.54 Write a program that can process a sequence of stock buy and sell orders as described in Exercise C-9.48.
- P-9.55 One of the main applications of priority queues is in operating systems—for *scheduling jobs* on a CPU. In this project you are to build a program that schedules simulated CPU jobs. Your program should run in a loop, each iteration of which corresponds to a *time slice* for the CPU. Each job is assigned a priority, which is an integer between -20 (highest priority) and 19 (lowest priority), inclusive. From among all jobs waiting to be processed in a time slice, the CPU must work on a job with highest priority. In this simulation, each job will also come with a *length* value, which is an integer between 1 and 100 , inclusive, indicating the number of time slices that are needed to process this job. For simplicity, you may assume jobs cannot be interrupted—once it is scheduled on the CPU, a job runs for a number of time slices equal to its length. Your simulator must output the name of the job running on the CPU in each time slice and must process a sequence of commands, one per time slice, each of which is of the form “add job *name* with length *n* and priority *p*” or “no new job this slice”.

- P-9.56 Let S be a set of n points in the plane with distinct integer x - and y -coordinates. Let T be a complete binary tree storing the points from S at its external nodes, such that the points are ordered left to right by increasing x -coordinates. For each node v in T , let $S(v)$ denote the subset of S consisting of points stored in the subtree rooted at v . For the root r of T , define $top(r)$ to be the point in $S = S(r)$ with maximal y -coordinate. For every other node v , define $top(v)$ to be the point in S with highest y -coordinate in $S(v)$ that is not also the highest y -coordinate in $S(u)$, where u is the parent of v in T (if such a point exists). Such labeling turns T into a **priority search tree**. Describe a linear-time algorithm for turning T into a priority search tree. Implement this approach.

Chapter Notes

Knuth's book on sorting and searching [61] describes the motivation and history for the selection-sort, insertion-sort, and heap-sort algorithms. The heap-sort algorithm is due to Williams [95], and the linear-time heap construction algorithm is due to Floyd [35]. Additional algorithms and analyses for heaps and heap-sort variations can be found in papers by Bentley [14], Carlsson [21], Gonnet and Munro [39], McDiarmid and Reed [69], and Schaffer and Sedgwick [82].